

UNIX И LINUX

РУКОВОДСТВО СИСТЕМНОГО АДМИНИСТРАТОРА

ОГЛАВЛЕНИЕ

Часть I. Основы администрирования	41
Глава 1. С чего начать	43
Глава 2. Сценарии и командная оболочка	70
Глава 3. Запуск и останов системы	119
Глава 4. Управление доступом: сила привилегий	145
Глава 5. Управление процессами	163
Глава 6. Файловая система	184
Глава 7. Добавление новых пользователей	218
Глава 8. Дисковая память	252
Глава 9. Периодические процессы	330
Глава 10. Резервное копирование	339
Глава 11. Система Syslog и журнальные файлы	388
Глава 12. Управление программным обеспечением и конфигурацией	412
Глава 13. Драйверы и ядро	464
Часть II. Работа в сетях	493
Глава 14. Сети TCP/IP	495
Глава 15. Маршрутизация	558
Глава 16. Сетевые аппаратные средства	577
Глава 17. Система доменных имен	597
Глава 18. Сетевой протокол Network File System	735
Глава 19. Совместное использование системных файлов	764
Глава 20. Электронная почта	787
Глава 21. Управление сетями	907
Глава 22. Безопасность	943
Глава 23. Веб-хостинг	1001
Часть III. Разное	1025
Глава 24. Виртуализация	1027
Глава 25. Система X Window System	1055
Глава 26. Печать	1076
Глава 27. Центры обработки данных	1129
Глава 28. Экологичные информационные технологии	1141
Глава 29. Анализ производительности	1157
Глава 30. Взаимодействие с системой Windows	1180
Глава 31. Последовательные устройства и терминалы	1208
Глава 32. Управление, стратегия и политика	1228
Краткая история системного администрирования	1278
В защиту AIX	1288
Предметный указатель	1291

СОДЕРЖАНИЕ

Часть I. Основы администрирования	41
Глава 1. С чего начать	43
1.1. Основные задачи системного администратора	44
Инициализация пользователей	44
Подключение и удаление аппаратных средств	44
Резервное копирование	45
Инсталляция и обновление программ	45
Мониторинг системы	45
Поиск неисправностей	45
Ведение локальной документации	46
Слежение за безопасностью системы	46
Оказание помощи пользователям	46
1.2. Что необходимо знать	46
1.3. Взаимосвязь систем Linux и UNIX	48
1.4. Дистрибутивы Linux	49
1.5. Примеры систем, используемых в этой книге	51
Примеры Linux-дистрибутивов	51
Примеры UNIX-дистрибутивов	53
1.6. Административные средства конкретных систем	54
1.7. Типографские особенности книги	54
1.8. Единицы измерения	55
1.9. Использование справочных страниц и другой оперативной документации	56
Организация справочных страниц интерактивного руководства	56
Команда man: чтение страниц интерактивного руководства	57
Хранение страниц интерактивного руководства	58
GNU-система Texinfo	58
1.10. Другие виды официальной документации	59
Руководства по конкретным системам	59
Документация по конкретным пакетам	60
Книги	60
Документы RFC и другие интернет-ресурсы	60
Проект документации по Linux	60
1.11. Другие источники информации	61
1.12. Как устанавливать программное обеспечение	62
Определение факта инсталляции программного обеспечения	63
Добавление новых программ	64
Сборка программного обеспечения из исходного кода	66
1.13. Издержки профессии	67
1.14. Рекомендуемая литература	67
Системное администрирование	68

Основной инструментарий	68
1.15. Упражнения	69
Глава 2. Сценарии и командная оболочка	70
2.1. Основы работы с командной оболочкой	71
Редактирование команд	71
Каналы и перенаправление потоков	72
Использование переменных и кавычек	73
Команды общих фильтров	74
2.2. Написание bash-сценариев	78
От команд к сценариям	79
Организация ввода и вывода данных	81
Функции и аргументы командной строки	82
Область видимости переменных	84
Поток управления	84
Циклы	86
Массивы и арифметика	88
2.3. Регулярные выражения	89
Процесс сопоставления	90
Литеральные символы	91
Специальные символы	91
Примеры использования регулярных выражений	92
Захваты	93
Жадность, лень и катастрофический поиск с возвратом	94
2.4. Программирование на языке Perl	96
Переменные и массивы	97
Массив и строковые литералы	97
Вызовы функций	98
Преобразования типов в выражениях	98
Раскрытие строк и устранение неоднозначности при ссылках на переменные	99
Хеши	99
Ссылки и их самооживление	101
Регулярные выражения в Perl	101
Ввод и вывод данных	102
Поток управления	103
Прием входных данных и проверка их достоверности	105
Использование языка Perl в качестве фильтра	106
Модули расширения для Perl	107
2.5. Создание сценариев на языке PYTHON	108
Быстрое погружение в языковую среду Python	109
Объекты, строки, числа, списки, словари, кортежи и файлы	111
Пример контроля входных данных	112
Циклы	113
2.6. Передовой опыт создания сценариев	115
2.7. Рекомендуемая литература	117
Основы работы в командной оболочке и написание сценариев в среде bash	117
Регулярные выражения	117
Написание сценариев на языке Perl	117
Написание сценариев на языке Python	117
2.8. Упражнения	118

Глава 3. Запуск и останов системы	119
3.1. Начальная загрузка	119
Главное — активизировать командную оболочку	120
Этапы загрузки	120
Инициализация ядра	121
Конфигурирование аппаратных средств	121
Создание процессов ядра	121
Действия оператора (только в режиме восстановления)	122
Выполнение сценариев запуска системы	123
Завершение процесса загрузки	123
3.2. Загрузка системы на персональном компьютере	124
3.3. GRUB: универсальный загрузчик	125
Параметры ядра	126
Мультисистемная загрузка	127
3.4. Загрузка в однопользовательском режиме	128
Однопользовательский режим при использовании GRUB	128
Однопользовательский режим в архитектуре SPARC	128
Однопользовательский режим на рабочих станциях HP-UX	129
Однопользовательский режим в системах AIX	129
3.5. Работа со сценариями запуска системы	129
Демон init и его уровни выполнения	130
Обзор сценариев запуска	131
Сценарии запуска в системах Red Hat	133
Сценарии запуска в системах SUSE	135
Сценарии запуска Ubuntu и демон Upstart	136
Сценарии запуска в системах HP-UX	137
Запуск систем AIX	138
3.6. Загрузка систем SOLARIS	139
Механизм управления службами в системе Solaris	139
Прекрасный новый мир: загрузка с помощью механизма SMF	142
3.7. Перезагрузка и останов системы	142
Команда shutdown: корректный способ останова системы	143
Команды halt и reboot: более простой способ останова	144
3.8. Упражнения	144
Глава 4. Управление доступом: сила привилегий	145
4.1. Традиционные методы управления доступом в системах UNIX	146
Управление доступом в файловой системе	146
Владение процессом	147
Учетная запись суперпользователя	147
Использование битов “setuid” и “setgid”	148
4.2. Современная организация управления доступом	149
Управление доступом на основе ролей	150
SELinux: Linux-системы с улучшенной безопасностью	151
Возможности POSIX (Linux)	151
Подключаемые модули аутентификации	152
Сетевой протокол криптографической аутентификации Kerberos	152
Списки управления доступом	152
4.3. Управление доступом в реальном мире	153

Пароль суперпользователя	153
Регистрация под именем root	155
Команда su: замена идентификатора пользователя	155
Утилита sudo: ограниченный вариант команды su	156
Хранилища паролей и их депонирование	159
4.4. О псевдопользователях	160
4.5. Упражнения	161
Глава 5. Управление процессами	163
5.1. Атрибуты процесса	163
Идентификатор процесса (PID)	164
Идентификатор родительского процесса (PPID)	164
Идентификатор пользователя (UID) и текущий идентификатор пользователя (EUID)	165
Идентификатор группы (GID) и текущий идентификатор группы (EGID)	165
Приоритет и фактор уступчивости	166
Управляющий терминал	166
5.2. Жизненный цикл процесса	166
5.3. Сигналы	167
5.4. Отправка сигналов: команда kill	170
5.5. Состояния процесса	171
5.6. Изменение приоритета выполнения: команды nice и renice	172
5.7. Текущий контроль процессов: команда ps	173
5.8. Динамический мониторинг процессов с помощью команд top, prstat и topas	177
5.9. Файловая система /proc	178
5.10. Отслеживание сигналов и системных вызовов: команды strace, truss и tusc	179
5.11. Процессы, вышедшие из-под контроля	181
5.12. Рекомендуемая литература	182
5.13. Упражнения	182
Глава 6. Файловая система	184
6.1. Имена файлов и каталогов	186
Абсолютные и относительные пути	186
Использование пробелов в именах файлов	186
6.2. Монтирование и демонтирование файловой системы	187
6.3. Организация файловой системы	189
6.4. Типы файлов	192
Обычные файлы	193
Каталоги	193
Файлы символьных и блочных устройств	194
Локальные сокеты	195
Именованные каналы	195
Символические ссылки	195
6.5. Атрибуты файлов	196
Биты режима	196
Биты setuid и setgid	197
Дополнительный бит	198
Команда ls: просмотр атрибутов файла	198
Команда chmod: изменение прав доступа	200

Команды <code>chown</code> и <code>chgrp</code> : смена владельца и группы	201
Команда <code>umask</code> : задание стандартных прав доступа	202
Дополнительные флаги в системе Linux	202
6.6. Списки контроля доступа	203
Краткий обзор развития UNIX-списков ACL	204
Реализация списков ACL	205
Системная поддержка списков ACL	205
Обзор POSIX ACL	206
Списки NFSv4ACL	210
6.7. Упражнения	216
Глава 7. Добавление новых пользователей	218
7.1. Файл <code>/etc/passwd</code>	219
Регистрационное имя	220
Зашифрованный пароль	223
Идентификатор пользователя	224
Идентификатор группы по умолчанию	225
Поле GECOS	226
Домашний каталог	226
Регистрационная оболочка	226
7.2. Файлы <code>/etc/shadow</code> и <code>/etc/security/passwd</code>	227
7.3. Файл <code>/etc/group</code>	230
7.4. Подключение пользователей: основные действия	232
Редактирование файлов <code>passwd</code> и <code>group</code>	233
Задание пароля	233
Создание домашнего каталога пользователя и инсталляция конфигурационных файлов	233
Установка прав доступа и прав собственности	235
Назначение каталога для электронной почты	235
Конфигурирование ролей и административных привилегий	235
Заключительные действия	236
7.5. Добавление пользователей с помощью программы <code>useradd</code>	236
Команда <code>useradd</code> в системе Ubuntu	237
Команда <code>useradd</code> в системе SUSE	238
Команда <code>useradd</code> в системе Red Hat	238
Команда <code>useradd</code> в системе Solaris	239
Команда <code>useradd</code> в системе HP-UX	240
Команда <code>useradd</code> в системе AIX	240
Пример использования команды <code>useradd</code>	242
7.6. Добавление пользователей “пакетом” с помощью команды <code>newusers</code> (Linux)	243
7.7. Удаление пользователей	243
7.8. Отключение учетной записи	245
7.9. Управление учетными записями системными средствами	246
7.10. Уменьшение риска с помощью PAM	247
7.11. Централизация управления учетными записями	247
Протокол LDAP и служба Active Directory	247
Системы “единого входа”	248
Системы управления учетными данными	249
7.12. Рекомендуемая литература	250

7.13. Упражнения	250
Глава 8. Дисковая память	252
8.1. Добавление диска	253
Инструкции для системы Linux	253
Инструкции для системы Solaris	254
Инструкции для системы HP-UX	254
Инструкции для системы AIX	255
8.2. Аппаратное обеспечение дисковой памяти	255
Жесткие диски	256
Флеш-диски	258
8.3. Интерфейсы устройств хранения данных	259
Интерфейс PATA	261
Интерфейсы SATA	262
Параллельный интерфейс SCSI	262
Последовательный интерфейс SCSI	265
Что лучше: SCSI или SATA?	265
8.4. Программное обеспечение накопителей	266
8.5. Присоединение и низкоуровневое управление накопителями	269
Верификация инсталляции на уровне аппаратного обеспечения	269
Файлы накопителя	270
Форматирование и плохое управление блоками	273
Безопасное стирание дисков ATA	274
Команда <code>hdparm</code> : параметры диска и интерфейса (Linux)	275
Мониторинг жесткого диска с помощью стандарта SMART	276
8.6. Разбиение диска	278
Традиционное разбиение	279
Разбиение диска в стиле системы Windows	281
GPT: таблица разделов GUID	281
Разбиение дисков в системе Linux	282
Разбиение дисков в системе Solaris	283
Разбиение дисков в системе HP-UX	283
8.7. RAID: избыточные массивы недорогих дисков	283
Программная и аппаратная реализации системы RAID	284
Уровни системы RAID	284
Восстановление диска после сбоя	287
Недостатки конфигурации RAID 5	288
Команда <code>mdadm</code> : программное обеспечение RAID в системе Linux	289
8.8. Управление логическими томами	292
Реализации управления логическими томами	292
Управление логическими томами в системе Linux	294
Управление логическими томами в системе HP-UX	298
Управление логическими томами в системе AIX	300
8.9. Файловые системы	301
Файловые системы Linux: семейство ext	302
Файловые системы HP-UX: VxFS и HFS	303
Файловая система JFS2 в операционной системе AIX	303
Терминология файловой системы	303
Полиморфизм файловых систем	304
Команда <code>mkfs</code> : форматирование файловых систем	305

Команда fsck: проверка и исправление файловых систем	305
Монтирование файловой системы	306
Настройка автоматического монтирования	307
Монтирование USB-накопителя	310
Включение подкачки	310
8.10. Файловая система ZFS: все проблемы решены	311
Архитектура ZFS	311
Пример: добавление диска в системе Solaris	312
Файловые системы и свойства	313
Наследование свойств	314
Одна файловая система на пользователя	315
Мгновенные копии и клоны	316
Неразмеченные логические тома	318
Распределение файловой системы между NFS, CIFS и SCSI	318
Управление пулом хранения	319
8.11. Сеть хранения данных	320
Сети SAN	322
Протокол iSCSI: интерфейс SCSI на основе протокола IP	323
Загрузка с тома iSCSI	324
Специфика инициаторов iSCSI от разных производителей	324
8.12. Упражнения	328
Глава 9. Периодические процессы	330
9.1. Демон cron: системный планировщик	330
9.2. Формат crontab-файлов	331
9.3. Управление crontab-файлами	333
9.4. Использование версии демона Vixie-cron	334
9.5. Стандартные применения демона cron	335
Простые напоминания	335
Чистка файловой системы	336
Распространение конфигурационных файлов по сети	337
Ротация журнальных файлов	338
9.6. Упражнения	338
Глава 10. Резервное копирование	339
10.1. Принципы резервного копирования	340
Создавайте резервные копии на центральном компьютере	340
Маркируйте носители	340
Правильно выбирайте периодичность резервного копирования	341
Будьте осмотрительны при выборе архивируемых файловых систем	341
Старайтесь уместать каждодневные архивы на одном носителе	342
Храните носители вне рабочего помещения	342
Защищайте резервные копии	343
Активность файловой системы во время создания архива должна быть низкой	343
Проверяйте состояние своих носителей	344
Определите жизненный цикл носителя	345
Компонуите данные с учетом резервного копирования	346
Будьте готовы к худшему	346
10.2. Устройства и носители, используемые для резервного копирования	347
Оптические носители: CD_R/RW, DVD±R/RW, DVD-RAM и Blu-ray	347

Переносные и съемные жесткие диски	348
Магнитные ленты	349
Малые лентопротяжные устройства: 8-миллиметровые и DDS/DAT	349
Устройства DLT/S-DLT	349
Устройства AIT и SAIT	350
Устройства VXA/VXA-X	350
Устройства LTO	351
Системы с автоматической загрузкой носителей (автозагрузчики, ленточные массивы и библиотеки)	351
Жесткие диски	351
Интернет и службы облачного резервирования данных	352
Типы носителей	352
Что покупать	353
10.3. Экономия пространства и времени с помощью инкрементного архивирования	354
Простая схема	355
Умеренная схема	355
10.4. Команда dump: настройка режима резервирования	356
Архивирование файловых систем	356
Команда gestore: восстановление файлов из резервных копий	359
Восстановление файловых систем	361
Восстановление системы на новом оборудовании	362
10.5. Архивирование и восстановление при модификации операционной системы	363
10.6. Другие архиваторы	363
Команда tar: упаковка файлов	363
Команда dd: манипулирование битами	364
Резервирование файловых систем ZFS	365
10.7. Запись нескольких архивов на одну ленту	366
10.8. Программа Bacula	367
Модель, используемая программой Bacula	367
Настройка программы Bacula	369
Установка базы данных и демонов Bacula	369
Конфигурирование демонов Bacula	370
Разделы конфигурации	371
Конфигурирование демона управления: файл bacula-dir.conf	372
Конфигурирование демона хранения: файл bacula-sd.conf	375
Конфигурирование консоли: файл bconsole.conf	377
Установка и конфигурирование демона управления файлами клиента	377
Запуск демонов Bacula	378
Добавление носителей в пулы	378
Выполнение архивирования вручную	378
Выполнение задания восстановления	379
Архивирование клиентов Windows	382
Мониторинг конфигураций программы Bacula	382
Советы по использованию программы Bacula	383
Альтернатива программе Bacula	383
10.9. Коммерческие системы резервного копирования	384
ADSM/TSM	384
Veritas NetBackup	385
EMC NetWorker	385
Прочие программы	386

10.10. Рекомендуемая литература	386
10.11. Упражнения	386
Глава 11. Система Syslog и журнальные файлы	388
11.1. Обнаружение файлов регистрации	389
Специальные журнальные файлы	390
Особенности систем	392
11.2. Syslog: система регистрации событий	393
Архитектура системы Syslog	394
Конфигурирование демона syslogd	394
Примеры конфигурационных файлов	398
Отладка системы Syslog	400
Альтернатива системе Syslog	400
Журнальная регистрация на уровне ядра и на этапе начальной загрузки	401
11.3. Регистрация сообщений и обработка ошибок в системе AIX	402
Конфигурация системы Syslog в среде AIX	404
11.4. Утилита logrotate: управление журнальными файлами	405
11.5. Поиск полезной информации в журнальных файлах	407
11.6. Методы обработки журнальных файлов	408
11.7. Упражнения	410
Глава 12. Управление программным обеспечением и конфигурацией	412
12.1. Установка систем Linux и OpenSolaris	413
Загрузка по сети на персональном компьютере	413
Конфигурирование протокола PXE в Linux	414
Дистанционная загрузка на специализированных компьютерах	414
Использование Kickstart — автоматизированного инсталлятора Red Hat Enterprise Linux	415
Использование AutoYaST: автоматизированный инсталлятор SUSE	417
Автоматизированная установка систем Ubuntu	418
12.2. Установка Solaris	420
Сетевые установки с помощью JumpStart	421
Сетевые установки с помощью автоматизированного инсталлятора	425
12.3. Установка HP-UX	426
Автоматизация установок Ignite-UX	429
12.4. Установка системы AIX с помощью сетевого менеджера установки	429
12.5. Управление пакетами	430
12.6. Управление Linux-пакетами	431
Команда rpm: управление пакетами RPM	432
Команда dpkg: управление пакетами .deb в системе Ubuntu	433
12.7. Использование высокоуровневых систем управления пакетами в Linux	434
Хранилища пакетов	435
Служба Red Hat Network	436
APT: усовершенствованное средство управления пакетами	437
Конфигурирование apt-get	438
Создание локального зеркала хранилища	439
Автоматизация работы утилиты apt-get	440
Система yum: управление выпусками для RPM	441
Система управления пакетами Zypper для SUSE: теперь еще более мощная!	441

12.8. Управление пакетами для систем UNIX	442
Управление пакетами в Solaris	443
Управление пакетами в HP-UX	444
Управление программами в AIX	446
12.9. Управление изменениями	446
Создание резервных файлов	447
Формальные системы управления изменениями	447
Система Subversion	448
Система Git	450
12.10. Локализация и конфигурирование программного обеспечения	454
Организация локализации	454
Тестирование	456
Локальная компиляция	456
Распространение локализаций	457
12.11. Использование средств управления конфигурацией	458
Утилиты cfengine: компьютерная иммунная система	458
LCFG: крупномасштабная система конфигурирования	459
Template Tree 2: помощник cfengine	459
DMTF/CIM: общая информационная модель	460
12.12. Организация совместного использования программ через nfs	460
Пространства имен пакетов	461
Управление зависимостями	462
Сценарии упаковщика	462
12.13. Рекомендуемая литература	463
12.14. Упражнения	463
Глава 13. Драйверы и ядро	464
13.1. Адаптация ядра	465
13.2. Драйверы и файлы устройств	466
Файлы и номера устройств	467
Создание файлов устройств	468
Соглашения об именовании устройств	469
Сравнение пользовательских ядер с загружаемыми модулями	469
13.3. Конфигурирование ядра Linux	470
Конфигурирование параметров ядра linux	470
Сборка ядра linux	472
Не чините то, что еще не сломано	472
Конфигурирование параметров ядра	472
Компиляция ядра	473
Добавление драйвера устройства в Linux	474
13.4. Конфигурирование ядра Solaris	475
Пространство ядра Solaris	476
Конфигурирование ядра с помощью файла /etc/system	477
Добавление в систему Solaris драйверов устройств	478
Отладка Solaris-конфигурации	478
13.5. Конфигурирование ядра HP-UX	480
13.6. Управление ядром AIX	480
Администратор объектных данных	481
Настройка ядра	482
13.7. Загружаемые модули ядра	483

Загружаемые модули ядра в Linux	483
Загружаемые модули ядра в Solaris	485
13.8. Linux-менеджер устройств udev — полезно и приятно	486
Файловая система sysfs: доступ к “сердцу” устройств в Linux	486
Исследование устройств с помощью команды udevadm	487
Создание правил и постоянных имен	488
13.9. Рекомендуемая литература	491
13.10. Упражнения	492

Часть II. Работа в сетях 493

Глава 14. Сети TCP/IP 495

14.1. Система TCP/IP и Интернет	495
Кто сегодня управляет Интернетом	496
Сетевые стандарты и документация	497
14.2. Дорожная карта сети	498
Версии IPv4 и IPv6	499
Пакеты и их инкапсуляция	500
Стандарты формирования фреймов Ethernet	501
14.3. Адресация пакетов	502
Аппаратная адресация (MAC)	502
IP-адресация	503
“Адресация” имен машин	504
Порты	504
Типы адресов	504
14.4. IP-адреса	505
Классы адресов в протоколе IPv4	505
Подсети	506
Трюки и инструменты для арифметических вычислений, связанных с подсетям	507
CIDR: протокол бесклассовой междоменной маршрутизации	508
Выделение адресов	509
Частные адреса и система NAT	510
Адресация в стандарте IPv6	511
14.5. Маршрутизация	513
Таблицы маршрутизации	513
Директивы переадресации протокола ICMP	515
14.6. ARP: протокол преобразования адресов	515
14.7. DHCP: протокол динамического конфигурирования узлов	516
Программное обеспечение DHCP	517
Схема работы DHCP	518
Программное обеспечение DHCP, созданное организацией ISC	518
14.8. Вопросы безопасности	520
Перенаправление IP-пакетов	520
Директивы переадресации протокола ICMP	520
Маршрутизация “от источника”	520
Широковещательные ICMP-пакеты и другие виды направленных широковещательных сообщений	521
Подмена IP-адресов	521
Встроенные брандмауэры	522
Виртуальные частные сети	522

14.9. PPP: протокол двухточечного соединения	523
14.10. Основы конфигурирования сети	524
Присвоение сетевых имен и IP-адресов	525
Команда <code>ifconfig</code> : конфигурирование сетевых интерфейсов	526
Параметры сетевого оборудования	528
Команда <code>route</code> : конфигурирование статических маршрутов	529
Конфигурирование DNS	530
14.11. Сетевое конфигурирование в различных системах	531
14.12. Сетевое конфигурирование в системе Linux	532
Демон <code>NetworkManager</code>	532
Сетевое конфигурирование в системе Ubuntu	533
Сетевое конфигурирование в системе SUSE	534
Сетевое конфигурирование в системе Red Hat	535
Настройка сетевого оборудования в системе Linux	536
Опции протокола Linux TCP/IP	537
Переменные ядра, связанные с безопасностью	540
Система Linux NAT и фильтрация пакетов	540
14.13. Работа в сети под управлением системы Solaris	541
Основная сетевая конфигурация системы Solaris	541
Примеры конфигураций системы Solaris	543
Конфигурирование протокола DHCP в системе Solaris	544
Команда <code>ndd</code> : протокол TCP/IP и настройка интерфейса в системе Solaris	545
Безопасность в системе Solaris	546
Брандмауэры и фильтрация в системе Solaris	546
Механизм NAT в системе Solaris	547
Особенности сетевого конфигурирования	548
14.14. Работа в сети под управлением системы HP-UX	548
Базовое конфигурирование сетей в системе HP-UX	548
Примеры конфигураций в системе HP-UX	549
Конфигурирование протокола DHCP в системе HP-UX	551
Динамическое переконфигурирование и настройка в системе HP-UX	551
14.15. Работа в сети под управлением системы AIX	553
Команда <code>po</code> : настройка сетевых параметров в системе AIX	555
14.16. Рекомендуемая литература	555
14.17. Упражнения	557
Глава 15. Маршрутизация	558
15.1. Подробнее о маршрутизации пакетов	559
15.2. Демоны и протоколы маршрутизации	561
Дистанционно-векторные протоколы	562
Топологические протоколы	563
Метрики стоимости	563
Внутренние и внешние протоколы	564
15.3. Основные протоколы маршрутизации	564
Протоколы RIP и RIPv6	565
Протокол OSPF	566
Протокол EIGRP	566
IS-IS: протокол маршрутизации между промежуточными системами	567
Протоколы RDP и NDP	567
Протокол BGP	567

15.4. Выбор стратегии маршрутизации	567
15.5. Демоны маршрутизации	569
Демон route: устаревшая реализация в протоколе RIP	569
Демон gated: первый многопротокольный демон маршрутизации	570
Пакет Quagga: основной демон маршрутизации	570
Демон gamd: многопротокольная система маршрутизации для HP-UX	571
Маршрутизатор XORP	571
Специфика поставщиков	572
15.6. Маршрутизаторы Cisco	572
15.7. Рекомендуемая литература	575
15.8. Упражнения	576

Глава 16. Сетевые аппаратные средства 577

16.1. Технология Ethernet: сетевая панацея	578
Как работает Ethernet	579
Топология Ethernet	580
Неэкранированная витая пара	580
Оптическое волокно	582
Соединение и расширение сетей Ethernet	583
16.2. Беспроводной стандарт: локальная сеть для кочевников	587
Беспроводные коммутаторы и облегченные точки беспроводного доступа	589
16.3. DSL и кабельные модемы: “последняя миля”	589
16.4. Тестирование и отладка сетей	590
16.5. Прокладка кабелей	591
Неэкранированная витая пара	591
Офисные точки подключения	591
Стандарты кабельных систем	592
16.6. Проектирование сетей	593
Структура сети и архитектура здания	593
Расширение сетей	594
Перегрузка	594
Обслуживание и документирование	594
16.7. Управление сетью	594
16.8. Рекомендуемые поставщики	595
Кабели и разъемные соединения	595
Тестовые приборы	596
Маршрутизаторы/коммутаторы	596
16.9. Рекомендуемая литература	596
16.10. Упражнения	596

Глава 17. Система доменных имен 597

17.1. Основные задачи системы DNS	598
Управление системой DNS	599
17.2. Как работает система DNS	599
Записи о ресурсах	600
Делегирование	600
Кеширование и эффективность	601
Неоднозначные ответы	602
17.3. DNS для нетерпеливых: подключение нового компьютера	602

Добавление новой машины в систему DNS	602
Настройка конфигурации клиента DNS	605
17.4. Серверы имен	607
Авторитетные и кеширующие серверы	608
Рекурсивные и нерекурсивные серверы	609
17.5. Пространство имен DNS	610
Регистрация домена второго уровня	611
Создание собственных поддоменов	612
17.6. Разработка собственной среды DNS	612
Управление пространством имен	612
Авторитетные серверы	613
Кеширующие серверы	614
Требования к аппаратному обеспечению	614
Безопасность	615
Итоги	615
17.7. Что нового в системе DNS	616
17.8. База данных DNS	618
Команды в файлах зон	619
Записи о ресурсах	620
Запись SOA	623
Записи NS	625
Записи A	626
Записи PTR	626
Записи MX	627
Записи CNAME	628
Специальное применение записей CNAME	629
Записи SRV	630
Записи TXT	632
Записи ресурсов IPv6	632
Записи SPF	633
Записи DKIM и ADSP	635
Записи о ресурсах SSHFP	638
Записи о ресурсах DNSSEC	639
Связующие записи: связи между зонами	639
17.9. Программное обеспечение системы BIND	641
Определение версии	641
Компоненты системы BIND	643
Файлы конфигурации	643
Инструкция include	645
Инструкция options	646
Инструкция acl	652
Инструкция key (TSIG)	653
Инструкция trusted-keys	653
Инструкция server	654
Инструкция masters	655
Инструкция logging	655
Инструкция statistics	655
Инструкция zone	656
Инструкция controls для команды rndc	659
Расщепление DNS и инструкция view	660

17.10. Примеры конфигурации системы BIND	661
Зона локального узла	662
Небольшая компания, предоставляющая консалтинговые услуги в области безопасности	663
Консорциум The Internet Systems Consortium (isc.org)	666
17.11. Программное обеспечение NSD/Unbound	668
Инсталляция и конфигурирование системы NSD	668
Пример конфигурации системы NSD	670
Запуск демона nsd	675
Инсталляция и конфигурирование сервера Unbound	676
17.12. Обновление файлов зон	682
Передача зоны	683
Динамические обновления в системе BIND	684
17.13. Вопросы безопасности	686
Еще раз о списках управления доступом в сервере BIND	687
Открытые распознаватели	688
Работа в виртуальном окружении chroot	689
Безопасные межсерверные взаимодействия посредством технологий TSIG и TKEY	689
Настройка технологии TSIG для сервера BIND	690
Механизм TSIG на сервере NSD	692
Технология DNSSEC	692
Правила протокола DNSSEC	696
Записи о ресурсах DNSSEC	697
Настройка протокола DNSSEC	698
Генерирование пар ключей	699
Подписание зоны	701
Цепочка доверия в протоколе DNSSEC	704
Сервер DLV: динамическая проверка доменов	705
Смена ключей DNSSEC	706
Инструменты DNSSEC	707
Отладка протокола DNSSEC	709
17.14. Microsoft и DNS	711
17.15. Тестирование и отладка	711
Журнальная регистрация в пакете BIND	712
Журнальная регистрация в пакетах NSD и Unbound	717
Управляющие программы сервера имен	718
Сбор статистических данных	720
Отладка с помощью команды dig	721
Некорректное делегирование	722
Инструменты для проверки корректности системы DNS	723
Производительность системы	725
17.16. Специфика различных дистрибутивов	725
Специфика системы Linux	726
Специфика системы Solaris	728
Специфика системы HP-UX	729
Специфика системы AIX	730
17.17. Рекомендуемая литература	731
Списки рассылки и новостные группы	731
Книги и другая документация	731
Ресурсы в Интернете	732

Документы RFC	732
17.18. Упражнения	733
Глава 18. Сетевой протокол Network File System	735
18.1. Введение в протокол NFS	735
Проблемы, связанные с состоянием	736
Проблемы производительности	736
Безопасность	736
18.2. Серверная часть NFS	737
Версии и история протокола	737
Транспортные протоколы	738
Состояние	738
Экспорт файловой системы	738
Блокировка файлов	739
Вопросы безопасности	740
Идентифицирующее отображение в версии 4	741
Учетные записи root и nobody	742
Производительность версии 4	743
Дисковые квоты	743
18.3. Серверная часть протокола NFS	744
Команда share и файл dfstab (Solaris, HP-UX)	745
Команда exports и файл exports (Linux, AIX)	747
Файл exports в системе AIX	747
Файл exports в системе Linux	748
Демон nfsd: обслуживание файлов	750
18.4. Клиентская часть протокола NFS	751
Монтирование файловых систем NFS на этапе начальной загрузки	753
Ограничения на выбор порта	754
18.5 Идентифицирующее отображение в протоколе NFS 4	755
18.6. Команда nfsstat: отображение статистики NFS	755
18.7. Специализированные файловые серверы NFS	756
18.8. Автоматическое монтирование	757
Таблицы косвенных назначений	758
Таблицы прямых назначений	759
Главные таблицы	759
Исполняемые таблицы	760
Видимость демона automount	760
Реплицированные файловые системы и демон automount	761
Автоматическое монтирование (V3; все, кроме Linux)	761
Специфика системы Linux	762
18.9. Рекомендуемая литература	762
18.10. Упражнения	763
Глава 19. Совместное использование системных файлов	764
19.1. Предмет совместного использования	765
19.2. Копирование файлов	766
Использование сервера NFS	766
Сравнение модели принудительной рассылки с моделью рассылки по запросу	767
Утилита rdist: принудительная рассылка файлов	767

Утилита rsync: более безопасная рассылка файлов	770
Рассылка файлов по запросу	772
19.3. LDAP: упрощенный протокол доступа к каталогам	773
Структура данных LDAP	774
Особенности LDAP	775
Документация и спецификации LDAP	776
OpenLDAP: традиционный LDAP-сервер с открытым исходным кодом	776
389 Directory Server: альтернативный LDAP-сервер с открытым исходным кодом	777
LDAP вместо /etc/passwd и /etc/group	778
Создание LDAP-запросов	779
LDAP и безопасность	780
19.4. NIS: сетевая информационная служба	781
Модель NIS	781
Схема работы NIS	782
Безопасность NIS	784
19.5. Задание приоритетов для источников административной информации	784
Демон nscd: кеширование результатов поиска	785
19.6. Рекомендуемая литература	786
19.7. Упражнения	786
Глава 20. Электронная почта	787
20.1. Системы электронной почты	788
Пользовательские агенты	789
Агенты представления	790
Транспортные агенты	791
Локальные агенты доставки	791
Хранилища сообщений	792
Агенты доступа	792
Так много компонентов, так мало времени	793
20.2. Структура почтового сообщения	793
Заголовки почтовых сообщений	793
20.3. Протокол SMTP	795
Вы прислали мне привет	796
Коды ошибок протокола SMTP	797
Аутентификация SMTP	797
20.4. Принципы организации электронной почты	798
Почтовые серверы	799
20.5. Почтовые псевдонимы	802
Загрузка списков рассылки из файла	804
Направление почты в файл	804
Направление почты в программу	805
Примеры псевдонимов	805
Хешированная база данных псевдонимов	806
Списки рассылки и программы для работы с ними	806
Программы для работы со списками рассылки	806
20.6. Сканирование содержимого: спам и вредоносные программы	807
Спам	808
Подделки	809
Конфиденциальность сообщений	809
Фильтрация спама	809

Когда следует фильтровать	810
“Серые” списки/DCC	810
Программа SpamAssassin	811
Черные списки	812
Белые списки	812
Фильтрация почты	813
Технология SPF и спецификации Sender ID	813
Системы DomainKeys, DKIM и ADSP	814
Функциональные возможности транспортных агентов по борьбе со спамом	815
Программа MailScanner	815
Интерфейс amavisd-new	816
Проверка эффективности сканирования с помощью транспортных агентов	819
20.7. Конфигурация электронной почты	820
20.8. Почтовый агент sendmail	821
Файл переключения	822
Запуск программы sendmail	823
Почтовые очереди	824
20.9. Конфигурация программы sendmail	825
Препроцессор m4	825
Фрагменты конфигурации программы sendmail	826
Конфигурационный файл, построенный на основе эталонного файла с расширением .mc	827
20.10. Примитивы конфигурации программы sendmail	828
Таблицы и базы данных	828
Обобщенные макросы и функциональные возможности	829
Конфигурация клиентов	835
Параметры конфигурации	835
Средства программы sendmail для борьбы со спамом	837
Ретрансляция	838
Дроссели, скорость и ограничения на количество соединений	840
Конфигурирование почтовых фильтров в программе sendmail	841
Соединение сканера amavisd и программы sendmail	842
20.11. Безопасность и программа sendmail	843
Владельцы файлов	843
Права доступа	844
Безопасная пересылка почты в файлы и программы	845
Опции безопасности	846
Выполнение программы sendmail в виртуальном каталоге (для настоящих параноиков)	847
Отражение атак типа “отказ от обслуживания”	847
SASL: простой протокол аутентификации и защиты	848
TLS: безопасность транспортного уровня	848
20.12. Производительность программы sendmail	849
Режимы доставки	849
Группы очередей и разбивка конвертов	849
Обработчики очередей	850
Контроль средней загрузки	850
Обработка недоставленных сообщений	850
Настройка ядра	851
20.13. Сбор статистических данных, тестирование и отладка	853

Мониторинг очереди	853
Журнальная регистрация	854
20.14. Почтовый агент Exim	855
Инсталляция почтового сервера EXIM	856
Загрузка почтового сервера Exim	857
Утилиты почтового сервера Exim	858
Язык конфигурации программы Exim	859
Файл конфигурации программы Exim	859
Глобальные параметры	861
Сканирование содержимого на этапе применения списков управления доступом	866
Аутентификаторы	867
Маршрутизаторы	868
Маршрутизатор accept	869
Маршрутизатор dnslookup	870
Транспортные механизмы	872
Конфигурация getty	873
Конфигурация перезаписи	873
Функция локального сканирования	873
Сочетание программ amavisd и Exim	873
Регистрация	874
Отладка	875
20.15. Почтовый агент Postfix	876
Архитектура системы Postfix	876
Безопасность	878
Команды и документация системы Postfix	878
Конфигурация системы Postfix	879
Виртуальные домены	883
Управление доступом	885
Борьба со спамом и вирусами	888
Фильтрация содержимого с помощью программы amavisd	890
Отладка	891
20.16. Конфигурация механизма DKIM	893
Технология DKIM: DomainKeys Identified Mail	893
Почтовые фильтры DKIM	894
Конфигурация DKIM в программе amavisd-new	896
Технология DKIM в системе sendmail	897
Технология DKIM в системе Exim	898
Технология DKIM в системе Postfix	900
20.17. Интегрированные почтовые системы	900
20.18. Рекомендуемая литература	901
Общие вопросы по борьбе со спамом	901
Литература по программе sendmail	902
Литература о системе Exim	902
Литература о системе	902
Документы RFC	903
20.19. Упражнения	903
Упражнения, связанные с программой sendmail	904
Упражнения, связанные с системой Exim	905
Упражнения, связанные с системой Postfix	905

Глава 21. Управление сетями	907
21.1. Поиск неисправностей в сетях	908
21.2. Команда ping: проверка доступности компьютера	909
21.3. Инструмент SmokePing: сбор статистики в работе команды ping во времени	911
21.4. Команда traceroute: трассировка IP-пакетов	912
21.5. Команда netstat: получение информации о состоянии сети	915
Контроль состояния сетевых соединений	915
Отслеживание состояния сетевых соединений	917
Идентификация прослушивающих сетевых служб	918
Проверка таблицы маршрутизации	919
Просмотр статистических данных функционирования различных сетевых протоколов	919
21.6. Проверка функционирования интерфейса в реальном времени	921
21.7. Анализаторы пакетов	922
Утилита tcpdump: стандартный анализатор	923
Утилиты Wireshark и TShark: усовершенствованный вариант tcpdump	924
21.8. Служба Netalizr Института ICSI	926
21.9. Протоколы управления сетями	926
21.10. SNMP: простой протокол управления сетями	928
Структура протокола SNMP	928
Операции протокола SNMP	930
RMON: база MIB для дистанционного мониторинга	930
21.11. Агент NET-SNMP	931
21.12. Программы управления сетями	932
Команды пакета NET-SNMP	932
Сбор и накопление данных протокола SNMP	933
Nagios: событийная служба мониторинга	934
Совершенный пакет для мониторинга: поиски продолжаются	935
Коммерческие системы сетевого управления	936
21.13. Протокол NetFlow: мониторинг соединений	937
Мониторинг данных протокола NetFlow с помощью утилит nfdump и NfSen	937
Настройка протокола NetFlow для маршрутизатора Cisco router	939
21.14. Рекомендуемая литература	940
21.15. Упражнения	941
Глава 22. Безопасность	943
22.1. Безопасна ли система UNIX	944
22.2. Слабые места в системе защиты	945
Человеческий фактор	945
Ошибки в программах	946
Ошибки конфигурации	947
22.3. Ключевые аспекты безопасности	947
Программные “заплаты”	948
Ненужные службы	948
Удаленная регистрация событий	949
Резервные копии	949
Вирусы и черви	949
Троянские программы	950
Руткиты	951

Фильтрация пакетов	951
Пароли	951
Бдительность	951
Общие принципы защиты	952
22.4. Пароли и учетные записи пользователей	952
Устаревание паролей	953
Групповые и совместно используемые учетные записи	954
Пользовательские оболочки	954
Привилегированные учетные записи	954
22.5. Модули PAM: украшение или чудо аутентификации	954
Системная поддержка для моделей PAM	955
Конфигурация модулей PAM	955
Подробный пример конфигурации системы Linux	958
22.6. Программы с установленным битом SETUID	959
22.7. Эффективное использование команды chroot	960
22.8. Инструментальные средства защиты	961
Команда nmap: сканирование сетевых портов	961
Nessus: сетевой сканер следующего поколения	962
John the Ripper: средство для выявления слабых паролей	963
Команда hosts_access: управление доступом к узлу	964
Vro: программная система для распознавания вторжения в сеть	964
Snort: популярная программная система для распознавания проникновения в сеть	965
OSSEC: система для распознавания вторжения в сеть на уровне узла	966
22.9. Мандатное управление доступом	969
Система Linux с усиленной системой безопасности (SELinux)	969
22.10. Системы криптографической защиты	970
Kerberos: унифицированный подход к сетевой безопасности	971
PGP: высокая конфиденциальность	972
SSH: безопасная оболочка	972
Пакет Stunnel	976
22.11. Брандмауэры	978
Брандмауэры, фильтрующие пакеты	979
Принципы фильтрации служб	979
Брандмауэры, осуществляющие инспекцию пакетов с отслеживанием состояния соединений	980
Насколько безопасны брандмауэры	981
22.12. Особенности брандмауэров в системе Linux	981
Правила, цепочки и таблицы	982
Целевые директивы для правил	982
Настройка команды iptables в качестве брандмауэра	983
Полный пример	983
22.13. Модуль IPFilter для систем UNIX	985
22.14. Виртуальные частные сети	988
Туннели IPSEC	989
Так ли уж нужны виртуальные частные сети	989
22.15. Сертификация и стандарты	989
Сертификации	990
Стандарты безопасности	991
22.16. Источники информации по вопросам обеспечения безопасности	993

Организация CERT	993
Сервер SecurityFocus.com и список рассылки BugTraq	994
Блог Брюса Шнайера	994
Организация SANS	994
Информационные ресурсы отдельных дистрибутивов	995
Другие списки рассылки и веб-сайты	996
22.17. Что нужно делать в случае атаки на сервер	996
22.18. Рекомендуемая литература	998
22.19. Упражнения	999
Глава 23. Веб-хостинг	1001
23.1. Основы веб-хостинга	1001
Обнаружение ресурсов в сети веб	1002
Унифицированные указатели ресурсов	1002
Принцип работы HTTP	1003
Генерирование содержимого “на лету”	1004
Серверы приложений	1005
Распределение нагрузки	1006
23.2. Установка HTTP-сервера	1008
Выбор сервера	1008
Установка сервера Apache	1009
Конфигурирование сервера Apache	1010
Запуск сервера Apache	1011
Анализ регистрационных файлов	1011
Высокопроизводительный хостинг	1011
23.3. Виртуальные интерфейсы	1012
Конфигурирование виртуальных интерфейсов	1013
Передача серверу Apache информации о виртуальном интерфейсе	1015
23.4. Протокол Secure Sockets Layer	1016
Генерирование файла Certificate Signing Request	1016
Конфигурация веб-сервера Apache для использования протокола SSL	1018
23.5. Кеширование и прокси-серверы	1018
Использование кеша Squid и прокси-сервера	1019
Установка сервера Squid	1020
Настройка обратного прокси с помощью веб-сервера Apache	1020
23.6. Расширение возможностей	1022
Облачные вычисления	1022
Хостинг совместного размещения серверов	1022
Сети для распределения контента	1023
23.7. Упражнения	1023
Часть III. Разное	1025
Глава 24. Виртуализация	1027
24.1. Виртуальный жаргон	1028
Полная виртуализация	1029
Паравиртуализация	1029
Виртуализация на основе операционной системы	1030
Естественная виртуализация	1031
Облачные вычисления	1031

Динамическая миграция	1032
Сравнение технологий виртуализации	1032
24.2. Преимущества виртуализации	1032
24.3. Практичный подход	1033
24.4. Виртуализация с помощью системы linux	1035
Введение в платформу Xen	1035
Основы платформы Xen	1036
Инсталляция гостя на платформе Xen с помощью программы virt-install	1037
Динамическая миграция на платформе Xen	1038
Платформа KVM	1039
Инсталляция платформы KVM и ее использование	1040
24.5. Зоны и контейнеры системы solans	1042
24.6. Разделы рабочей нагрузки в системе AIX	1045
24.7. Программное обеспечение Integrity Virtual Machines в системе HP-UX	1047
Создание и инсталляция виртуальных машин	1047
24.8. VMware: операционная система со своими собственными правами	1049
24.9. Веб-службы компании Amazon	1049
24.10. Рекомендуемая литература	1054
24.11. Упражнения	1054
Глава 25. Система X Window System	1055
25.1. Диспетчер дисплеев	1057
25.2. Процесс запуска X-приложения	1058
Переменная окружения DISPLAY	1059
Аутентификация клиентов	1060
Перенаправление соединений с помощью протокола SSH	1061
25.3. Конфигурирование X-сервера	1063
Раздел Device	1065
Раздел Monitor	1065
Раздел Screen	1066
Раздел InputDevice	1066
Раздел ServerLayout	1068
Утилита xrandr: конфигуратор X-сервера	1068
Установка режима ядра	1069
25.4. Устранение неполадок и отладка X-сервера	1070
Специальные комбинации клавиш в системе X	1070
Когда с X-сервером творится что-то неладное	1070
25.5. Краткое замечание о настольных средах	1072
KDE	1073
GNOME	1073
Что лучше: GNOME или KDE?	1074
25.6. Рекомендуемая литература	1074
25.7. Упражнения	1074
Глава 26. Печать	1076
26.1. Архитектура системы печати	1077
Основные системы печати	1077
Очереди печати	1078
26.2. Система печати CUPS	1078

Интерфейсы для системы печати	1078
Очередь на печать	1079
Множество принтеров	1080
Экземпляры принтеров	1080
Сетевая печать	1080
Фильтры	1081
Управление сервером CUPS	1082
Настройка сетевого сервера печати	1083
Автоматическое конфигурирование принтера	1083
Конфигурирование сетевых принтеров	1084
Примеры конфигурирования принтеров	1084
Создание класса принтеров	1085
Отключение принтера	1085
Другие связанные с конфигурированием задачи	1086
26.3. Печать в настольных системах	1087
Документы kprinter: printing	1088
Konqueror и печать	1088
26.4. Система печати System V	1089
Обзор	1089
Пункты назначения и классы	1090
Краткое описание команды lp	1090
Команды lpsched и lpshut: начало и конец печати	1091
Команда lpadmin: конфигурирование среды печати	1091
Примеры использования команды lpadmin	1094
Команда lpstat: получение информации о состоянии системы печати	1094
Команда cancel: удаление заданий печати	1095
Команды accept и reject: управление очередью печати	1095
Команды enable и disable: управление печатью	1096
Команда lpmove: перемещение заданий	1096
Интерфейсные программы	1096
Что делать, если система печати вышла из строя	1097
26.5. Печать BSD и ADC	1098
Обзор архитектуры системы BSD	1098
Управление средой печати	1099
Демон lpd: буферизация заданий на печать	1100
Команда lpr: выдача заданий на печать	1100
Команда lprq: просмотр очереди печати	1100
Команда lprm: удаление заданий на печать	1101
Команда lpc: внесение административных изменений	1101
Файл /etc/printcap	1103
Переменные файла printcap	1104
26.6. Долгая странная история	1108
История печати и появления систем печати	1108
Разнообразие принтеров	1109
26.8. Основные программы печати	1110
26.8. Языки принтеров	1111
PostScript	1112
PCL	1112
PDF	1113
XPS	1113

PJL	1114
Драйверы принтеров и как они обрабатывают PDL-языки	1114
26.9. Файлы PPD	1115
26.10. Форматы бумаги	1116
26.11. Практические советы	1118
Выбор принтера	1118
GDI-принтеры	1119
Двусторонняя печать	1119
Другие аксессуары для принтера	1120
Принтеры с последовательным и параллельным интерфейсом	1120
Сетевые принтеры	1121
Другие советы	1121
26.12. Советы по выявлению проблем	1124
Повторный запуск демона печати	1125
Регистрационные журналы	1125
Проблемы с прямой печатью	1125
Проблемы с печатью в сети	1126
Проблемы с распространением	1127
26.13. Рекомендуемая литература	1127
26.14. Упражнения	1127
Глава 21. Центры обработки данных	1129
27.1. Уровни надежности центров обработки данных	1130
27.2. Терморегуляция	1131
Теплые и холодные отсеки	1133
Влажность	1134
Мониторинг окружающей среды	1135
27.3. Электропитание	1135
Требования к электроснабжению стоек	1136
Удаленное управление	1137
27.4. Стойки	1138
27.5. Инструменты	1138
27.6. Рекомендованная литература	1139
27.7. Упражнения	1139
Глава 28. Экологичные информационные технологии	1141
28.1. Введение в экологичные информационные технологии	1142
28.2. Экологическая пирамида	1144
28.3. Стратегии разработки экологичных информационных технологий:	
центр обработки данных	1144
Консолидация приложений	1145
Консолидация серверов	1146
Сеть SAN	1147
Серверная виртуализация	1147
Только самые необходимые серверы	1148
Использование детализированных данных и планирование потребления	1148
Конфигурация серверов, оптимизированная по потреблению энергии	1148
Облачные вычисления	1150
Свободное охлаждение	1150

Эффективное охлаждение центра обработки данных	1151
Режим ограниченной функциональности во время простоя	1151
Продление срока эксплуатации оборудования	1151
Более высокая температура в центре обработки данных	1152
Энергосберегающее оборудование	1152
28.4. Экологические информационные стратегии: рабочее место пользователя	1153
28.5. Компании, поддерживающие экологичные информационные технологии	1154
28.6. Упражнения	1155

Глава 29. Анализ производительности 1157

29.1. Способы повышения производительности	1159
29.2. Факторы, влияющие на производительность	1160
29.3. Как анализировать проблемы производительности	1161
29.4. Проверка производительности системы	1162
Инвентаризуйте свое оборудование	1162
Сбор данных о производительности	1166
Анализ использования центрального процессора	1166
Управление памятью в системе	1168
Анализ использования памяти	1170
Анализ операций обмена с диском	1172
Утилита xdd: анализ производительности дисковой подсистемы	1174
Команда sag: сбор статистических данных и генерирование отчетов по ним	1174
nmon и nmon_analyser: мониторинг производительности в системе AIX	1175
Выбор Linux-планировщика ввода-вывода	1175
Программа oprofile: универсальный профилировщик системы Linux	1176
29.5. Помогите! Моя система почти остановилась!	1177
29.6. Рекомендуемая литература	1178
29.7. Упражнения	1179

Глава 30. Взаимодействие с системой Windows 1180

30.1. Вход в систему UNIX из Windows	1180
30.2. Получение доступа к удаленным настольным средам	1181
Запуск сервера X Server на компьютере Windows	1182
VNC: система виртуальных сетей	1183
Протокол RDP в системе Windows	1184
30.3. Запуск системы Windows и Windows-приложений	1184
Двухвариантная загрузка и почему ею не стоит пользоваться	1185
Альтернативы Microsoft Office	1185
30.4. Использование утилит командной строки в системе Windows	1186
30.5. Совместимость Windows со стандартами электронной почты и веб	1187
30.6. Совместное использование файлов при помощи Samba и CIFS	1187
Samba: сервер CIFS для UNIX	1188
Установка Samba	1189
Кодирование имен файлов	1190
Аутентификация пользователей	1191
Совместное использование основных файлов	1191
Групповые ресурсы	1192
Прозрачная переадресация при помощи MS DFS	1193
Программа smbclient: простой клиент CIFS	1194

30.7. Совместное использование принтеров при помощи Samba	1195
Установка драйвера принтера из системы Windows	1197
Инсталляция принтера из командной строки	1197
30.8. Отладка сервера Samba	1198
30.9. Аутентификация службы Active Directory	1200
Подготовка к интеграции службы Active Directory	1201
Конфигурирование протокола Kerberos для интеграции службы Active Directory	1202
Программа Samba как член домена Active Directory	1203
Конфигурация системы PAM	1205
Альтернатива демону winbind	1206
30.10. Рекомендуемая литература	1206
30.11. Упражнения	1206

Глава 31. Последовательные устройства и терминалы 1208

31.1. Стандарт RS-232C	1209
31.2. Альтернативные разъемные соединения	1211
Разъем DB-9	1211
Разъем RJ-45	1212
31.3. Аппаратная и программная несущие	1213
31.4. Аппаратный контроль передачи данных	1213
31.5. Файлы последовательных устройств	1214
31.6. Команда setserial: передача драйверу параметров последовательного порта в системе Linux	1215
31.7. Псевдотерминалы	1216
31.8. Конфигурирование терминалов	1216
Процедура регистрации в системе	1217
Файл /etc/ttytype	1218
Файл /etc/gettytab	1218
Файл /etc/gettydefs	1219
Файл /etc/inittab	1219
Демон Upstart в системе Ubuntu	1221
31.9. Специальные символы и драйвер терминала	1222
31.10. Команда stty: задание параметров терминала	1223
31.11. Команда tset: автоматическое задание параметров терминала	1224
31.12. Как справиться с “зависшим” терминалом	1225
31.13. Отладка последовательной линии	1225
31.14. Соединение с консолями последовательных устройств	1226
31.15. Упражнения	1227

Глава 32. Управление, стратегия и политика 1228

32.1. Цель информационных технологий	1228
Составление сметы и расходование средств	1229
Стратегия в области информационных технологий	1230
Соглашения о качестве оказываемых услуг	1231
32.2. Информационная структура организации	1235
Основы: система отслеживания запросов и управления задачами	1235
Общие функции систем отслеживания запросов	1236
Владение запросом	1237
Система отслеживания запросов с точки зрения пользователей	1238

Типичные диагностические системы	1238
Распределение сообщений о неполадках	1239
Перечень навыков	1240
Управление временем	1241
32.3. Служба поддержки	1241
Диапазон услуг	1241
Доступность службы поддержки	1242
Зависимость от службы поддержки	1242
32.4. Архитектура предприятия	1242
Процессы должны быть воспроизводимыми	1243
Сохранение отладочных данных	1243
Осознание важности документации	1244
Настройка и кодирование	1244
Содержание системы в чистоте	1244
32.5. Операции	1244
Время простоев должно быть минимальным	1245
Документирование зависимостей	1245
Переналадка или списывание старого оборудования	1245
Поддержка локальной документации	1246
Разделение окружающей среды	1250
Автоматизируйте, автоматизируйте, автоматизируйте!	1250
32.6. Управление	1251
32.7. Инструкции и процедуры	1260
Различие между инструкциями и процедурами	1260
Эффективные инструкции	1261
Процедуры	1261
32.8. Восстановление после аварий	1262
Оценка рисков	1262
Борьба со стихийными бедствиями	1263
Подбор персонала на случай аварии	1264
Электропитание и кондиционирование	1265
Сетевая избыточность	1266
Проблемы с безопасностью	1267
32.9. Соответствие законам и стандартам	1267
Библиотека ITIL: Information Technology Infrastructure Library	1270
NIST: Национальный Институт стандартов и технологии	1270
32.10. Правовые вопросы	1271
Конфиденциальность	1271
Реализация стратегии	1272
Контроль — это ответственность	1273
Лицензии на программное обеспечение	1273
32.11. Организации, конференции и другие ресурсы	1274
32.12. Рекомендуемая литература	1276
32.13. Упражнения	1276
Краткая история системного администрирования	1278
В защиту AIX	1288
Предметный указатель	1291

ПРЕДИСЛОВИЕ

Двадцать семь лет назад, т.е. в 1983 году, я написал, пожалуй, первое руководство по системному администрированию для операционной системы UNIX. Меня пригласили в Массачусеттскую компанию (Massachusetts Computer Company — MASSCOMP), специализирующуюся на UNIX-машинах, для написания документации. Завершив означенную в контракте работу над руководством по графическому программированию, я прикидывал, к чему бы еще мне можно было здесь приложить свои силы. “При возникновении системных проблем мы всегда обращаемся к Тому Тейкшейре, — сказал я. — А что делать нашим клиентам?”

Ответ не заставил себя ждать: “Нам просто позарез нужно практическое руководство”. И вскоре я снова работал, но на этот раз должен был извлечь максимум полезной информации из головы Тома Тейкшейры и перенести ее на бумагу.

В результате получилась книга, в которой были описаны такие основные понятия, как суперпользователь, добавление учетной записи, управление разрешениями, резервирование и восстановление, организация сети с помощью протокола UUCP и пр. Книга была ориентирована на System V, одну из двух главных (на то время) разновидностей UNIX (другой была Berkeley UNIX).

В конечном счете я вполне справился с поставленной передо мной задачей — добыл ценную информацию от Тома и других членов немногочисленной (на то время) касты элитных системных администраторов. И когда в 1989 году увидела свет книга *UNIX System Administration Handbook* (USAH), у меня уже не было сомнений в том, что это авторитетное руководство — “библия” в своем роде — получило заслуженное признание, и неудивительно: ведь оно вышло непосредственно из-под пера (то бишь клавиатуры) истинных мастеров своего дела.

К тому времени О’Рейли стал издателем. Узнав, что многие мои клиенты (в сфере написания технической документации) перешли на UNIX, я начал сохранять права на написанные мною руководства, чтобы можно было перепродавать их другим компаниям. В конце 1985 года мы представили наши первые книги, которые уже не лицензировались для компаний, а находились в открытой продаже. Сначала мы выпускали небольшие книги, посвященные таким отдельным темам, как *vi*, *sed* и *awk*, *termcap* и *terminfo*, или протоколу UUCP. Мы называли их “*Nutshell Handbooks*” (“Справочники в двух словах” — *Примеч. ред.*), поскольку хотели охватить все темы и собрать их под одной “оболочкой”.

В действительности мы ничего не знали об издательском деле. Наши книги не имели ни корешков (они сшивались скобками), ни предметных указателей, ни ISBN (международный стандартный номер книги). Мы продавали их по почте, а не через книжные магазины. Но мало-помалу мы осваивали эту сферу деятельности и в конце концов стали конкурировать с известными издателями компьютерных книг.

Любимая для нас тема — общие методы администрирования UNIX, но до недавних пор мы не брались за нее. И вот почему. Я — сторонник удовлетворения возникших потребностей без конкуренции с кем-либо ради этого. И было абсолютно ясно, что по означенной теме уже есть не просто хорошая, а ВЕЛИКОЛЕПНАЯ книга! Я не видел ни необходимости соперничать с такой всеобъемлющей книгой, ни возможности добиться успеха в этом направлении.

Но со временем, когда наш бизнес стал на ноги и мы вышли на рынок компьютерных книг, стало понятно, что конкуренция действительно может способствовать расширению рынка. Люди, как правило, видя одну книгу, воспринимают ее как нечто исключительное. Если же им на глаза попадается несколько книг по одной теме, то, как говорил певец Арло Гатри (Arlo Guthrie), “они могут подумать, что это движение”.

Кроме того, в первом издании *USAH* авторы четко взяли курс на BSD-ориентированные системы (BSD UNIX), и мы подумали, что осталась свободной ниша для книги с уклоном в сторону System V.

В 1991 году мы выпустили в свет собственную книгу по системному администрированию UNIX, а именно Eelen Frisch, *Essential System Administration*.

Как автор, редактор и издатель, я никогда не придавал большого значения конкуренции — за исключением нескольких случаев. И это один из таких случаев. *UNIX System Administration Handbook* — это одна из немногих книг, на которые мы равняемся. Могли бы мы сделать так же хорошо? Могли бы мы сделать еще лучше? Подобно дуэли Мэджика Джонсона и Ларри Берда в NBA, соперничество выявляет лучшее, что есть в нас.

Ну вот, опять! Четвертое издание? Элин не мешало бы вернуться к работе!:-)

ВВЕДЕНИЕ

Когда мы писали первое издание этой книги в середине 1980-х, нам очень хотелось сравнить нашу рукопись с другими книгами по системному администрированию. К нашему удивлению, мы отыскиали только три. Теперь же вы можете выбирать из сотен книг. Назовем преимущества нашего издания.

- Эта книга является практическим руководством, в котором мы делимся с читателями нашим коллективным опытом системного администрирования и предлагаем рекомендации, которые выдержали испытание временем. Она содержит множество практических примеров и советов.
- Здесь не говорится о том, как использовать UNIX или Linux в домашних условиях. Мы описываем применение операционных систем в коммерческих компаниях, правительственных учреждениях, университетах.
- В книге подробно излагаются вопросы, связанные с работой UNIX- и Linux-систем в сетях. Это самый трудный аспект системного администрирования, и именно здесь наша помощь, как нам кажется, будет для читателей наиболее полезной.
- В этом руководстве рассмотрены основные варианты UNIX и Linux.

Структура книги

Книга разбита на три части — “Основы администрирования”, “Работа в сети” и “Разное”.

В части I, “Основы администрирования”, приводится общий обзор систем UNIX и Linux с точки зрения системного администратора. В главах этой части представлены основные сведения и описаны методики, необходимые для управления автономной системой. В этой части описаны протоколы, используемые в UNIX-системах, а также способы построения, расширения и администрирования сетей и интернет-серверов. Здесь же рассматривается высокоуровневое сетевое программное обеспечение. Среди изучаемых тем можно выделить систему доменных имен (DNS), сетевую файловую систему (NFS), систему электронной почты и инструменты сетевого управления.

В части II, “Работа в сети”, описываются протоколы, используемые в системах UNIX, а также способы настройки, расширения и эксплуатации сетей и серверов, с выходом в Интернет. Кроме того, в ней рассматривается высокоуровневое сетевое программное обеспечение, а также система доменных имен, сетевая файловая система, электронная почта и управление сетью.

В части III, “Разное”, содержит разнообразную вспомогательную информацию. В некоторых главах обсуждаются дополнительные функциональные возможности, такие как поддержка виртуализации серверов. В других главах даются рекомендации по самым разным темам: от экологического аспекта до принципов функционирования групп системного администрирования.

В конце каждой главы приводится набор практических заданий. Каждому упражнению дана оценка уровня сложности, причем учитывается как сложность самой задачи, так и время, затрачиваемое на ее выполнение.

Упражнения соответствуют четырем уровням сложности,

звездочки отсутствуют Простое задание, не требующее особых усилий

Более сложное или трудоемкое задание, для которого может потребоваться лабораторная работа

Наиболее сложное или трудоемкое задание, требующее лабораторной работы

Курсовые проекты (лишь в нескольких главах)

Для выполнения некоторых упражнений требуется доступ к системе с правами суперпользователя. Иногда нужно получить предварительное разрешение у системного администратора локальной сети. Подобные случаи оговариваются в задании.

ЧАСТЬ I

Основы администрирования

Глава 1

С чего начать

В настоящее время в море информации по UNIX и Linux можно утонуть, поэтому мы решили с помощью этой книги проложить свой курс и удовлетворить потребности именно системных администраторов, заняв тем самым вполне конкретную нишу в экосистеме разного рода справочных страниц (man pages), блогов, журналов, книг и прочих справочных материалов.

Во-первых, это — руководство, в котором рассматриваются различные компоненты основных систем администрирования и принципы их совместной работы. Во многих случаях, когда нам приходилось выбирать между разными реализациями некоторой идеи, мы описывали преимущества и недостатки основных вариантов.

Во-вторых, это — краткий справочник, в котором собрано все, что необходимо знать для выполнения задач общего характера в различных версиях UNIX- и Linux-систем. Например, команда `ps`, отображающая статус текущих процессов, поддерживает более 80 ключей (опций) командной строки в системах Linux. Но всего несколько комбинаций ключей удовлетворяют 99% нужд системного администратора (см. раздел 5.7).

Наконец, в этой книге делается акцент на администрировании корпоративных серверов и сетей, т.е. на серьезной теме системного администрирования. Несложно установить операционную систему на отдельном компьютере, гораздо труднее поддерживать Устойчивую работу виртуализированной сетевой среды при больших нагрузках, сбоях в работе дисков и умышленных атаках. Поэтому мы описываем методы и практические способы, которые позволят вам “поднимать” “упавшие” сети, а также выбирать решения, которые останутся актуальными при расширении и усложнении вашего сайта и изменении его гетерогенности.

Мы не претендуем на абсолютную степень объективности в решении перечисленных выше задач, поэтому по ходу изложения постарались максимально ясно пояснить свои

субъективные взгляды и предпочтения. Особенность системного администрирования заключается в том, что опытные администраторы могут иметь совершенно разные представления о правилах и процедурах управления системами. Читателям придется самостоятельно решать, какой именно материал и в какой степени соответствует той среде, в которой они работают.

1.1. Основные задачи системного администратора

Подробнее о написании сценариев рассказывается в главе 2.

В Википедии в статье “системный администратор” представлено довольно полное описание функций, которые, как общепринято считать, должен выполнять системный администратор. В английской версии этой статьи проводится четкое разграничение понятий администрирования и разработки программ, но мы знаем не понаслышке, сколько времени профессиональные администраторы отдают написанию сценариев (scripts). Сам по себе этот факт не превращает системных администраторов в разработчиков, но это означает, что им необходимо иметь практически такие же аналитические способности и знания в области архитектуры компьютерных сетей.

В этом разделе перечислены задачи, обычно возлагаемые на системного администратора. Совсем не обязательно, чтобы эти функции выполнял один человек. Во многих организациях работа поручается команде специалистов. В любом случае необходим хотя бы один человек, который понимал бы все поставленные задачи и обеспечивал их корректное выполнение другими людьми.

Инициализация пользователей

Подробнее о добавлении новых пользователей рассказывается в главе 7.

В круг обязанностей системного администратора входит создание учетных записей для новых пользователей, удаление учетных записей тех пользователей, которые уже не работают в системе, и решение всех проблем, возникающих во время “системной жизни” своих подопечных (например, “подсказка” забытых паролей). Процесс управления записями можно автоматизировать, но ряд решений, связанных с включением в систему нового пользователя (место размещения его начального каталога, компьютер, на котором должна создаваться учетная запись, и так далее), должен принимать только администратор.

Если необходимо прекратить доступ пользователя к системе, его учетная запись должна быть аннулирована. Все файлы, относящиеся к этому пользователю, необходимо удалить, чтобы они не занимали место на диске.

Подключение и удаление аппаратных средств

Дополнительная информация по данной теме приведена в главах 8,13 и 26.

В случае приобретения новых аппаратных средств или подключения уже имеющихся устройств к другому компьютеру, нужно переконфигурировать систему таким образом, чтобы она распознавала и активизировала эти устройства. Изменение конфигурации может быть как простой задачей (например, подключение принтера), так и более сложной (например, подключение жесткого диска).

Теперь, когда в сферу корпоративной обработки данных прочно вошло понятие виртуализации, конфигурация технических средств может усложниться как никогда ранее. Установка устройств может занимать несколько уровней стека виртуализации, и си-

системному администратору приходится вырабатывать новые стратегии, которые позволяют безопасно и качественно использовать общие аппаратные средства.

Резервное копирование

Подробнее о резервном копировании рассказывается в главе 10.

Резервное копирование является, наверное, одной из наиболее важных задач системного администратора, которая, к сожалению, чаще всего игнорируется или выполняется спустя рукава. Процедура резервного копирования довольно утомительна и занимает много времени, но осуществлять ее необходимо. Этот процесс можно автоматизировать или поручить подчиненным, но все равно системный администратор обязан убедиться в том, что резервное копирование выполнено правильно и по графику (а с полученного носителя можно восстановить данные).

Инсталляция и обновление программ

Подробнее об утилитах работы с программами рассказывается в главе 12.

После приобретения нового программного обеспечения его нужно установить и протестировать, часто в нескольких операционных системах и на различном оборудовании. Если программы работают нормально, пользователям необходимо сообщить об их наличии и местонахождении. Выпускаемые пакеты обновлений для исправления ошибок и устранения брешей в системе безопасности должны без проблем устанавливаться в локальных системах.

Локальные программы и административные сценарии следует соответствующим образом упаковать и организовать в виде, обеспечивающем совместимость с процедурами обновления, используемыми в системах вашего сайта. Новые версии ваших программных продуктов, прежде чем использовать их во всем узле сети, необходимо представлять для тестирования.

Мониторинг системы

Крупные системы требуют неусыпного контроля. Не стоит надеяться, что пользователи всегда сами будут сообщать вам о затруднениях (если, конечно, они не столкнутся по-настоящему с серьезными проблемами). Обычно пользователи идут по пути наименьшего сопротивления, полагая, что на решение проблемы у них уйдет меньше времени, чем на ее описание и сообщение о ее возникновении.

Существует множество обязательных ежедневных операций, а именно: проверка правильности функционирования электронной почты и веб-служб; просмотр журнальных файлов на предмет наличия ранних признаков неисправностей; контроль над подключением локальных сетей; контроль доступности системных ресурсов (в частности, проверка наличия свободного места на диске). Все эти рутинные операции прекрасно поддаются автоматизации, да и множество готовых систем мониторинга могут помочь системным администраторам в решении этой задачи.

Поиск неисправностей

Сбои систем неизбежны. Задача администратора — диагностировать сбои в системе и в случае необходимости вызвать специалистов. Как правило, найти неисправность бывает намного сложнее, чем устранить ее.

Ведение локальной документации

Рекомендации, касающиеся ведения документации, даны в разделе 32.5.

При настройке конфигурации системы под конкретные требования очень скоро обнаруживается, что она значительно отличается от базовой конфигурации, которая описана в документации. А поскольку за реализацию этих настроек отвечает системный администратор, он должен документировать все изменения. Администратор должен также вести учет обслуживания всех аппаратных средств, регистрировать состояние резервных копий, документировать разводку сетевых кабелей и локальные правила работы в системе.

Слежение за безопасностью системы

Вопросы безопасности рассматриваются в главе 22.

Системный администратор отвечает за реализацию стратегии защиты и должен периодически проверять, не нарушена ли безопасность системы. В системах с низким уровнем безопасности эта процедура может быть сведена к нескольким элементарным проверкам на предмет несанкционированного доступа. В системах с высоким уровнем безопасности обычно применяется сложная сеть ловушек и программ контроля.

Оказание помощи пользователям

О необходимости оказания помощи пользователям в решении различных проблем редко упоминается в должностной инструкции системного администратора, хотя выполнение подобного рода обязанностей “съедает” большую часть его рабочего времени. Системных администраторов терроризируют самыми разными вопросами, начиная от “Вчера моя программа работала, а сегодня нет! Что вы поменяли?” до “Я пролила кофе на клавиатуру! Нужно ли теперь полить ее водой, чтобы смыть кофе?”.

В большинстве случаев ваша реакция на все эти “сигналы тревоги” гораздо больше влияет на вашу оценку как администратора, чем реальные технические навыки, которыми, возможно, вы обладаете. Вы можете либо роптать на несправедливость такого положения вещей, либо радоваться тому, что успешным решением одной-единственной проблемы вы можете заслужить такую же благосклонность сотрудников (или начальства), как и пятичасовой отладкой в ночное время. Выбирайте!

1.2. Что необходимо знать

Мы предполагаем, что у читателей есть определенный опыт работы с Linux или UNIX. В частности, необходимо иметь общее представление о системе с точки зрения пользователя, поскольку мы не будем на этом останавливаться. Книги, перечисленные в разделе 1.14, помогут заложить необходимый фундамент знаний.

Даже в дни композитных оконных 3D-менеджеров (например, Compiz Fusion) GUI-инструменты для UNIX- и Linux-систем остаются довольно простыми по сравнению с навороченными базовыми программами. На практике большинство задач администрирования по-прежнему решается путем редактирования конфигурационных файлов и написания сценариев, поэтому читатели должны быть знакомы как с оболочкой командной строки, так и с каким-нибудь текстовым редактором.

Ваш редактор может иметь графический интерфейс (как у `gedit`) или работать в среде интерпретации командной строки (как `vi` или `emacs`). Такие текстовые процессоры,

как Microsoft Word и OpenOffice Writer, кардинально отличаются от упомянутых текстовых редакторов и практически бесполезны для решения задач администрирования. Командные редакторы имеют неоспоримое преимущество, поскольку они могут преодолевать простые SSH-соединения и работать на слабых системах, которые отказываются выполнять начальную загрузку; поэтому в оконных системах нет никакой необходимости. Кроме того, они действуют гораздо быстрее маленьких проворных редакторов, которые часто создают администраторы.

Рекомендуем изучить редактор **vi** (или его версию **vim**). Он является стандартным для всех UNIX- и Linux-систем и, хотя выглядит несколько “бледным” на фоне более мощных программ (в частности, **emacs**), абсолютно пригоден для работы. Нам также нравится GNU-редактор **nano**, который прост и прекрасно подходит новичкам, к тому же он оснащен экранными подсказками. Остерегайтесь нестандартных редакторов. Если отдать предпочтение одному из них, очень быстро надоеет устанавливать его в каждой новой системе.

Одним из важнейших инструментов системного администратора являются сценарии для автоматизации основных задач. Примеры такого рода сценариев приводятся на протяжении всей книги. Для того чтобы стать профессионалом, необходимо научиться читать и модифицировать сценарии, написанные на языке интерпретатора **sh** (в Linux его эквивалент — интерпретатор **bash**).

Подробнее о написании сценариев рассказывается в главе 2.

Для новых проектов мы рекомендуем применять язык **Perl** или **Python**. Как язык программирования **Perl** несколько необычен, однако содержит много средств, необходимых администраторам. В качестве учебника по этому языку советуем прочесть книгу Ларри Уолла *Programming Perl*; она также представляет собой образец профессионального технического руководства. Библиографическое описание этой книги приведено в разделе 1.14.

Многие администраторы предпочитают иметь дело с языком **Python**. В определенном смысле это более элегантный язык, чем **Perl**, и его сценарии обычно легче читать и сопровождать. (Вот что сказал по этому поводу бывший сотрудник издательства Amazon Стив Йег: “Python уже давно стал пристанищем для тех, кто, наконец, принял красную пилюлю и освободился от Perl-матрицы”.) Статьи, посвященные сравнению языка **Python** с другими языками (включая **Perl**), можно найти по следующему адресу: www.python.org/doc/Comparisons.html.

В большинство дистрибутивов сейчас входит **Ruby** (в переводе с англ. “рубин”) — перспективный язык программирования, в котором сохранена мощь языка **Perl** и который при этом освобожден от его синтаксических “подводных камней”. Но главное то, что он обладает современными объектно-ориентированными свойствами. **Ruby** еще не стал для системных администраторов традиционным языком написания сценариев, но, по всей вероятности, станет таковым в ближайшие несколько лет.

Мы рекомендуем также изучить **expect**, который представляет собой скорее интерфейс для управления интерактивными программами, а не язык программирования. По сути, это эффективная интегрирующая технология, которая может заменить традиционный процесс написания сценариев. Освоить **expect** можно достаточно быстро.

Все самое важное, что необходимо знать о написании сценариев на языках **bash**, **Perl** и **Python**, можно почерпнуть в главе 2. В ней также рассматриваются регулярные выражения (шаблоны, задающие правило поиска) и некоторые идиомы оболочки, полезные для системных администраторов.

1.3. Взаимосвязь систем Linux и UNIX

Подробнее об истории Linux и UNIX читайте в разделе “Краткая история системного администрирования”.

В этой книге описываются как UNIX-, так и Linux-системы ввиду их схожести. Однако употребление названий Linux и UNIX в одном предложении чем-то сродни хождению по политическому минному полю или зыбучим пескам. Тем не менее мы возьмем на себя смелость максимально лаконично и объективно изложить ряд фактов. Однако, поскольку отношения между системами UNIX и Linux, мягко говоря, носят запутанный характер, мы не можем обойти молчанием создавшуюся ситуацию и возьмем на себя смелость максимально лаконично и объективно изложить ряд фактов.

Linux — измененная и усовершенствованная реализация ядра операционной системы UNIX. Она соответствует стандарту POSIX, работает на различных аппаратных платформах и совместима с большинством существующих приложений UNIX. Отличие от множества других (но не всех) вариантов UNIX заключается в том, что Linux — бесплатная операционная система с открытым исходным кодом, разрабатываемая коллективными усилиями тысяч энтузиастов и организаций. Вместе с тем, операционная система Linux содержит ряд технических усовершенствований, которые отсутствуют в UNIX, и поэтому представляет собой нечто большее, чем простой клон UNIX. В то же время традиционные производители UNIX не перестают заниматься усовершенствованием своих продуктов, что говорит о наличии областей, в которых коммерческие UNIX-системы превосходят Linux-системы.

Linux совершенно законно может считаться отдельным программным продуктом, который уже неправильно называть “UNIX” или версией “UNIX”. В то же время было бы ошибкой настаивать на том, что Linux — совсем “не UNIX”. Если ваше творение ходит как утка и крикает как утка, то, скорее всего, вы изобрели утку.

Раскольники нашлись даже в самом лагере Linux. Часто утверждают, и отчасти это справедливо, что ссылаться на дистрибутивы Linux просто как на “Linux” — значит не признавать труд разработчиков, вложенный в утилиты, выполняющиеся вне ядра (которые, по сути, составляют подавляющее большинство программ в средней системе). К сожалению, альтернативная версия, которую наиболее часто предлагают, — “GNU/Linux” — обременена собственным политическим багажом и официально поддерживается только дистрибутивом Debian. В статье Википедии “Спор об именовании GNU/Linux” приведены аргументы обеих сторон¹. Интересно отметить факт уже преобладающего использования открытых программных средств в большинстве UNIX-систем, но ни для одной из них, насколько нам известно, пока не предложено использовать название “GNU/UNIX”.²

Программы, написанные для Linux, являются UNIX-программами. Во многом благодаря проекту GNU наиболее значительные программные компоненты, составляющие основу UNIX-систем, разрабатываются теперь в соответствии с принципом открыто-

¹ Поскольку Википедия содержит информацию о Linux, а значит, она должна часто ссылаться на соответствующую статью, эта дискуссия имеет большое значение для самой Википедии. Рекомендуется также ознакомиться с содержимым вкладки Discussion для упомянутой статьи Википедии.

² В конце концов. “GNU — не UNIX”!

сти исходного кода.³ Один и тот же код может выполняться как в Linux, так и в любой другой однотипной среде. Веб-серверу Apache, к примеру, все равно, где работать: Linux или Solaris. С позиции коммерческих приложений и большинства программ администрирования Linux — это просто одна из наиболее поддерживаемых и самых доступных разновидностей UNIX.

Следует заметить, что Linux — не единственная свободно доступная UNIX-подобная операционная система. OpenSolaris также имеет открытый исходный код, хотя ее строго оговоренные сроки лицензирования кажутся подозрительными для некоторых борцов за чистоту проектов с открытым кодом. Системы FreeBSD, NetBSD и OpenBSD, являющиеся потомками BSD UNIX, имеют своих пылких сторонников. Эти операционные системы сопоставимы с Linux по функциональным возможностям и надежности, хотя и не пользуются такой популярностью у сторонних разработчиков программного обеспечения.

Как UNIX-, так и Linux-системы уже многие годы эффективно функционируют в промышленных условиях.⁴ Здесь выбор между ними должен делаться исходя из организации пакетирования, поддержки и корпоративных традиций, а не из реальных различий в качестве или современности.

В этой книге слово “Linux”, как правило, относится к дистрибутивам Linux, а не к традиционным версиям UNIX. Но слово “UNIX” здесь употребляется не всегда в одном и том же значении, и иногда мы применяем его к атрибутам, используемым всеми производными от UNIX системами, включая Linux (например, “разрешения UNIX-файла”). Если описываемые детали касаются как UNIX, так и Linux-систем, то во избежание двусмысленности мы обычно так и пишем: “UNIX и Linux”.

1.4. Дистрибутивы Linux

Все дистрибутивы основаны на едином семействе ядер, однако набор служебных программ, дополняющих ядро, может существенно варьироваться. Дистрибутивы различаются по своему назначению, наличию служб поддержки и степени популярности. В настоящее время существуют сотни независимых дистрибутивов Linux, но мы считаем, что дистрибутивы из семейств Debian, Red Hat и SUSE будут доминировать в производственных средах еще как минимум в течение ближайший пяти лет.

В целом между дистрибутивами Linux нет огромных различий. Остается загадкой: зачем столько? Причем отличительными свойствами каждого из них являются “простота установки” и “внушительная библиотека программных средств”. Нет-нет да и проскакивает мысль о том, что кто-то просто испытывает удовлетворение от выпуска новых дистрибутивов Linux.

На удивление, многие другие (более компактные) дистрибутивы так же конкурентоспособны по возможностям настройки. Все значительные дистрибутивы (в том числе

³ Некоторые наши технические рецензенты не соглашались с тем, что мы, как им казалось, связывали GNU с созданием самого открытого в мире программного обеспечения. Отнюдь нет! Однако участники проекта GNU определенно сделали больше, чем любая другая группа для распространения идеи открытого программного обеспечения как социальной инициативы и систематизации непрекращающегося спора о сроках лицензирования и взаимодействии между открытыми и неоткрытыми программными продуктами.

⁴ Под “производственной” средой мы понимаем среду, которую организация использует для выполнения реальных задач (в отличие от использования для тестирования, исследований или Разработки).

и второго яруса) включают относительно безболезненную процедуру инсталляции, изящную среду, реализующую концепцию рабочего стола, и форму управления пакетированием. Кроме того, большинство дистрибутивов позволяет выполнять загрузку с DVD-диска, что удобно для отладки и ознакомления с новым продуктом.

Поскольку эта книга сфокусирована на управлении крупномасштабными Linux-системами, мы больше склоняемся к дистрибутивам наподобие Red Hat, которые предусматривают управление сетями. Одни дистрибутивы создавались в расчете на промышленное применение, другие — нет. Какой-нибудь дополнительный программный компонент системы промышленного уровня, на первый взгляд малозначительный, в действительности существенно упрощает администрирование.

Принимая решение в пользу того или иного дистрибутива, обращайтесь внимание не только на его функциональные возможности, но и на то, как будет протекать сотрудничество вашей организации и поставщика дистрибутива в ближайшие годы.

Задайте себе ряд важных вопросов.

- Будет ли дистрибутив существовать в ближайшие пять лет?
- Будут ли оперативно устраняться бреши в системе защиты?
- Будут ли оперативно выпускаться новые версии программных продуктов?
- Будет ли оказана помощь в решении возникших проблем?

Под этим углом зрения некоторые чрезвычайно интересные, компактные дистрибутивы уже не кажутся столь привлекательными. Но не стоит сбрасывать их со счетов: например такая компания, как E*Trade, работает в системе Gentoo Linux.

Наиболее жизнеспособные дистрибутивы не обязательно имеют корпоративный статус. Например, мы ожидаем, что система Debian Linux (простите, Debian GNU/Linux!) просуществует довольно долго, несмотря на то что Debian — это не коммерческая компания, она ничего не продает и не предлагает сервисного обслуживания. Несмотря на то что Debian не входит в число широко используемых дистрибутивов, эта система извлекает пользу из группы спонсоров и огромной популярности дистрибутива Ubuntu, который основан на системе Debian.

Наиболее популярные дистрибутивы общего назначения перечислены в табл. 1.1.

Таблица 1.1. Наиболее популярные дистрибутивы Linux общего назначения

Дистрибутив	Веб-узел	Комментарии
CentOS	centos.org	Бесплатный аналог Red Hat Enterprise Linux
Debian	debian.org	Популярный некоммерческий дистрибутив
Fedora	fedoraproject.org	Вариант Red Hat Linux, предназначенный для использования отдельными пользователями
Gentoo	gentoo.org	Оптимизированный дистрибутив, предназначенный для самостоятельной компиляции
Linux Mint	linuxmint.com	Компактный дистрибутив, основанный на Ubuntu
Mandriva	mandriva.com	Один из наиболее дружелюбных по отношению к пользователю дистрибутивов
openSUSE	opensuse.org	Бесплатный аналог SUSE Linux Enterprise
Oracle Enterprise Linux	oracle.com	Версия RHEL, поддерживаемая системой Oracle
PLinuxOS	pclinuxos.com	Ответвление от Mandriva, ориентированное на KDE
Red Flag	redflag-linux.com	Ведущий дистрибутив в Китае, подобный Red Hat

Окончание табл. 11

Дистрибутив	Веб-узел	Комментарии
Red Hat Enterprise	redhat.com	Коммерческий дистрибутив Red Hat, отличающийся высокой надежностью
Slackware	slackware.com	Один из старейших дистрибутивов
SUSE Linux Enterprise	novell.com/linux	Многоязыковой дистрибутив, особенно популярен в Европе
Ubuntu	ubuntu.com	Доработанная версия Debian

Обновляемый перечень дистрибутивов Linux (в том числе и не англоязычных) можно найти по следующим адресам: linux.org/dist, lwn.net/Distributions и distrowatch.com.

1.5. Примеры систем, используемых в этой книге

В этой книге мы остановились на трех популярных дистрибутивах Linux и трех версиях UNIX: Ubuntu Linux, openSUSE, Red Hat Enterprise Linux, Solaris, HP-UX и AIX. Они представляют собой срез рынка корпоративных систем и охватывают большинство инсталляций в крупных организациях.

Приведенная в книге информация относится, как правило, ко всем упомянутым выше дистрибутивам, если нет специальных уточнений. Подробности, касающиеся конкретного дистрибутива, помечаются значком поставщика.

Ubuntu® 9.10 “Karmic Koala”

openSUSE® 11.2

Red Hat® Enterprise Linux® 5.5

Solaris™ 11 and OpenSolaris™ 2009.06

HP-UX® 11i v3

AIX® 6.1

Эти значки использованы с разрешения их владельцев. Следует заметить, что разработчики приведенных дистрибутивов не рецензировали эту книгу. Подробнее о каждой из упомянутых систем описано в следующих разделах.

Примеры Linux-дистрибутивов

Информация, относящаяся именно к Linux-системам, но не к какому-то конкретному дистрибутиву, отмечается логотипом с изображением талисмана Linux — пингвина Такса (Tux).

Дистрибутивы Ubuntu сохраняют идеологическую направленность на разработку членами сообщества пользователей и разработчиков и открытый доступ,

поэтому вопрос о том, какие части дистрибутива бесплатны или разрешены для дальнейшего распространения, даже не возникает. Ubuntu в настоящее время финансируется за счет благотворительных пожертвований южноафриканского предпринимателя Мака Шаттлворта (Mark Shuttleworth).

Ubuntu основан на дистрибутиве Debian и использует его систему пакетирования. Он выпускается в двух основных вариантах: Desktop Edition и Server Edition. В сущности, они идентичны, но ядро Server Edition предварительно настроено для использования на сервере и не устанавливает графический интерфейс (GUI) или такие GUI-приложения, как OpenOffice.

Компания SUSE, которая теперь является одним из подразделений корпорации Novell, пошла по пути Red Hat и начала распространять два связанных дистрибутива: openSUSE, который содержит только бесплатное программное обеспечение, и платный SUSE Linux Enterprise, который включает средства формальной поддержки и предоставляет несколько дополнительных возможностей. Эта книга не содержит никакой информации, характерной для того или иного дистрибутива SUSE, поэтому мы называем их всех просто “SUSE”.

В течение более десяти последних лет Red Hat занимает ведущее положение среди вариантов Linux, и его дистрибутивы наиболее популярны в Северной Америке. В 2003 году первоначальный дистрибутив Red Hat Linux был разделен на серию версий, ориентированных на производственные среды, которые получили название Red Hat Enterprise Linux (в этой книге мы иногда употребляем аббревиатуру RHEL), и на версии, разрабатываемые в рамках проекта с привлечением всех членов сообщества пользователей и разработчиков, который получил название Fedora. Это разделение было обусловлено рядом технических, экономических, логических и юридических причин.

Сначала эти дистрибутивы были схожими, но за последние пять лет Fedora претерпела значительные изменения, и теперь эти две системы не синхронизированы. RHEL отличается высоким уровнем поддержки и стабильностью, но ее по существу невозможно использовать, не приобретя лицензию в компании Red Hat.

Проект CentOS (centos.org) концентрирует исходный код, который компания Red Hat обязана распространять в соответствии с различными лицензионными соглашениями (наиболее значимой является общедоступная лицензия GNU), и комплектует их в законченный дистрибутив, который во многом подобен дистрибутиву Red Hat Enterprise Linux, но доступен бесплатно. Этот дистрибутив лишен торговой марки компании Red Hat и не содержит некоторых запатентованных программных средств, но в остальном аналогичен платной версии. CentOS стремится к полной совместимости с RHEL, как по двоичному коду, так и по степени исправления ошибок.

Этот дистрибутив — прекрасный выбор для тех сайтов, которые стремятся развернуть крупномасштабную производственную систему, не платя при этом “десятину” компании Red Hat. Возможен также смешанный подход: сетевые серверы могут работать под управлением Red Hat Enterprise Linux, используя при этом преимущества, предоставляемые прекрасным уровнем поддержки со стороны Red Hat, а рабочие станции могут функционировать под управлением CentOS. Такой подход обеспечивает оптимизацию с точки зрения риска и поддержки, в то же время сводя к минимуму затраты и сложность администрирования.

Примеры UNIX-дистрибутивов

Solaris — операционная система, построенная на основе System V и “обросшая” множеством расширений. Разработана компанией Sun Microsystems, которая ныне принадлежит корпорации Oracle.⁵ Изначально Sun UNIX (так эта система называлась в середине 80-х гг.) — потомок Berkeley UNIX, но в результате корпоративного партнерства Sun и AT&T в системе была изменена кодовая основа. Solaris работает на различных аппаратных платформах, в частности на Intel x86 и SPARC.

Под “крышей” компании Sun систему Solaris можно было свободно загружать и использовать. Но корпорация Oracle изменила условия лицензирования, и теперь для загружаемых версий устанавливается бесплатный пробный период (90 дней). Наличие же OpenSolaris (свободно распространяемой версии Solaris с открытым исходным кодом) явно усугубляет сложившуюся ситуацию. На момент написания книги (середина 2010 г.) планы Oracle в отношении Solaris и OpenSolaris были неясны.

Выпуск системы Solaris 11, очень похожей на систему OpenSolaris, ожидался уже в 2010 году. В этой книге, употребляя слово “Solaris”, мы имеем в виду смешанную систему, основанную на выпусках Solaris 10 и OpenSolaris (все данные получены от нашей разветвленной сети сильно засекреченных шпионов, действующих в недрах Oracle). В определенных случаях при необходимости мы конкретно указываем названия Solaris 10 или OpenSolaris.

Система HP-UX основана на System V и привязана к аппаратным платформам Hewlett-Packard. По исходному коду она ближе к родительскому генеалогическому дереву, чем Solaris или AIX, но HP сохранила темп ведения разработок в мире операционных систем и расширила круг собственных усовершенствований. Теперь, когда HP начала поддержку и Linux-систем, будущее HP-UX несколько затуманено.

AIX — операционная система компании IBM, изначально построенная на базе BSD 4.2 (Berkeley UNIX), но уже в версии AIX 4 (1994 г.) большинство ее компонентов перешло в System V. В настоящее время систему AIX “отнесло” довольно далеко от обоих источников.

В целом, у нас создалось впечатление, что перекрестное опыление от других систем характерно для AIX в меньшей степени, чем для большинства вариантов UNIX. Вместе с тем мы считаем, что эта система попала под “дьявольское” влияние некоторых мэйнфреймовых и среднего класса (AS/400) операционных систем IBM, от которых она унаследовала такие компоненты, как Object Data Manager (см. раздел 13.6), команды конфигурации (вместо файлов конфигурации) и административный интерфейс SMIT. Со временем эта система, возможно, станет более самобытной.

Интересно отметить, что в последние десять лет IBM демонстрирует ОС-инвариантный (т.е. независимый от операционной системы) подход к маркетингу своих аппаратных средств. IBM, продолжая разработку и продвижение AIX, также заинтересована в сотрудничестве с Red Hat и Novell, чтобы убедиться в устойчивой работе их дистрибутивов Linux на оборудовании IBM. Посмотрим, как этот подход проявит себя в будущем.

⁵ Подробнее о системах BSD, System V и истории возникновения UNIX читайте в разделе “краткая история системного администрирования”.

1.6. Административные средства конкретных систем

Современные системы включают графические средства и панели управления (такие, как YaST2 в SUSE и SMIT в IBM), упрощающие конфигурирование и администрирование определенных компонентов системы. Эти средства очень удобны, особенно администраторам-новичкам, но они недостаточно полно отражают возможности базового программного обеспечения.

В этой книге мы описываем низкоуровневые механизмы, а не высокоуровневые инструментальные средства. На то есть ряд причин. Во-первых, графические средства зачастую являются платными или по крайней мере зависят от используемой системы. В результате то, что на низком уровне является одинаковым для всех систем, начинает казаться хаотичным и непостоянным. Во-вторых, мы считаем, что для системного администратора важно четко знать, как работает система. Когда в системе происходит сбой, графические утилиты не всегда помогают найти и устранить проблему. Наконец, конфигурировать систему на низком уровне обычно удобнее: так быстрее и надежнее, больше возможностей для манипуляции, легче писать сценарии.

1.7. Типографские особенности книги

Имена файлов, команды и аргументы команд, которые следует набирать на клавиатуре без изменений, даны полужирным шрифтом. Аргументы, вместо которых следует подставлять конкретные значения, даны курсивом. Например, в команде

ср *файл каталог*

предполагается, что аргумент *файл* следует заменить именем реального файла, а аргумент *каталог* — именем реального каталога.

Результаты работы команд, а также фрагменты сценариев и файлов конфигурации даны моноширинным шрифтом. Комментарии к интерактивным сеансам иногда даются курсивом, например:

```
% grep Bob /pub/phonelist      # Найти номер телефона Боба
Bob Knowles 555-2834
Bob Smith 555-2311
```

При описании синтаксиса команд мы, как правило, используем те же обозначения, что и в интерактивном руководстве:

- текст, заключенный в квадратные скобки (“[” и “]”), является необязательным;
- текст, после которого стоит многоточие (“...”), можно повторять;
- фигурные скобки (“{” и “}”) указывают на то, что необходимо выбрать один из элементов, разделенных вертикальной чертой (“|”).

Например, спецификации

```
bork [-x] {on|off} имя_файла...
```

соответствует любая из следующих команд:

```
bork on /etc/passwd
bork -x off /etc/passwd /etc/termcap
bork off /usr/lib/tmac
```

В выражениях с шаблонами поиска используются следующие метасимволы:

- звездочка (*) обозначает нуль или более символов;
- знак вопроса (?) обозначает один символ;

- тильда (~) обозначает начальный каталог текущего пользователя⁶;
- выражение *~пользователь* обозначает начальный каталог указанного пользователя.

Например, иногда мы обозначаем каталоги, где хранятся сценарии запуска (*etc/rc*.d, etc/rc*.d* и т.д.), сокращенным шаблоном *etc/rc*.d*.

1.8. Единицы измерения

Такие метрические приставки, как кило-, мега- и гига-, определяются показателями степени числа 10 (например, один мегагерц составляет тысячу килогерц или миллион герц). Эти приставки “заимствованы” и для компьютерных типов данных, но с использованием степеней числа 2. Например, один “мегабайт” памяти составляет 2²⁰ или 1 048 576 байт. Ассоциация твердотельных технологий (Solid State Technology Association) Объединенного инженерного совета по электронным устройствам (Joint Electronic Device Engineering Council — JEDEC) даже превратила эти “заимствованные” единицы измерения в такой официальный стандарт, как Standard 100B.01, который признает их степенями двойки (не без некоторых натяжек).

Пытаясь восстановить справедливость (или внести ясность), Международная электротехническая комиссия (International Electrotechnical Commission — IEC) определила ряд числовых приставок, основанных именно на степенях числа 2: киби- (kibi-), меби- (mebi-), гиби- (gibi-) и так далее с аббревиатурами КиБ (Ki), МиБ (Mi) и ГиБ (Gi) соответственно. С вводом этих единиц измерения ликвидируется двусмысленность, но их только начинают использовать. Поэтому привычный набор кило-, мега- и гига-префиксов все еще в ходу (в обоих значениях: десятичном и двоичном).

Определить конкретное значение можно по контексту Объем ОЗУ всегда измеряется в степенях двойки, но пропускная способность сети — в степенях числа 10. Размер дисков их производители обычно указывают в десятичных единицах, а размер блоков и страниц — в двоичных.

В этой книге мы используем единицы измерения МЭК (IEC) для степеней двойки и метрические — для степеней числа 10. Метрические единицы измерения мы применяем также для приблизительных значений и в тех случаях, когда точное основание степени неясно, недокументировано или его невозможно определить. В командах файлов конфигурации *output* и *in* мы оставляем исходные значения и указатели единиц измерений. Для обозначения бита служит буква **b**, а для байта — буква **B**. Некоторые примеры интерпретации единиц измерения приведены в табл. 1.2.

Таблица 1.2. Примеры интерпретации единиц измерения

Пример	Описание возможного варианта
56 кбит/с (kb/s)	Скорость передачи данных по последовательной линии связи, равная 56 000 бит в секунду
1 Кбайт(кВ)	Размер файла, равный 1000 байт
4 Кибайт (KiВ)	Общий размер страниц полупроводникового диска (SSD) 4 096 байт
8 Кбайт (KB)	Размер памяти — не используется в этой книге (см. описание ниже)
100 Мбайт (MB)	Ограничение по размеру файла (неоднозначность, понимается по контексту) номинально составляет 10 ⁸ байт

⁶ В дистрибутиве Solaris 10 в качестве стандартной оболочки для пользователя *root* используется оригинальная оболочка Bourne, которая (что удивительно) не воспринимает метасимвол *~* или *~пользователь*.

Окончание табл. 1.2

Пример	Описание возможного варианта
100 Мбайт (MB)	Размер раздела диска (понимается по контексту) номинально составляет 10 ⁸ байт, возможно, 99 999 744 байт ^а
1 Гбайт (GiB)	Размер ОЗУ, в точности равный 1 073 741 824 байт ^б
1 Гбит/с (Gb/s)	Скорость передачи информации в сети Ethernet составляет 1 гигабит в секунду (1 000 000 000 бит в секунду)
1 Тбайт (TB)	Объем жесткого диска составляет 1 терабайт (один триллион байт), или 1 000 000 000 000 байт

^а Число 10⁸ округлено в меньшую сторону до ближайшего целого, кратного размеру 512-байтового блока диска.

^б По заявлению компании Microsoft, этого недостаточно для функционирования 64-разрядной версии Windows 7.

Аббревиатура *K*, как в случае “8 Кбайт (KB)”, не является частью стандарта. Это компьютерная адаптация метрической аббревиатуры *к* (обозначение приставки кило-), которая означает 1 024 в отличие от 1 000. Но поскольку в аббревиатурах для многих метрических приставок уже используется прописная буква, стала возникать путаница при новом и исходном использовании буквы *K* в качестве множителя 1 000.

В дистрибутиве Ubuntu Linux реализуется героическая попытка внести здравый смысл в этот вопрос и ликвидировать противоречивость (подробнее см. wiki.ubuntu.com/UnitsPolicy).

1.9. Использование справочных страниц и другой оперативной документации

Справочные страницы онлайн-руководства (так называемые man-страницы, поскольку их можно просмотреть с помощью команды `man`) составляют традиционную “интерактивную” документацию (хотя, конечно, в настоящее время практически вся документация в той или иной мере является интерактивной). Эти материалы в основном устанавливаются вместе с системой, а справочной страницы отдельных программ — вместе с соответствующим пакетом.

Интерактивное руководство содержит краткое описание отдельных команд, драйверов, форматов файлов и библиотечных функций. В них не найти ответа на общие вопросы, например, “Как установить новое устройство?” или “Почему моя система работает так медленно?”. Для получения ответов на подобные вопросы обращайтесь к руководствам по администрированию систем соответствующих производителей (см. раздел 1.10) или, если вопрос касается систем Linux, к документам, находящимся в ведении проекта Linux Documentation Project (LDP).

Организация справочных страниц интерактивного руководства

Во всех системах справочные страницы обычно делятся на разделы, при этом возможны небольшие вариации на тему принципов такого разделения. Основная схема разделения, используемая в наших примерах систем, показана в табл. 1.3.

Таблица 1.3. Разделы справочных страниц

linux	Solaris	HP-UX	AIX	Содержание
1	1	1	1	Команды пользовательского уровня и приложения
2	2	2	2	Системные вызовы и коды ошибок ядра
3	3	3	3	Библиотечные функции
4	7	7	4	Драйверы устройств и сетевые протоколы
5	4	4	5	Стандартные форматы файлов
6	6	-	6	Игры и демонстрационные программы
7	5	5	7	Различные файлы и документы
8	1m	1m	8	Команды системного администрирования
9	9	-	-	Внутренние интерфейсы и спецификации ядра
-	-	9	-	Общая информация по HP-UX

Некоторые разделы делятся на подразделы. Например, подраздел 3с руководства по Solaris содержит страницы с информацией о библиотеке стандартных C-функций системы. В некоторых системах раздел 8 остается пустым, а описание команд, относящихся к системному администрированию, как правило, содержится в разделе 1. Игры и демонстрационные программы во многих системах уже удалены из раздела 6. Раздел руководства под названием “l” (прописная буква от “L”) зачастую отводится для локальных справочных страниц.

Точная структура разделов не так уж важна, поскольку команда `man` находит соответствующую страницу в большинстве случаев. При этом в нескольких разделах может быть одно и то же имя, и вот тут важно указать нужный вам раздел. Например, `passwd` — это и команда, и файл конфигурации, поэтому соответствующие статьи есть как в разделе 1, так и разделе 4 или 5.

Команда `man` : чтение страниц интерактивного руководства

Команда `man` *заголовок* форматирует конкретную страницу интерактивного руководства и выводит ее на терминал пользователя посредством `more`, `less` или другой программы постраничной разбивки, которая задана в переменной среды `PAGER`. Аргумент *заголовок* — это, как правило, имя команды, устройства или файла, о которых необходимо получить справочную информацию. Поиск по разделам руководства осуществляется в порядке возрастания номеров, хотя разделы, посвященные описанию команд (1,8 и 6), обычно просматриваются в первую очередь.

Команда `man` *раздел заголовок* вызывает справочную страницу из указанного раздела. Так, в большинстве систем команда `man sync` вызывает справочную страницу для команды `sync`, а команда `man 2 sync` — для системного вызова `sync`.

В системе Solaris необходимо перед номером раздела использовать флаг `-s`, например `man -s 2 sync`.

Команда `man -k` *ключевое_слово* или `apropos` *ключевое_слово* выводит список справочных страниц, в строке пояснений к которым имеется указанное ключевое слово.

```
% man ~k translate
objcopy (1)      - copy and translate object files
dcgettext (3)    - translate message
```

```
tr (1)                - translate or delete characters
snmptranslate (1)    - translate SNMP OID values into more useful information
tr (lp)              - translate characters
```

База данных ключевых слов может устаревать. При добавлении новых справочных страниц к системе вам, возможно, придется перестроить (перекомпоновать) этот файл с помощью команд `mandb` (Ubuntu, SUSE), `makewhatis` (Red Hat) или `catman -w` (Solaris, HP-UX, AIX).

Хранение страниц интерактивного руководства

Неформатированная информация для справочных страниц (входные данные команды `nroff`) обычно хранится в подкаталогах каталога `/usr/share/man`. В целях экономии места на диске системы Linux сжимают страницы с помощью утилиты `gzip` (команда `man` может очень быстро разархивировать их). Команда `man` поддерживает кеш отформатированных страниц в каталогах `/var/cache/man` или `/usr/share/man`, если соответствующие каталоги доступны для записи, но эти операции рискованны с точки зрения безопасности. В большинстве систем предварительное форматирование справочных страниц выполняется однократно во время инсталляции (см. команду `catman`) или не выполняется совсем.

В дополнение к традиционной команде `nroff` система Solaris воспринимает справочные страницы, отформатированные с использованием обобщенного языка разметки SGML. SGML-страницы имеют собственные каталоги раздела в каталоге `/usr/share/man`.

Команда `man` ищет страницы в ряде каталогов. В Linux-системах выяснить путь поиска позволяет команда `manpath`. Результат ее работы (в системе Ubuntu) обычно таков.

```
ubuntu$ manpath
/usr/local/man:/usr/local/share/man:/usr/share/man
```

Эта установка хранится в переменной среды `MANPATH`, и в случае необходимости ее можно изменить.

```
export MANPATH=/home/share/localman:/usr/share/man
```

Некоторые системы позволяют установить общесистемный параметр пути поиска для справочных страниц, который может оказаться полезным в случае, если вам придется поддерживать параллельное дерево справочных страниц, например, сгенерированных кроссплатформенным менеджером пакетов `OpenPKG`. Но если вы хотите распространять локальную документацию в виде справочных страниц, проще всего использовать стандартный системный механизм пакетирования и выложить справочные страницы в стандартные справочные каталоги. Подробнее об этом написано в главе 12.

GNU-система Texinfo

Системы Linux включают дополнительный вариант интерактивных справочных страниц, именуемый `Texinfo`. Система `Texinfo` была изобретена энтузиастами проекта GNU после того, как выяснилось, что команда `nroff`, применяемая для форматирования справочных страниц, является собственностью AT&T. Теперь то же самое делает GNU-команда `groff`, так что проблема утратила актуальность, хотя `Texinfo` все еще бродит как зомби в поисках человеческих мозгов.

Несмотря на то что использование системы Texinfo, казалось бы, постепенно сходит на нет, некоторые GNU-пакеты требуют, чтобы документация к ним была представлена именно в формате Texinfo. Чтобы обойти встроенную навигационную систему Texinfo, перенаправьте поток вывода команды `info` посредством команды `less`.

Но есть и приятный момент. Пакеты, документируемые с помощью системы Texinfo, обычно устанавливают страницы-заглушки, в которых сообщается о том, что информацию о соответствующем пакете можно получить, вызвав команду `info`. Таким образом, можно продолжать пользоваться командой `man`, обращаясь к команде `info` лишь в случае необходимости. Команда `info info` выдает справку по запутанной системе Texinfo.

1.10. Другие виды официальной документации

Справочные страницы — это лишь малая часть официальной документации. Остальная, в основном, рассеяна в веб-пространстве.

Руководства по конкретным системам

Одни производители систем ведут собственные онлайн-проекты по подготовке документации, другие выпускают полезные руководства в виде объемных книг. В настоящее время нужное руководство быстрее найти в Интернете, чем в форме печатного издания. Объем и качество документации бывают разными, но большинство производителей выпускает, по меньшей мере, руководство по администрированию и установке системы. Где найти документацию по каждому из наших примеров систем, показано в табл. 1.4.

Безусловно, нельзя не выделить компанию IBM, которая выпускает множество толстых книг по различным темам администрирования систем. Вы можете купить их или даже бесплатно скачать на свой компьютер. Но, к сожалению, многие документы от IBM отстают на одну или две версии от текущего выпуска AIX.

Таблица 1.4. Где найти документацию от производителей операционных систем

Система	URL	Описание
Ubuntu	help.ubuntu.com	Наиболее дружелюбная к пользователю; см. раздел “server guide”
SUSE	novell.com/documentation	Материал по администрированию собран в разделе “reference guide”
RHEL	redhat.com/docs	В основном, документы по расширениям системы Red Hat
Solaris	docs.sun.com	Громадный каталог материалов
HP-UX	docs.hp.com	Книги, технические описания и руководства
AIX	www.redbooks.ibm.com ibm.com/support	Многочисленные книги в формате PDF Доступ к комментариям, разделам FAQs и пр.

В “соревнованиях” по созданию документации система Red Hat, к сожалению, отстает от лидеров. Большая часть выпускаемых документов касается не общих вопросов администрирования Linux-систем, а собственных систем с дополнительными позитивными характеристиками.

Документация по конкретным пакетам

Большинство важнейших программных пакетов в мире UNIX и Linux поддерживают отдельные лица или такие сторонние организации, как Internet Systems Consortium и Apache Software Foundation. Сотрудники этих групп пишут собственную документацию, качество которой часто оставляет желать лучшего. При этом существуют и такие прекрасно выполненные документы, как Version Control with Subversion от svnbook.red-bean.com (оказывается, надо знать, где искать!).

Производители систем UNIX и дистрибуторы систем Linux всегда включают в свои пакеты соответствующие справочные страницы. К сожалению, многие из них стараются сэкономить на документации другого типа, главным образом, ввиду отсутствия стандартного места для ее представления (см. `/usr/share/doc`). Всегда имеет смысл посетить веб-сайт первоисточника программного продукта и поинтересоваться наличием дополнительных материалов и доступа к ним.

В число дополнительных документов входят: технические отчеты (техническое описание), проектные обоснования и трактовки конкретных тем. Эти дополнительные материалы зачастую не ограничиваются простым описанием команд и могут включать самоучители и другие разделы. Многие программные пакеты, помимо справочных страниц, содержат и статьи по соответствующим темам. Например, после вызова справочной страницы для редактора `vi` вам будет предложено не только ознакомиться с аргументами командной строки, но и перейти к разделу, из которого вы узнаете, как в действительности отредактировать файл.

Книги

К самым лучшим источникам информации для системных администраторов в книжном мире (в дополнение к этой книге :-), конечно) можно отнести серию книг Тима О'Рейли (Tim O'Reilly). Начало этой серии было положено более 20 лет назад книгой *UNIX in a Nutshell*. Теперь же практически всем важным подсистемам UNIX и Linux (и даже командам) посвящены ее отдельные тома. Эта серия также включает книги по Интернету, Windows и другим темам, не связанным с UNIX. Все эти книги имеют приемлемую цену, своевременны и ориентированы на конкретную аудиторию.

Тим О'Рейли, заинтересовавшись движением, основанным на принципе открытого исходного кода, регулярно проводит конференции (OSCON) по этой и другим злободневным темам. Конференции OSCON проводятся дважды в год (по одной в США и Европе). Подробнее см. на сайте oreilly.com.

Документы RFC и другие интернет-ресурсы

Документы из серии RFC (Request for Comments — запрос на комментарий) описывают протоколы и процедуры, используемые в Интернете. Большинство из этих документов достаточно детализированы и специализированы, но некоторые написаны в виде обзоров. Информации, которую они содержат, вполне можно доверять, и она во многом оказывается полезной для системных администраторов. Более полное описание этих документов можно найти в разделе 14.1.

Проект документации по Linux

Для систем Linux существует и такой источник справочной информации, как проект Linux Documentation Project (LDP), доступный на веб-сайте tldp.org. Этот сайт пред-

ставляет собой центральное хранилище разного рода информации, посвященной Linux: от FAQ (часто задаваемые вопросы) до объемных руководств. Здесь же концентрируются результаты усилий по переводу документации на различные языки.

К сожалению, многие LDP-документы практически не обновляются, поэтому быстро устаревают. Всегда обращайтесь внимание на дату публикации документа, так как по ней можно судить об актуальности информации.

1.11. Другие источники информации

Источники информации, рассмотренные в предыдущем разделе, можно охарактеризовать как самые надежные, но вряд ли это последнее слово в документации по UNIX и Linux. В Интернете же можно найти многочисленные блоги, дискуссионные форумы и новостные веб-каналы и получить с их помощью самую “свежую” информацию.

Как и следовало ожидать, система Google стала лучшим другом системного администратора. Именно к Google многие администраторы обращаются за консультацией, если вопрос не касается спецификации конкретной команды или формата файла (но и в этом случае Google обязательно поможет). Возьмите себе за правило (если не хотите впустую терять время и репутацию) при возникновении затруднений выносить свои вопросы на онлайн-форумы, ссылающиеся при ответах на Google.⁷

Невозможно перечислить все полезные интернет-источники информации по Linux, поэтому упомянем лишь важнейшие из них (табл. 1.5).

Можем порекомендовать еще один забавный и полезный ресурс. Это страница “Rosetta Stone” (розеттский камень) Брюса Гамильтона (Bruce Hamilton), доступная по адресу bhami.com/rosetta.html. Она содержит указатели на команды и утилиты, применяемые для решения основных административных задач в различных операционных системах.

Если вы работаете в Linux-системе, не стесняйтесь обращаться к UNIX-ресурсам общего назначения. Большая часть информации в них напрямую применима к Linux.

Таблица 1.5. Веб-ресурсы, посвященные системному администрированию

Веб-сайт	Описание
blogs.sun.com	Огромная коллекция технических статей, посвященных, в основном, Solaris
cpan.org	Заслуживающая доверия коллекция Perl-модулей
freshmeat.net	Большой каталог программ для Linux и UNIX
kernel.org	Официальный веб-сайт разработчиков ядра Linux
linux.com	Linux-форум, прекрасный сайт для новичков
linux.org	Неофициальное хранилище информации по Linux
linux.slashdot.org	Огромный архив новейшей справочной технической информации по Linux
linuxhq.com	Хранилище информации по ядру и “заплат” к нему
lwn.net	Сборник материалов по Linux и программам с открытым исходным кодом
lxxer.com	Агрегатор новинок по Linux
rootvg.net	Сайт, ориентированный на системы AIX с множеством ссылок и форумов
securityfocus.com	Хранилище информации по общим вопросам компьютерной безопасности
serverfault.com	Совместно редактируемая база данных вопросов от системных администраторов

⁷ Или, в худшем случае, ссылающиеся на Google через lmgtyf.com.

Окончание табл. 1.5

Веб-сайт	Описание
ServerFiles.com	Каталог сетевых программных и аппаратных средств
slashdot.org	Хранилище новейшей справочной технической информации по различным темам
solariscentral.org	Открытый блог с новостями и статьями по Solaris
sun.com/bigadmin	Сайт, содержащий информацию по администрированию Sun-систем
sunhelp.org	Прекрасная коллекция Sun-материалов
ugu.com	Сайт “Вселенная гуру UNIX” (UNIX Guru Universe) с информацией для системных администраторов

^a Сейчас его поддерживает Linux Foundation — некоммерческий консорциум развития Linux.

1.12. Как устанавливать программное обеспечение

Подготовка к работе программного обеспечения подробно рассматривается в главе 12. Но для самых нетерпеливых мы прямо здесь расскажем о том, как выяснить, что уже установлено в вашей системе, и как получить новое программное обеспечение и установить его.

В современных операционных системах программное обеспечение разделено на пакеты, которые можно устанавливать независимо друг от друга. При стандартной установке системы используется группа “стартовых” пакетов, которую можно при необходимости расширить.

Добавочные программные продукты зачастую предоставляются также в виде предварительно скомпилированных пакетов, но далеко не во всех системах. Большая часть программного обеспечения создается независимыми группами разработчиков, выпускающих программы в виде исходных кодов. Репозитории пакетов затем берут исходные коды, компилируют их в соответствии с особенностями конкретной системы и включают в пакеты полученные бинарные файлы. Как правило, намного проще установить бинарную версию пакета, предназначенную для данной системы, чем искать и компилировать исходный код. Однако нужно учитывать, что иногда создатели пакетов на один-два выпуска отстают от текущей версии системы.

Если в двух системах используется один и тот же формат обработки пакетов, то это еще не означает, что пакеты обеих систем будут взаимозаменяемыми. Например, в системах Red Hat и SUSE применяется формат RPM, однако структура их файловых систем неодинакова. Рекомендуется всегда пользоваться пакетами, созданными для конкретной системы.

В известных Linux-дистрибутивах предусмотрены отлаженные системы управления пакетами, которые включают средства доступа к репозиториям программных продуктов и поиска их в Интернете. Дистрибуторы активно поддерживают эти репозитории от имени сообщества, которое они представляют, поэтому у администраторов Linux-систем редко возникает необходимость выходить за рамки стандартного менеджера пакетов. Другими словами, жизнь прекрасна!

При этом UNIX-системы демонстрируют больше неопределенности по части управления пакетами. Solaris, HP-UX и AIX предоставляют соответствующие программные средства работы с пакетами, которые функционируют на уровне отдельных машин. Однако производители этих систем не поддерживают репозитории продуктов с откры-

тым исходным кодом, и поэтому пользователям, в основном, приходится заниматься самообслуживанием.⁸ К сожалению, одним из главных звеньев цепочки, которая связывает систему пакетирования в одно целое, является метод ссылки одних пакетов на другие, позволяющий выразить информацию о взаимной зависимости и совместимости. Без центральной координации вся эта экосистема может быстро развалиться на отдельные части.

Действительность демонстрирует совершенно разные результаты. Дистрибутив Solaris предлагает добавочную систему (pkgutil от blastwave.org), которая предоставляет простую установку программного обеспечения из интернет-репозитория, во многом подобную системам, содержащимся на Linux-дистрибутивах. Система HP-UX содержит прекрасный интернет-репозиторий в форме HP-UX Porting и Archiving Centre (hpux.connect.org.uk), но пакеты должны быть загружены вручную и по отдельности. Что касается AIX, то пока трудно говорить о возможности использования предварительно скомпонованного программного обеспечения для этой системы.

Администраторы без доступа к предварительно скомпонованным бинарным версиям пакетов должны устанавливать системы по-старому, т.е. загрузить архив исходного кода tar, а затем вручную сконфигурировать, скомпилировать и установить систему. Иногда этот процесс проходит гладко, а иногда может превратиться в кошмар: все зависит от используемого программного обеспечения и конкретной операционной системы.

В этой книге мы предполагаем, что дополнительное программное обеспечение уже установлено, и не собираемся мучить вас шаблонными инструкциями по установке каждого пакета. При необходимости мы будем указывать точные названия пакетов для выполнения конкретного проекта. Однако большей частью мы не будем повторять инструкции по установке, поскольку они практически идентичны для всех пакетов.

Определение факта установки программного обеспечения

По ряду причин иногда нелегко определить, какой программный пакет содержит нужный компонент. Для того чтобы выяснить, “прописан” ли важный для вас бинарный файл в строке поиска, лучше использовать команду `which` (а не действовать на пакетном уровне). Например, следующая команда сообщает о том, что компилятор GNU C установлен на данном компьютере.

```
aix$ which gcc
/opt/pware/bin/gcc
```

Если команда `which` не помогла, воспользуйтесь командой `whereis`, которая ведет поиск в системных каталогах без учета строки поиска. Другой вариант — это чрезвычайно удобная команда `locate`, которая просматривает предварительно скомпилированный индекс файловой системы в поиске имен файлов, соответствующих заданному шаблону. Команда `locate` входит в состав GNU-пакета `findutils`, который по умолчанию включен в большинство систем Linux, но для UNIX его необходимо устанавливать вручную.

С помощью команды `locate` можно искать не только программы или пакеты, но и файлы любых типов. Например, если не известно точно, где искать файл заголовков `signal.h` (в нем содержатся определения сигналов Linux), попробуйте поступить так.

⁸ Разработчики OpenSolaris не предлагают систему управления пакетами и интернет-репозиторий. Хотя этих средств нет в Solaris 10, ими предполагается оснастить Solaris 11.

```
ubuntu$ locate signal.h
/usr/include/signal.h
/usr/include/asm/signal.h
/usr/include/asm-generic/signal.h
/usr/include/linux/signal.h
```

База данных команды `locate` периодически обновляется командой `updatedb`, которая запускается демоном `cron`. Следовательно, результаты работы команды `locate` не всегда отражают последние изменения в системе.

Подробнее об управлении пакетами рассказывается в главе 12.

Если известно имя искомого пакета, воспользуйтесь системными утилитами обработки пакетов, которые непосредственно проверяют наличие пакета. Например, в системах Red Hat или SUSE следующая команда определяет наличие интерпретатора языка Python (и его установленной версии):

```
redhat$ rpm -q python
python-2.4.3-21.el5
```

Добавление новых программ

Если возникнет необходимость установить дополнительное программное обеспечение, вам, прежде всего, нужно определить каноническое имя соответствующего программного пакета. Например, вам следует преобразовать ваше желание “хочу установить `locate`” в версию “нужно установить пакет `findutils`” или перевести намерение “установить `named`” в вариант “установить `BIND`”. В этом вам поможет множество веб-индексов, эффективно работающих при указании конкретной системы, особенно при использовании Google. Например, при введении в поле поиска “команда `locate`” вас направят точно по адресу. Если вы работаете в системе UNIX, укажите также имя операционной системы.

Если вам известно имя нужной утилиты, можете загрузить и установить ее. В Linux и Solaris полная установка обычно состоит в использовании одной команды, поскольку в этих системах уже установлена утилита `pkgutil`. Для HP-UX и AIX вам придется загрузить либо предварительно собранный бинарный пакет, либо исходный код проекта. В последнем случае попробуйте с помощью Google найти официальную веб-страницу проекта и загрузить оттуда его исходный код.

Следующие примеры демонстрируют установку утилиты `wget` для каждой системы, взятой в этой книге в качестве примера. Эта GNU-утилита, которая превращает сетевую загрузку файлов (поддерживающую протоколы HTTP и FTP) в элементарную команду, очень полезна для выполнения сценариев. Утилита `wget` устанавливается по умолчанию в каждом из рассматриваемых здесь примеров систем Linux; при этом отметим, что представленные ниже команды можно использовать как во время начальной установки, так и при последующих обновлениях системы.

В системе Ubuntu используется программа для установки, обновления и удаления программных пакетов в операционных системах Debian (Advanced Package Tool - APT).

```
ubuntu# apt-get install wget
Reading package lists... Done
Building dependency tree
Reading state information... Done
```

```
wget is already the newest version.
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
```

Вот как выглядит версия SUSE.

```
suse# yast --install wget
<runs in a terminal-based UI>
```

Вариант инсталляции в системе Red Hat.

```
redhat# yum install wget
Loaded plugins: fastestmirror
```

```
Parsing package install arguments
Package wget-1.10.2-7.el5.i386 is already installed and latest version
Nothing to do
```

В системе Solaris инсталляция выполняется с помощью уже установленной утилиты `pkgutil` (для получения инструкций по данной установке см. сайт blastwave.org).

```
solaris# /opt/csw/bin/pkgutil --install wget
<multiple pages of output as seven packages are installed>
```

Для системы HP-UX мы нашли на сайте hpux.connect.org.uk соответствующий бинарный пакет и загрузили его в каталог `/tmp`. Команды распаковки и инсталляции в этом случае выглядят так.

```
hpux# gunzip /tmp/wget-1.11.4-hppa-11.31.depot.gz
hpux# swinstall -s /tmp/wget-1.11.4-hppa-11.31.depot wget
===== 05/27/09 13:01:31 EDT BEGIN swinstall SESSION
(non-interactive) (jobid=hpux11-0030)
* Session started for user "root@hpux11".
* Beginning Selection
* Target connection succeeded for "hpux11:/".
* Source:      /tmp/wget-1.11.4-hppa-11.31.depot
* Targets:    hpux11:/
* Software selections:
    wget.wget-RUN,r=1.11.4,a=HP-UX_B./800
* Selection succeeded.
* Beginning Analysis and Execution

* Analysis and Execution succeeded.
```

Пакет `depot` (в строке с командой `swinstall`) должен быть задан как полный путь, начинающийся косой чертой `/`; в противном случае команда `swinstall` попытается найти нужный файл в сети. Утилита `wget` при завершении инсталляции сообщает команде `swinstall`, какой пакет надо загрузить из файла `depot`.

К сожалению, этот процесс инсталляции в действительности не так прост, как может показаться на первый взгляд. Инсталлируемая версия утилиты `wget` на самом деле не будет работать, поскольку некоторые библиотеки, от которых она зависит, не инсталлированы.

```
hpux$ wget http://samba.org/samba/docs/Samba3-HOWTO.pdf
/usr/lib/dld.sl: Can't open shared library: /usr/local/lib/libcrypto.sl
/usr/lib/dld.sl: No such file or directory
```

```
[HP ARIES32]: Core file for 32 bit PA-RISC application
[HP ARIES32]: /usr/local/bin/wget saved to /tmp/core.wget.
```

Утилита `swinstall` обладает встроенным механизмом управления зависимостями, но его возможности, к сожалению, не распространяются на интернет-репозитории. Прочитайте “паспортные данные” утилиты `swinstall` и установите все необходимые для нее пакеты (в данном случае — еще шесть).

Сборка программного обеспечения из исходного кода

Фактически существует, по крайней мере, один бинарный пакет `wget`, доступный для AIX в формате RPM. Поиск в Google при заданной строке “`aix wget rpm`” вполне оправдает ваши надежды. После загрузки вам останется выполнить совсем простую команду инсталляции.

```
aix# rpm --install wget-1.11.4-1.aix5.1.ppc.rpm
```

В целях иллюстрации построим AIX-версию пакета `wget` на основе оригинального исходного кода.

Прежде всего, нам нужно найти этот код, но это не проблема: первый же результат, предложенный системой Google для заданной строки поиска “`wget`”, направит нас прямо на страницу GNU-проекта, которая содержит нужные ссылки. После загрузки текущей версии в каталог `/tmp` мы распаковываем ее, задаем конфигурацию, компонуем и устанавливаем.

```
aix# cd /tmp; gunzip wget-1.11.4.tar.gz
aix# tar xfp wget-1.11.4.tar
aix# cd wget-1.11.4
aix# ./configure --disable-ssl --disable-nls # См. комментарии ниже
configure: configuring for GNU Wget 1.11.4
checking build system type... rs6000-ibm-aix
```

```
config.status: creating src/config.h
config.status: executing default commands
generating po/POTFILES from ./po/POTFILES.in
creating po/Makefile
```

```
aix# make
<несколько страниц компиляционных сообщений>
aix# make install
<о странице выходных сообщений>
```

Приведенную последовательность команд инсталляции `configure/make/make install` обычно применяют для большинства UNIX- и Linux-утилит во всех системах, но, безусловно, при наличии установленной среды разработки и программ, составляющих заданный пакет. При этом будет не лишним прочитать файлы `INSTALL` или `README`, сопровождающие данный пакет.

В данном случае использование ключей `--disable-ssl` и `--disable-nls` в команде `configure` позволяет опустить некоторые `wget`-функции, входящие в состав библиотек, которые не были установлены (при обслуживании реальной системы вам, возможно, захочется включить в инсталляцию опущенные здесь части пакета). Чтобы узнать все опции процесса конфигурации, используйте команду `configure --help`. Обратите внимание на полезную опцию `--prefix=каталог` команды `configure`, которая позволит вам поместить программное обеспечение не в каталог `/usr/local`, а в более подходящий для вас каталог.

1.13. Издержки профессии

Системные администраторы — настоящие “многостаночники”. Они часто имеют другую работу, просто их попросили присмотреть за несколькими компьютерами “на стороне”. Если вы один из таких людей, подумайте, к чему в конце концов это может привести.

Чем больше вы будете знать о своей системе, тем больше пользователи будут зависеть от вас. Сети неуклонно разрастаются, и вы будете вынуждены тратить все больше и больше времени на выполнение функций администратора. Вскоре окажется, что вы — единственный человек во всей организации, который знает, как решить ряд важнейших проблем.

Если коллеги стали считать вас локальным системным администратором, от этой роли уже трудно отказаться. Мы знаем нескольких людей, которым пришлось поменять место работы, чтобы избавиться от дополнительной нагрузки. Поскольку круг обязанностей системного администратора четко очертить нельзя, от вас, скорее всего, потребуют, чтобы вы были не только штатным администратором, но и штатным инженером, писателем, а также специалистом по психоанализу.

Практика показывает, что некоторые администраторы, ставшие таковыми волей обстоятельств (а не по “велению души”), стараются избежать потока просьб о помощи, демонстрируя раздражение и плохо выполняя свои обязанности.⁹

Не стоит придерживаться такого подхода, так как вы будете плохо выглядеть в глазах окружающих и у вас могут возникнуть дополнительные проблемы. Вместо этого мы предлагаем просто фиксировать время, затрачиваемое на системное администрирование. Ведите работу на должном уровне и собирайте доказательства, которые можно будет использовать, когда вы попросите начальника освободить вас от обязанностей администратора. В большинстве организаций для того, чтобы добиться замены, приходится упрашивать руководство полгода, а то и год, так что учитывайте это в своих планах.

С другой стороны, может оказаться, что системное администрирование вам нравится и вы хотите стать штатным администратором. В этом случае проблем с поиском работы у вас не будет. К сожалению, сама работа от этого не станет легче. О политических аспектах системного администрирования рассказывается в главе 32.

1.14. Рекомендуемая литература

- Robbins Arnold. *UNIX in a Nutshell (4th Edition)*. Sebastopol, CA: O'Reilly Media, 2008.
- Siever Ellen, Aaron Weber and Stephen Figgins. *Linux in a Nutshell (5th Edition)*. Sebastopol, CA: O'Reilly Media, 2006.
- Gancarz Mike. *Linux and the Unix Philosophy*. Boston: Digital Press, 2003.
- Salus Peter H. *The Daemon, the GNU & the Penguin: How Free and Open Software is Changing the World*. Reed Media Services, 2008.

В настоящее время эта захватывающая история развития программного обеспечения с открытым исходным кодом, исследованная наиболее известным историком UNIX, переводится в цифровую форму на сайте groklaw.com в соответствии с лицензией Creative Commons (Сообщество представителей творческих профессий).

⁹ Обозначенная тенденция с любовью и черным юмором описана в рассказах Синона Травальи (Simon Travaglia *Bastard Operator from Hell* на сайте bofh.ntk.net в разделе BOFH.).

URL-адрес самой книги достаточно длинный; попытайтесь найти действующую ссылку на сайте groklaw.com или проверьте следующий сокращенный вариант: tinyurl.com/d6u7j.

- Raymond Eric S. *The Cathedral & The Bazaar: Musings on Linux and Open Source by an Accidental Revolutionary*. Sebastopol, CA: O'Reilly Media, 2001.

Системное администрирование

- Limoncelli Thomas A, Christina J. Hogan and Strata R. Chalup. *The Practice of System and Network Administration (Second Edition)*. Reading, MA: Addison-Wesley, 2008.

Это прекрасная книга, описывающая политические и процедурные аспекты системного администрирования. Авторы ведут блог по системному администрированию (everythingsysadmin.com).

- Frisch JEleen. *Essential System Administration (3rd Edition)*. Sebastopol, CA: O'Reilly Media, 2002.

Это классическое универсальное руководство по администрированию систем UNIX, которое, к сожалению, уже несколько устарело. Надеемся, что его новая версия уже в работе!

Основной инструментарий

- Robbins Arnold, Elbert Hannah and Linda Lamb. *Learning the vi and Vim Editors*. Sebastopol, CA: O'Reilly Media, 2008.
- Powers Shelly, Jerry Peek, Tim O'reilly and Mike Loukides. *UNIX Power Tools (3rd Edition)*. Sebastopol, CA: O'Reilly Media, 2003.
- Michael Randal K. *Mastering UNIX Shell Scripting: BASH, Bourne, and Korn Shell Scripting for Programmers, System Administrators, and UNIX Gurus*. Indianapolis, IN: Wiley Publishing, Inc., 2008.
- Robbins Arnold and Nelson H. F. Beebe. *Classic Shell Scripting*. Sebastopol, CA: O'Reilly Media, 2005.
- Wall Larry, Tom Christiansen and Jon Orwant. *Programming Perl (3rd Edition)*. Cambridge, MA: O'Reilly Media, 2000.
- Christiansen Tom and Nathan Torkington. *Perl Cookbook (2nd Edition)*. Sebastopol, CA: O'Reilly Media, 2003.
- Blank-Edelman David N. *Automating System Administration with Perl (2nd Edition)*. Sebastopol, CA: O'Reilly Media, 2009.
- Pilgrim Mark. *Dive Into Python*. Berkeley, CA: Apress, 2004.

К этой книге есть свободный веб-доступ по адресу: diveintopython.org.

- Flanagan David and Yukihiro Matsumoto. *The Ruby Programming Language*. Sebastopol, CA: O'Reilly Media, 2008.

Эта книга, имеющая оптимистический подзаголовок *Everything You Need to Know* (Все, что вам необходимо знать), к сожалению, написана несколько сухо. Но она подкупает полнотой описания языка Ruby (версии 1.9) с богатством деталей, которые могут быть известны лишь создателю языка. Это самая лучшая книга для тех, кто уже владеет (на практическом уровне) языками Perl или Python.

1.15. Упражнения

- 1.1. Какую команду нужно ввести, чтобы получить информацию о драйвере терминала `tty` (а не о команде `tty`)? Как прочитать соответствующую страницу документации, хранящуюся в каталоге `/usr/local/share/man`?
- 1.2. Определяет ли общий конфигурационный файл поведение команды `man` в вашей системе? Какие строки нужно в него добавить, чтобы локальные материалы хранились в каталоге `/doc/man`? Как организовать каталог `/doc/man`, чтобы его структура соответствовала иерархии справочных страниц?
- 1.3. Как в настоящее время протекает разработка ядра Linux? Каковы наиболее актуальные вопросы? Какие организации играют в этом процессе ключевую роль? Кто руководит проектом?
- 1.4. Изучите возможности нескольких систем UNIX и Linux и порекомендуйте операционную систему для каждого из перечисленных ниже случаев. Аргументируйте свой выбор.
 - а) домашний компьютер пользователя
 - б) университетская компьютерная лаборатория
 - в) корпоративный веб-сервер
 - д) серверный кластер, обеспечивающий работу базы данных для судоходной компании
- 1.5. Предположим, вы обнаружили, что определенное свойство демона `httpd` сервера Apache работает не так, как описано в документации к Ubuntu.
 - а) что нужно сделать перед тем, как сообщать об ошибке?
 - б) если вы решили, что это действительно ошибка, то кого нужно уведомить и каким способом?
 - в) какую информацию необходимо включить в отчет об ошибке?
- 1.6. Системы Linux серьезно заявили о себе в производственных средах. Обречены ли UNIX-системы? Почему?

Глава 2

Сценарии и командная оболочка

Настоящие системные администраторы пишут сценарии. Сценарии позволяют стандартизировать и автоматизировать выполнение рутинных операций по администрированию систем, тем самым освобождая столь ценное время администраторов для решения более важных и более интересных задач. В известном смысле сценарии — это документирование действий в виде официального перечня команд, необходимых для выполнения конкретной задачи.

Сценарии бывают самыми разными: от очень простых, которые инкапсулируют всего несколько статических команд, до значительных программных проектов, управляющих конфигурациями хостов и административной информацией всего узла. В этой книге нас, главным образом, интересуют небольшие повседневные сценарные проекты, с которыми обычно имеют дело системные администраторы, поэтому мы не будем подробно останавливаться на функциях поддержки (например, отслеживание ошибок и оценка проекта), необходимых для более крупных проектов.

При написании административных сценариев следует делать акцент на эффективности работы программиста и ясности кода, а не на производительности вычислительных средств. Не может служить оправданием для небрежно написанного сценария время его выполнения, т.е. не так уж важно, выполняется он за полсекунды или за две. Удивительно, но оптимизация слабо оправдывает затраты, даже в случае сценариев, которые регулярно выполняются демоном `cron`.

В течение долгого времени стандартный язык написания административных сценариев определялся командной оболочкой. В большинстве систем используется оболочка `bash` (*“Bourne-again” shell* — “возрожденная” оболочка Борна), но в ряде систем UNIX

все еще работают `sh` (оригинальная оболочка Борна) и `ksh` (оболочка Корна). Сценарии оболочки обычно применяются для выполнения таких несложных задач, как автоматизация последовательности команд или сборка нескольких фильтров для обработки данных.

Командная оболочка всегда доступна, поэтому ее сценарии отличаются относительной переносимостью и независимостью, в отличие от вызываемых ими команд. Пока вы будете выбирать оболочку, она может выбрать вас: большинство сред включает объемистое дополнение уже существующих `sh`-сценариев, с которыми администраторам часто приходится знакомиться, разбираться и заниматься их настройкой.

При написании более сложных сценариев желательно использовать такие языки программирования, как Perl или Python, поскольку они оба хорошо подходят для выполнения административной работы. Эти языки (по сравнению с командной оболочкой) выбрали в себя такие мощные достижения в области создания языков программирования последних двух десятилетий и оснащены такими возможностями по части обработки текстов (столь бесценные для системных администраторов), что оболочке `sh` остается лишь “рыдать от стыда где-то в уголке”.

Основной недостаток языков Perl и Python состоит в отсутствии четкости в установке их сред, особенно в случае, если вы начинаете использовать библиотеки с откомпилированными компонентами от сторонних производителей. Командная оболочка не страдает такой проблемой, поскольку не имеет модульной структуры и не содержит сторонних библиотек.

В этой главе дается краткий обзор языковых инструментов написания сценариев `bash`, Perl и Python, а также регулярных выражений.

2.1. Основы работы командной оболочки

Прежде чем обсуждать написание сценариев оболочки, рассмотрим некоторые базовые команды и синтаксис языка командной оболочки. Материал этого раздела актуален для всех основных оболочек `sh`-семейства (включающего оболочки `bash` и `ksh`, но не `csh` или `tcsh`), вне зависимости от используемой вами платформы. Попробуйте формы, с которыми вы пока еще не знакомы, и не бойтесь экспериментировать!

Редактирование команд

Мы наблюдали, как многие редактируют командные строки с помощью клавиш управления курсором. Вы ведь так не поступаете в своем текстовом редакторе, верно?

Все основные команды редактора `emacs` доступны при редактировании уже выполненного ряда команд (т.е. “предыстории”). Нажав комбинацию клавиш `<Ctrl+E>`, вы переместитесь в конец строки, а с помощью комбинации клавиш `<Ctrl+A>` — в начало. Нажатие комбинации клавиш `<Ctrl+P>` позволяет вернуться к предыдущим командам и вызвать их для редактирования. С помощью комбинации клавиш `<Ctrl+R>` вы сможете шаг за шагом “прокрутить” свою “предысторию” и найти старые команды.

Если вы предпочитаете использовать редактор `vi`, переключите свою командную оболочку в режим `vi`.

```
$ set -o vi
```

В режиме `vi` редактирование является модалным, но вы начинаете работать в режиме ввода команд. Для того чтобы выйти из режима ввода, нажмите вначале клавишу `<Esc>`, а затем клавишу `<i>`, чтобы вернуться в него. В режиме редактирования нажатие клавиши `<w>` переместит вас вперед на одно слово; использование комбинаций

клавиш <f+X> позволит найти следующий символ “X” в строке и т.д. “Прогуляться” по командам, зафиксированным в “предыстории”, можно с помощью клавиш <Esc> и <к>. Решили вернуться в режим **emacs**-редактирования? Выполните эту команду.

```
$ set -o emacs
```

Каналы и перенаправление потоков

Каждому процессу доступны по меньшей мере три информационных канала: “стандартный ввод” (STDIN), “стандартный вывод” (STDOUT) и “стандартная ошибка” (STDERR). Эти каналы устанавливаются ядром системы “от имени процесса”, и поэтому сам процесс не обязательно знает их направленность. Они, например, могут быть связаны с окном терминала, файлом, подключением к сети или с каналом, принадлежащим к другому процессу.

Для систем UNIX используется унифицированная модель ввода-вывода, в которой каждому каналу присваивается целое число, именуемое *файловым дескриптором*. Точное число (номер), назначаемое каналу, обычно не имеет значения, но каналам STDIN, STDOUT и STDERR гарантированно соответствуют файловые дескрипторы 0, 1 и 2 соответственно, чтобы обеспечить безопасное обращение к ним по номерам. В контексте интерактивного окна терминала канал STDIN обычно считывает данные с клавиатуры, а оба канала STDOUT и STDERR записывают свои выходные данные на экран.

Большинство команд принимает входные данные из канала STDIN. Выходная информация записывается ими в канал STDOUT, а сообщения об ошибках — в канал STDERR. Это соглашение позволяет объединять команды подобно строительным блокам для организации конвейерной обработки данных.

Командная оболочка интерпретирует символы “<”, “>” и “>>” как инструкции по изменению направления передаваемых командой данных в файл или принимаемых данных из файла. Символ “<” связывает канал STDIN с содержимым некоторого существующего файла. Символы “>” и “>>” перенаправляют поток STDOUT; причем символ “>” используется для замены содержимого файла, а “>>” — для добавления данных в его конец. Например, команда

```
$ echo "Это тестовое сообщение." > /tmp/mymessage
```

сохраняет одну строку в файле `/tmp/mymessage` (при необходимости файл будет создан). Следующая команда отправляет содержимое этого файла по электронной почте пользователю johndoe.

```
$ mail -s "Mail test" johndoe < /tmp/mymessage
```

Для того чтобы перенаправить потоки STDOUT и STDERR в одно и то же место, используйте символ “>&”. Для того чтобы перенаправить только поток STDERR, используйте вариант “2>”.

На примере команды `find` можно показать, почему важно обрабатывать потоки STDOUT и STDERR отдельно. Дело в том, что она формирует выходные данные по обоим каналам, особенно в случае ее выполнения непривилегированным пользователем. Например, при выполнении команды

```
$ find / -name core
```

обычно генерируется так много сообщений об ошибках “permission denied” (отсутствие прав доступа), что настоящие результаты поиска теряются в “шуме”. Чтобы отбросить все сообщения об ошибках, можно использовать такой вариант.

```
$ find / -name core 2> /dev/null
```

В этой версии команды `find` только настоящие совпадения (где пользователь имеет разрешение на чтение родительского каталога) выводятся в окно терминала. Чтобы сохранить список совпадающих путей в файле, выполните такую команду.

```
$ find / -name core > /tmp/corefiles 2> /dev/null
```

Эта командная строка отправляет совпадающие пути в файл `/tmp/corefiles`, отбрасывая все ошибки и ничего не посылая в окно терминала.

Для того чтобы связать канал `STDOUT` одной команды с каналом `STDIN` другой, используйте символ `" | "`. Вот два примера.

```
$ ps -ef | grep httpd
$ cut -d: -f7 < /etc/passwd | sort -u
```

В первом примере выполняется команда `ps`, генерирующая список процессов, из которого при “проходе” через команду `grep` выбираются строки, содержащие слово “`httpd`”. Результат выполнения команды `grep` никуда больше не перенаправляется, поэтому совпадающие строки попадают в окно терминала.

Во втором примере команда `cut` выбирает из файла `/etc/passwd` пути к оболочке каждого пользователя. Список оболочек затем отправляется через команду `sort -u`, чтобы сформировать отсортированный список уникальных значений.

Для того чтобы последующая команда выполнялась только в случае успешного выполнения предыдущей, можно разделить эти команды символом `&&`. Например, командная строка

```
$ lpr /tmp/t2 && rm /tmp/t2
```

удалит файл `/tmp/t2` тогда и только тогда, когда она успешно дожидается очереди на печать. В данном случае успех выполнения команды `lpr` будет зафиксирован при получении кода завершения, равного нулю. Использование для этой цели символа, означающего операцию “логическое И”, может сбить с толку тех, кто в других языках программирования использовал вычисления в сокращенной форме записи. Если кому-то это непонятно, не стоит слишком долго здесь застревать; а просто примите это как идиому командной оболочки.

И наоборот, символ `||` обеспечит выполнение следующей команды только в том случае, если предыдущая команда не выполнится (т.е. сгенерирует ненулевое значение кода завершения).

Для того чтобы разбить команду на несколько строк, для наглядности отделяя тем самым код обработки ошибок от остальной части командного конвейера, можно использовать в сценариях обратную косую черту.

```
cp --preserve --recursive /etc/* /spare/backup \
|| echo "Did NOT make backup"
```

Для получения обратного эффекта, т.е. объединения нескольких команд в одной строке, можно использовать в качестве разделителя точку с запятой.

Использование переменных и кавычек

В операциях присваивания имена переменных никак не маркируются, но предваряются знаком доллара при ссылке на их значения.

```
$ etcdir='/etc'
$ echo $etcdir
/etc
```

Не ставьте до и после знака равенства (=) пробелы, в противном случае оболочка ошибочно примет имя вашей переменной за имя команды.

При ссылке на переменную можно заключить ее имя в фигурные скобки, чтобы синтаксический анализатор (да и сам человек) не сомневался в том, где заканчивается имя переменной и начинается другой текст; например, используйте запись `${etcdir}` вместо `etcdir`. Обычно без фигурных скобок можно обойтись, но они могут оказаться полезными в случае, если вам понадобится раскрывать переменные в строках с двойными кавычками. Часто нужно сделать так, чтобы после значения переменной стояли буквы или знаки препинания.

```
$ echo "Saved ${rev}th version of mdadm.conf."
Saved 8th version of mdadm.conf.
```

Для имен переменных командной оболочки не существует стандартного соглашения, но обычно предполагается, что имена, полностью написанные прописными буквами, принадлежат переменным среды или переменным, считанным из файлов глобальной конфигурации. И чаще всего все буквы в именах локальных переменных — строчные, а отдельные их компоненты разделяются символами подчеркивания. Вообще имена переменных чувствительны к регистру букв.

Переменные среды автоматически импортируются в пространство имен оболочки `bash`, поэтому их можно устанавливать и считывать с использованием стандартного синтаксиса. Для того чтобы перевести переменную оболочки в ранг переменной среды, используйте команду `export имя_переменной`. Команды для переменных среды, которые вы хотите установить во время регистрации (при входе в систему), должны быть включены в ваш файл `~/.profile` или `~/.bash_profile`. Другие переменные среды, такие как `PWD` (для хранения имени текущего рабочего каталога), автоматически создаются командной оболочкой.

Командная оболочка интерпретирует строки, заключенные в одинарные и двойные кавычки, почти одинаково. Различие состоит в том, что строки в двойных кавычках служат субъектами для универсализации файловых имен (замены реальных символов в имени и расширении файла универсальными, т.е. такими метасимволами, как “*” и “?”) и для раскрытия переменных (замены переменных их значениями).

```
$ mylang="Pennsylvania Dutch"
$ echo "I speak ${mylang}."
I speak Pennsylvania Dutch.
$ echo 'I speak ${mylang}.'
I speak ${mylang}.
```

Обратные апострофы (также известные как обратные галочки, левые кавычки, левые одиночные кавычки или открывающие кавычки) интерпретируются аналогично двойным кавычкам, но несут при этом дополнительную нагрузку. Эта нагрузка состоит в интерпретации содержимого такой строки, как команды оболочки, выполнении этой команды и замене строки результатом выполнения этой команды.

```
$ echo "There are 'wc -l /etc/passwd' lines in the passwd file."
There are 28 lines in the passwd file.
```

Команды общих фильтров

Любые корректные команды, которые считывают входные данные из канала `STDIN` и записывают результаты в канал `STDOUT`, могут быть использованы в качестве фильтра (т.е. компонента конвейера) для обработки данных. В этом разделе мы кратко рас-

смотрим некоторые из часто используемых команд фильтрации (хотя их полный список практически бесконечен). Команды фильтрации отличаются настолько сильным “коллективизмом”, что порой даже трудно продемонстрировать их индивидуальное использование.

Большая часть команд фильтрации принимает в командной строке одно или несколько имен файлов. И только в том случае, если вы не можете задать соответствующие файлы, указывайте в качестве источника считываемой информации стандартный входной поток.

Команда *cut*: разбивка строк на поля

Команда *cut* выводит выбранные части входных строк. Чаще всего эта команда используется для извлечения полей (ограниченных некоторыми разделителями), как в приведенном выше примере, который для удобства читателей повторим еще раз.

```
$ cut -d: -f7 < /etc/passwd | sort -u
```

Команда *cut* также может возвращать сегменты, определенные границами столбцов текста. В качестве стандартного разделителя служит символ <Tab>, но с помощью ключа *-d* разделители можно изменить. Ключ *-f* позволяет указать, какие поля нужно включить в результат.

Еще один пример использования команды *cut* показан в разделе, посвященном команде *uniq* (см. ниже).

Команда *sort*: сортировка строк

Команда *sort* сортирует входные строки. Звучит просто, да? Хотя на самом деле не все так безоблачно: существуют потенциальные нюансы, связанные с точно заданными частями сортируемых строк (т.е. с использованием сортировочного ключа) и порядком сортировки. Наиболее распространенные варианты применения этой команды показаны в табл. 2.1, в отношении же остальных случаев обратитесь к соответствующей таблице.

Таблица 2.1. Ключи команды *sort*

Ключ команды	Значение
-b	Игнорировать ведущие пробельные символы
-f	Сортировать независимо от регистра букв
-k	Указать столбцы, которые формируют сортировочный ключ
-n	Сравнивать поля как целые числа
-r	Изменить порядок сортировки на противоположный
-t	Установить разделитель полей (по умолчанию — пробельный символ)
-u	Выводить только уникальные записи

На примерах приведенных ниже команд демонстрируется различие между числовой и лексикографической сортировкой (последняя действует по умолчанию). В обеих командах используются ключи *-t:* и *-k3, 3* для сортировки файла */etc/group* по его третьему полю (в качестве разделителя используется двоеточие), содержащему идентификатор (ID) группы. Первая команда реализует числовую сортировку, а вторая — лексикографическую (в алфавитном порядке).

```
$ sort -t: -k3,3 -n /etc/group1
root:x:0:
bin:x:1:daemon
daemon:x:2:

$ sort -t: -k3,3 /etc/group
root:x:0:
bin:x:1:daemon
users:x:100:
```

Команда `uniq`: вывод уникальных строк

Команда `uniq` по существу подобна команде `sort -u`, но у нее есть некоторые полезные ключи, которые команда `sort` не эмулирует. Так, ключ `-c` используется для подсчета количества экземпляров каждой строки, ключ `-d` — для отображения только строк, имеющих дубликаты, а ключ `-u` — для отображения только строк, не имеющих дубликаты. При этом входные данные должны быть предварительно отсортированы, обычно путем “пропускания” через команду `sort`.

Например, по результатам выполнения следующей команды видно, что у 20 пользователей поле `login shell` (стартовая оболочка) содержит значение `/bin/bash` и у 12 — `/bin/false`. (Последний результат характерен либо для псевдопользователей, либо для пользователей, учетные записи которых были заблокированы.)

```
$ cut -d: -f7 /etc/passwd | sort | uniq -c
  20 /bin/bash
  12 /bin/false
```

Команда `wc`: подсчет строк слов и символов

Подсчет количества строк, слов и символов в файле — еще одна распространенная операция, удобным средством реализации которой и является команда `wc` (word count). Выполненная без каких-либо ключей, она выведет все три результата подсчета.

```
$ wc /etc/passwd
32 77 2003 /etc/passwd
```

В контексте написания сценариев команду `wc` часто применяют только с одним из ключей (`-l`, `-w` или `-c`), чтобы результат ее выполнения состоял из одного числа. Эту форму чаще всего можно встретить внутри обратных апострофов, и тогда результат может быть сохранен или использован по назначению.

Команда `tee`: копирование входного потока в два места

Командный конвейер, как правило, действует прямолинейно, но зачастую полезно распараллелить поток данных, т.е. “перехватить” его и отправить копию в файл или окно терминала. Это можно сделать с помощью команды `tee`, которая отправляет свой стандартный входной поток как в стандартный выходной канал, так и в файл, указанный в командной строке. Представьте действие этой команды в виде тройника в трубопроводе (перев. с англ. *tee* — тройник).

¹ Команда `sort` принимает спецификацию ключа `-k3` (вместо `-k3,3`), но в этом случае вы, скорее всего, не получите ожидаемого результата. Без завершающего номера поля сортировочный ключ продолжится до конца строки.

Устройство `/dev/tty` можно считать синонимом для текущего терминала.

Например, команда

```
$ find / -name core | tee /dev/tty | wc -l
```

выводит найденные путевые имена файлов `core` и результат подсчета их количества.

Часто работа конвейера с командой `tee`, выводящей результаты и в файл, и в окно терминала (для проверки), занимает много времени. Вы можете понаблюдать за первыми результатами, чтобы убедиться в том, что все работает как надо, а затем смело уходить: ведь результаты же сохранятся в файле.

Команды `head` и `tail`: чтение файла с начала или с конца

Просмотр строк с начала или конца файла — обычная административная операция. Команды `head` и `tail` отображают по умолчанию десять строк, но вы можете использовать ключ, задающий желаемое количество строк.

Для интерактивного использования команда `head` уже несколько устарела, и вместо нее (в этом контексте) часто используется команда `less`, которая разбивает файлы для отображения по страницам. Но команду `head` можно успешно применять в сценариях и с другой целью. С командой `tail` используется ключ `-f`, который чрезвычайно полезен для системных администраторов. Вместо немедленного завершения (после вывода требуемого количества строк) команда `tail -f` ожидает “прибытия” новых строк, которые (как только они появятся “на горизонте”) будут добавлены в конец файла, а затем выведены по назначению, что очень удобно для мониторинга журналов регистрации. Однако следует иметь в виду, что программа, которая реализует запись данных в файл, может буферизировать свои выходные данные. Даже если строки будут добавляться регулярно (с точки зрения логики), они могут стать видимыми только фрагментами объемом 1 или 4 Кбайт (KiB).²

Для остановки процесса мониторинга нажмите комбинацию клавиш `<Ctrl+C>`.

Команда `grep`: поиск текста

При выполнении команды `grep` по мере просмотра входного текста выводятся строки, которые совпадают с заданным образцом (шаблоном). Свое название эта команда получила “в наследство” от команды `g/regular_expression/p` из старого (и ныне действующего) редактора `ed`, используемого в самых первых версиях UNIX.

“Регулярные выражения” (regular expressions) — это шаблоны предназначенные для поиска совпадающего с ними текста и написанные на стандартном языке шаблонов. Они представляют собой универсальный стандарт, используемый большинством программ при поиске по совпадению с шаблоном, хотя и с небольшими различиями в реализациях. Странное имя команды уходит своими корнями в оригинальные изыскания соответствующих разделов теории вычислений. Более детально синтаксис регулярных выражений рассматривается в разделе 2.3.

Подобно большинству фильтров, команда `grep` имеет множество ключей. Например, ключ `-c` позволяет выводить количество совпавших строк, ключ `-i` служит для игнорирования регистра букв при сопоставлении, а ключ `-v` предназначен для вывода отличающихся (а не совпавших) строк. Очень полезным является ключ `-l` (прописная буква “L”), который заставляет команду `grep` выводить только имена файлов, содержащие совпавшие с шаблоном строки, а не все совпавшие строки.

Например, выполнение команды

² О единицах измерения см. в разделе 1.8 главы 1.


```
$ sudo grep -l mdadm /var/log/*  
/var/log/auth.log  
/var/log/syslog.0
```

демонстрирует, что записи с именем утилиты mdadm нашлись в двух различных файлах системных журналов.

Команду `grep` можно считать классической реализацией базового механизма использования регулярных выражений, но в некоторых версиях предлагается выбор других диалектов. Например, Linux-команда `grep -p` служит для поиска выражений в стиле языка Perl, хотя в справочных страницах можно найти неопределенное предупреждение о том, что такой вариант носит “экспериментальный характер”. Для полной уверенности в своих сценариях просто используйте язык Perl или Python.

2.2. Написание `bash`-сценариев

Командный арсенал оболочки `bash` позволяет успешно писать простые сценарии (без этого средства автоматизации системным администраторам пришлось бы вручную вводить команды в командную строку). Ваше мастерство в использовании командных строк вы должны воплотить в искусстве создания `bash`-сценариев (и наоборот), что поможет вам извлечь максимальную пользу из времени, потраченного на изучение оболочки `bash`. Но когда окажется, что ваш `bash`-сценарий превысил в объеме сотню строк или вам потребовались средства, которыми `bash` не обладает, это будет означать, что настало время переходить к языку Perl или Python.

В среде `bash` комментарии начинаются со знака `#` и продолжаются до конца строки. Как и в командной строке, вы можете разбить одну логическую строку на несколько физических строк, обозначив переход на новую строку символом обратной косой черты (`\`). И, наоборот, можно поместить на одной строке несколько операторов, разделив их точкой с запятой.

Сценарий для оболочки `bash` может состоять только из последовательности командных строк. Например, следующий сценарий `helloworld` просто выполняет команду `echo`.

```
#!/bin/bash  
echo "Hello, world!"
```

Первая строка содержит “банальный” описательный оператор, который сообщает, что данный текстовый файл является сценарием, предназначенным для интерпретации командной оболочкой `/bin/bash`. При принятии решения о том, как выполнить этот файл, ядро отыщет соответствующий синтаксис. С точки зрения оболочки, “стремящейся” выполнить этот сценарий, первая строка представляет собой просто комментарий. Если бы оболочка находилась в другом каталоге, вам пришлось бы отредактировать эту строку.

Для того чтобы подготовить этот файл к выполнению, достаточно установить его бит, “отвечающий” за выполнение (см. раздел 6.5).

```
$ chmod +x helloworld  
$ ./helloworld  
Hello, world!
```

³ Если ваша оболочка понимает команду `helloworld` без префикса `./`, это означает, что в вашем пути поиска указан текущий каталог (`.`). И это плохо, поскольку дает другим пользователям возможность устроить для вас “ловушки” в надежде, что вы будете пытаться выполнить определенные команды при переходе к каталогу, в котором они имеют доступ для записи.

Можно также непосредственно запустить (активизировать) оболочку в качестве интерпретатора сценария.

```
$ bash helloworld
Hello, world!
$ source helloworld
Hello, world!
```

Первая команда выполнит сценарий `helloworld` в новом экземпляре оболочки `bash`, а вторая — заставит вашу начальную оболочку прочитать содержимое файла, а затем выполнить его. Второй вариант полезен в случае, когда сценарий устанавливает переменные среды или выполняет другие операции настройки, применимые только к текущей оболочке. Обычно этот вариант используется при написании сценариев, включающих содержимое файла конфигурации, написанного в виде ряда присваиваний значений `bash`-переменным.⁴

Подробнее о битах разрешения (доступа) можно прочитать в разделе 6.5.

Если вы пришли сюда из мира Windows, то вам привычно использовать понятие расширения файла, по которому можно судить о типе файла, а также о том, можно ли его выполнить. В мире UNIX и Linux признак того, может ли файл быть выполнен (и если да, то кем), содержится в специальных битах полномочий. При желании вы можете наделить свои `bash`-сценарии суффиксом `.sh`, который бы напоминал вам об их типе, но тогда при выполнении соответствующей команды вам придется вводить суффикс `.sh`, поскольку UNIX не интерпретирует расширения специальным образом.

От команд к сценариям

Прежде чем переходить к особенностям написания `bash`-сценариев, остановимся на методике этого процесса. Многие пишут `bash`-сценарии так же, как Perl- или Python-сценарии, т.е. используя какой-нибудь текстовый редактор. Но все же удобнее рассматривать в качестве интерактивной среды разработки сценариев режим с приглашением на ввод команды.

Предположим, например, что у вас есть журналы регистрации, именованные с суффиксами `.log` и `.LOG` и разбросанные по всей иерархической системе каталогов. Допустим также, что вы хотите изменить эти суффиксы, приведя их к “прописному” виду. Прежде всего, попытаемся отыскать все эти файлы.

```
$ find . -name '*log'
.do-not-touch/important.log
admin.com-log/
foo.log
genius/spew.log
leather_flog
```

Так, похоже на то, что нам надо включить в шаблон точку и игнорировать при поиске каталоги. Нажмите комбинацию клавиш `<Ctrl+P>`, чтобы вернуть команду в командную строку, а затем модифицируйте ее.

```
$ find .-type f -name '*.log'
.do-not-touch/important.log
foo.log
```

⁴ Синонимом для команды `source` служит команда “с точкой” (`dot`-команда), например `. helloworld`.

```
genius/spew.log
```

Ну вот, это уже выглядит лучше. Правда, каталог `.do-not-touch` (т.е. “не трогать”) вызывает смутное чувство опасности; но мы можем избавиться от этого неприятного холода.

```
$ find . -type f -name '*.log' | grep -v .do-not-touch
foo.log
genius/spew.log
```

Теперь все в порядке: мы получили абсолютно точный список файлов, которые должны быть переименованы. Попробуем сгенерировать несколько новых имен.

```
$ find . -type f -name '*.log' | grep -v .do-not-touch | while read fname
> do
> echo mv $ fname $ {fname/. log/. LOG/}
> done
mv foo.log foo.LOG
mv genius/spew.log genius/spew.LOG
```

Да, это именно те команды, которые позволят переименовать нужные файлы. А как мы это делаем в реальности? Мы могли бы снова вызвать уже выполненную команду и отредактировать команду `echo`, чтобы заставить оболочку `bash` выполнять команды `mv`, а не просто выводить их. Ведь передача команд в отдельную копию оболочки `bash` — более надежный вариант работы, который к тому же требует меньшего объема редактирования.

Нажав комбинацию клавиш `<Ctrl+P>`, мы обнаруживаем, что оболочка `bash` заботливо свернула наш мини-сценарий в одну-единственную строку. К этой “уплотненной” командной строке мы просто добавляем канал, передающий наши выходные данные команде `bash -x`.

```
$ find . -type f -name '*.log' | grep -v .do-not-touch | while read fname;
do echo mv $fname $ {fname/. log/. LOG/}; done | bash -x
+ mv foo.log foo.LOG
+ mv genius/spew.log genius/spew.LOG
```

Ключ `-x` команды `bash` обеспечивает вывод каждой команды перед ее выполнением.

Теперь мы завершили переименование файлов, но нам хотелось бы сохранить этот сценарий, чтобы можно было использовать его снова. Встроенная в `bash` команда `fc` по своему действию во многом аналогична нажатию комбинации клавиш `<Ctrl+P>`, но вместо возврата последней команды в командную строку она передает команду в заданный вами редактор. Добавьте в свой файл строку идентификационного комментария, поместите сохраненный файл в приемлемый для вас каталог (например, `~/bin` или `/usr/local/bin`), сделайте файл исполняемым, и вы получите настоящий сценарий.

Итак, подытожим.

- Разрабатывайте сценарий (или его часть) в виде конвейера команд, причем пошагово и в режиме выполнения командных строк.
- Пересылайте результат в стандартный выходной поток, проверяя правильность работы используемых вами команд.
- На каждом этапе используйте буфер ранее выполненных команд для их появления в командной строке и инструменты редактирования — для их модификации.

- Пока вы получаете неправильные результаты, можно считать, что вы, по сути, ничего не сделали, и до тех пор, пока команды не заработают так, как надо, ничего (из уже сделанного) не надо удалять.
- Если результат выглядит правильным, выполните команды на реальном примере, чтобы убедиться, что все получилось так, как ожидалось.
- Используйте команду `fc`, чтобы зафиксировать свою работу в редакторе, оформите ее соответствующим образом и сохраните.

В приведенном выше примере мы вывели командные строки, а затем направили их в подоболочку для выполнения. Этот метод не является универсально применимым, но часто оказывается полезным. В качестве альтернативного варианта можно фиксировать результаты, перенаправляя их в файл. Терпеливо добивайтесь получения нужных результатов, и пока вы не увидите их, не выполняйте никаких потенциально деструктивных действий.

Организация ввода и вывода данных

Команда `echo` не отличается интеллектуальностью, но зато проста в применении. Для получения большего контроля над выводом данных используйте команду `printf`. Она не очень удобна, поскольку предполагает, что вы должны в явном виде указывать в нужных для вас местах символы перехода на новую строку (`\n`), но позволяет использовать символ табуляции и другие средства форматирования результата. Сравните результаты выполнения следующих двух команд.

```
$ echo "\taa\tbb\tcc\n"
\taa\tbb\tcc\n
$ printf "\taa\tbb\tcc\n"
aa bb cc
```

В некоторых системах работа команд `echo` и `printf` поддерживается на уровне ОС (обычно соответствующие им исполняемые файлы хранятся в каталогах `/bin` и `/usr/bin` соответственно). Хотя эти команды и встроенные в оболочку утилиты в целом подобны, они могут иметь незначительные отличия, и особенно это касается команды `printf`. Вы можете либо придерживаться `bash`-синтаксиса, либо вызывать “внешнюю” команду `printf`, указывая ее полный путь.

Для того чтобы сформировать для пользователя приглашение ввести данные, можно использовать команду `read`.

```
#!/bin/bash

echo -n "Введите свое имя: "
read user_name

if [ -n "$user_name" ]; then
    echo "Привет $user_name!"
    exit 0
else
    echo "Вы не назвали свое имя!"
    exit 1
fi
```

Ключ `-n` в команде `echo` подавляет привычный переход на новую строку, но здесь была бы кстати команда `printf`. Мы еще рассмотрим вкратце синтаксис оператора `if`, но его действие здесь, с нашей точки зрения, очевидно. Ключ `-n` в операторе `if` обе-

спечит значение истины, если его строковый аргумент не окажется нулевым. Вот как выглядит результат выполнения этого сценария.

```
$ sh readexample
Введите свое имя: Ron
Привет Ron!
```

Функции и аргументы командной строки

Аргументы командной строки служат для сценария переменными с числовыми именами: \$1 — первый аргумент командной строки, \$2 — второй и т.д. Аргумент \$0 содержит имя, по которому был вызван сценарий, например `./bin/example.sh`, т.е. это не фиксированное значение.

Переменная \$# содержит количество переданных аргументов командной строки, а переменная \$* — все эти аргументы. Ни одна из этих переменных не учитывает аргумент \$0.

При вызове сценария с некорректными аргументами или вообще без них должно быть выведено короткое сообщение с напоминанием о том, как следует использовать данный сценарий. Приведенный ниже сценарий, принимающий два аргумента, проверяет, являются ли оба эти аргумента каталогами, и, убедившись в этом, отображает их. Если аргументы оказываются недопустимыми, сценарий выводит сообщение о его правильном использовании и завершается с ненулевым значением кода возврата. Если программа, вызвавшая этот сценарий, проверит код возврата, она будет “знать”, что этот сценарий не удалось выполнить корректно.

```
#!/bin/bash

function show_usage {
    echo "Использование: $0 source_dir dest_dir"
    exit 1
}

# Основная программа начинается здесь

if [ $# -ne 2 ]; then
    show_usage
else # Существуют два аргумента
    if [ -d $1 ]; then
        source_dir=$1
    else
        echo 'Недопустимый каталог-источник'
        show_usage
    fi
    if [ -d $2 ]; then
        dest_dir=$2
    else
        echo 'Недопустимый каталог-приемник'
        show_usage
    fi
fi

printf "Каталог-источник: ${source_dir}\n"
printf "Каталог-приемник: ${dest_dir}\n"
```

Для вывода сообщения о правильном использовании данного сценария мы создали отдельную функцию `show_usage`. Если бы позже этот сценарий был модифицирован и стал бы принимать дополнительные аргументы, это сообщение нужно было бы изменить только в одном месте.⁵

```
$ mkdir aaa bbb
$ sh showusage aaa bbb
Каталог-источник: aaa
Каталог-приемник: bbb
$ sh showusage foo bar
Недопустимый каталог-источник
Использование: showusage source_dir dest_dir
```

Аргументы `bash`-функций обрабатываются практически так же, как аргументы командной строки: `$1` — первый аргумент командной строки, `$2` — второй и т.д. Как видно из приведенного выше примера, аргумент `$0` содержит имя сценария.

Для того чтобы сделать предыдущий пример более устойчивым к ошибкам, мы могли бы заставить функцию `show_usage` принимать в качестве аргумента код ошибки. Это позволило бы конкретизировать код возврата для каждого типа отказа. Реализация этой идеи показана в следующем фрагменте программы.

```
function show_usage {
    echo "Использование: $0 source_dir dest_dir"
    if [ $# -eq 0 ]; then
        exit 99 # Выход с любым ненулевым кодом возврата
    else
        exit $1
    fi
}
```

В этой версии функции добавляется анализ наличия аргументов. Внутри функции переменная `$#` сообщает, сколько аргументов было ей передано. Сценарий завершится с кодом возврата 99, если при его вызове не было передано никаких аргументов. Но при передаче конкретного значения, например, такого, как

```
show_usage 5
```

сценарий завершится с этим кодом после вывода сообщения об использовании сценария. (Переменная `$?` содержит статус завершения последней выполненной команды, независимо от того, была ли она использована в сценарии или в командной строке.)

Между функциями и командами в оболочке `bash` существует строгая аналогия. Вы можете определить полезные функции в своем файле `~/.bash_profile` , а затем использовать их в командной строке как команды. Например, если ваш узел стандартизировал в сети порт 7988 для протокола SSH (в форме “безопасность через сокрытие”), вы можете определить в своем файле `~/.bash_profile` функцию `ssh`, чтобы быть уверенным в том, что она всегда будет запускаться (как команда) с ключом `-p 7988`.

```
function ssh {
    /usr/bin/ssh -p 7988 $*
}
```

⁵ Обратите внимание на то, что сообщения об ошибках и сообщение об использовании сценария направляются в стандартный поток вывода данных. Разве не логичнее было бы направить их в стандартный канал вывода ошибок? В действительности это было бы корректнее, но поскольку не предполагается использовать этот сценарий в качестве фильтра, канал вывода данных здесь не так уж и важен.

Подобно многим командным оболочкам, `bash` обладает механизмом псевдонимов (`alias`), который может репродуцировать этот ограниченный пример в еще более лаконичном виде, тем не менее функции остаются более универсальным и более мощным средством. Поэтому забудьте о псевдонимах и просто используйте функции.

Область видимости переменных

Переменные в рамках сценария имеют глобальный статус, но функции могут создавать собственные локальные переменные с помощью объявления `local`. Рассмотрим следующий код.

```
#!/bin/bash

function localizer {
    echo "==> В функции localizer начальное значение a равно '$a' "
    local a
    echo "==> После объявления local значение a стало равным '$a' "
    a="localizer version"
    echo "==> При выходе из функции localizer значение a равно '$a' "
}

a="test"
echo "До вызова функции localizer значение a равно '$a' "
localizer
echo "После вызова функции localizer значение a равно '$a' "
```

По приведенным ниже результатам выполнения сценария `scopetest.sh` видно, что локальная версия переменной `$a` внутри функции **`localizer`** “забывает” глобальную переменную `$a`. Глобальная переменная `$a` видна в функции `localizer` до тех пор, пока не встретится объявление `local`, т.е. по сути, объявление `local` работает как команда, которая в момент выполнения создает локальную переменную.

```
$ sh scopetest.sh
До вызова функции localizer значение a равно 'test'
==> В функции localizer начальное значение a равно 'test'
==> После объявления local значение a стало равным ' '
==> При выходе из функции localizer значение a равно 'localizer version'
После вызова функции localizer значение a равно 'test'
```

Поток управления

Выше в этой главе мы уже рассмотрели несколько конструкций `if-then` и `if-then-else` (они работают вполне ожидаемым образом). Терминатором (признаком конца) для оператора `if` служит оператор `fi`. Для образования цепочки `if`-операторов можно использовать ключевое слово `elif`, означающее “else if”.

```
if [ $base -eq 1 ] && [ $dm -eq 1 ]; then
    installDMBase
elif [ $base -ne 1 ] && [ $dm -eq 1 ]; then
    installBase
elif [ $base -eq 1 ] && [ $dm -ne 1 ]; then
    installDM
else
    echo '==> Installing nothing'
fi
```

Как и специальный `[]` -синтаксис для выполнения операций сравнения, так и “ключеподобные” имена операторов целочисленного сравнения (например `-eq`) уходят “наследственными корнями” в использование утилиты `/bin/test` из ранней командной оболочки Стивена Борна. В действительности квадратные скобки — это не что иное, как условное обозначение вызова утилиты `test`; они не являются частью оператора `if`.⁶

В табл. 2.2 собраны `bash`-операторы сравнения для чисел и строк. В отличие от Perl, в `bash` используются текстовые операторы для чисел и символические операторы для строк.

Таблица 2.2. Элементарные `bash`-операторы сравнения

Строки	Числа	Истина, если
<code>x = y</code>	<code>x -eq y</code>	<code>x</code> равно <code>y</code>
<code>x != y</code>	<code>x -ne y</code>	<code>x</code> не равно <code>y</code>
<code>x < y</code>	<code>x -lt y</code>	<code>x</code> меньше <code>y</code>
<code>x <= y</code>	<code>x -le y</code>	<code>x</code> меньше или равно <code>y</code>
<code>x > y</code>	<code>x -gt y</code>	<code>x</code> больше <code>y</code>
<code>x >= y</code>	<code>x -ge y</code>	<code>x</code> больше или равно <code>y</code>
<code>-n x</code>	<code>-</code>	<code>x</code> не равно значению <code>null</code>
<code>-z x</code>	<code>-</code>	<code>x</code> равно значению <code>null</code>

В оболочке `bash` предусмотрены возможности оценки свойств файлов (снова-таки как освобождение от наследства `/bin/test`). Некоторые из операторов тестирования и сравнения файлов приведены в табл. 2.3.

Таблица 2.3. Некоторые `bash`-операторы оценки свойств файлов

Оператор	Истина, если
<code>-d файл</code>	Файл <code>файл</code> существует и является каталогом
<code>-e файл</code>	Файл <code>файл</code> существует
<code>-f файл</code>	Файл <code>файл</code> существует и является обычным
<code>-r файл</code>	У вас есть право доступа для чтения файла
<code>-s файл</code>	Файл <code>файл</code> существует и он — не пустой
<code>-w файл</code>	У вас есть право доступа для записи в файл
<code>файл1 -nt файл2</code>	Файл <code>файл1</code> новее, чем <code>файл2</code>
<code>файл1 -ot файл2</code>	Файл <code>файл1</code> старше, чем <code>файл2</code>

Несмотря на всю полезность формы `elif`, зачастую лучше (с точки зрения ясности программного кода) использовать `case`-структуру выбора варианта. Ниже показан ее синтаксис на примере функции, которая централизует процесс регистрации сообщений для сценария. Конкретные варианты описываются закрывающими скобками после каждого условия и двумя точками с запятой, завершающими блок операторов, который должен быть выполнен при реализации заданного условия. Оператор `case` завершается ключевым словом `esac`.

⁶ Сейчас эти операции встроены в оболочку и уже не запускают на выполнение утилиту `/bin/test`.


```
# Уровень протоколирования устанавливается в глобальной
# переменной LOG_LEVEL. Возможные варианты перечислены в порядке
# от самого строгого до наименее строгого: Error, Warning, Info и Debug.
```

```
function logMsg {
    message_level=$1
    message_itself=$2
    if [ $message_level -le $LOG_LEVEL ]; then
        case $message_level in
            0) message_level_text="Error" ;;
            1) message_level_text="Warning" ;;
            2) message_level_text="Info" ;;
            3) message_level_text="Debug" ;;
            *) message_level_text="Other"
        esac
        echo "${message_level_text}: $message_itself"
    fi
}
```

Эта функция иллюстрирует общепринятую парадигму “уровня регистрации” (log level), используемую многими приложениями административного характера. Код этого сценария позволяет генерировать сообщения на различных уровнях детализации, но действительно регистрируются или обрабатываются только те из них, которые “проходят” глобально устанавливаемый порог \$LOG_LEVEL. Чтобы прояснить важность каждого сообщения, его текст предваряется меткой, описывающей соответствующий уровень регистрации.

Циклы

Конструкция `for...in` предназначена для упрощения выполнения некоторых действий для группы значений или файлов, особенно при универсализации файловых имен, т.е. замене реальных символов в имени и расширении универсальными (например “*” и “?”) с целью формирования целых списков имен файлов. Шаблон *.sh в приведенном ниже цикле `for` позволяет обработать целый список совпадающих с ним (шаблоном) имен файлов из текущего каталога. Оператор `for`, проходя по этому списку, по очереди присваивает имя каждого файла переменной `$file`.

```
#!/bin/bash

suffix=BACKUP--`date +%Y%m%d-%H%M`

for script in *.sh; do
    newname="$script.$suffix"
    echo "Copying $script to $newname..."
    cp $script $newname
done
```

Результат выполнения этого сценария таков.

```
$ sh forexample
Copying rhel.sh to rhel.sh.BACKUP--20091210-1708...
Copying sles.sh to sles.sh.BACKUP--20091210-1708...
```

В раскрытии имени файла здесь нет ничего магического; все работает в точном соответствии с тем, как написано в командной строке. Другими словами, сначала имя файла

раскрывается (т.е. шаблон заменяется существующим именем), а затем уж обрабатывается интерпретатором в развернутом виде.⁷

С таким же успехом вы могли бы ввести имена файла статически, как показано в этой командной строке.

```
for script in rhel.sh sles.sh; do
```

В действительности любой список имен, содержащих пробельные символы (включая содержимое переменной), обрабатывается как объект циклической конструкции `for...in`.

Оболочке `bash` также присуща близость к циклу `for` из традиционных языков программирования, в которых задается стартовое выражение, инкрементация и условие окончания цикла.

```
for (( i=0 ; i < $CPU_COUNT ; i++ )) ; do
    CPU_LIST="$CPU_LIST $i"
done
```

На примере следующего сценария иллюстрируется `bash`-цикл `while`, который часто применяется для обработки аргументов командной строки и чтения строк файла.

```
#!/bin/bash

exec 0<$1
counter=1
while read line; do
    echo "$counter: $line"
   =$((counter++))
done
```

Вот как выглядит результат выполнения этого сценария.

```
ubuntu$ sh whileexample /etc/passwd
1: root:x:0:0:Superuser:/root:/bin/bash
2: bin:x:1:1:bin:/bin:/bin/bash
3: daemon:x:2:2:Daemon:/sbin:/bin/bash
```

В этом сценарии есть интересные особенности. Оператор `exec` переопределяет стандартный выходной поток сценария так, чтобы входные данные считывались из файлов, имена которых должны определяться в соответствии с первым аргументом командной строки.⁸ Если окажется, что такой файл не существует, данный сценарий сгенерирует ошибку.

Оператор `read` в конструкции `while` на самом деле является встроенным в оболочку, но действует подобно внешней команде. Внешние команды можно также помещать в конструкцию `while`. Цикл `while` в такой форме завершится тогда, когда внешняя команда возвратит ненулевое значение кода завершения.

Выражение `=$((counter++))` выглядит несколько странно. Обозначение `$(...)` говорит о вычислении выражения. Кроме того, оно делает необязательным использование символа `$` для обозначения имен переменных. Удвоенный знак “плюс” (`++`) — это оператор постинкремента, знакомый, например, по языку C. Он возвращает значение

⁷ Точнее, небольшая доля магии в раскрытии имен файлов здесь все же имеет место и состоит в поддержке понятия атомарности каждого имени файла. Имена файлов, которые включают пробелы, пройдут этот цикл `for` в один проход.

⁸ В зависимости от вызова, оператор `exec` также может иметь и такое значение: “остановить этот сценарий и передать управление другому сценарию или выражению”. В этом состоит еще одна странность оболочки: доступ к обоим упомянутым здесь функциям реализуется через один и тот же оператор.

переменной, с которой он связан, но также имеет побочный эффект, который состоит в приращении значения этой переменной.

Выражения `$ ( ( . . . ) )` работают в контексте двойных кавычек, поэтому тело рассматриваемого выше цикла можно свернуть до одной строки.

```
while read line; do
    echo "$((counter++)) : $line"
done
```

Массивы и арифметика

Сложные структуры данных и вычисления нельзя отнести к сильной стороне оболочки `bash`. Но, по крайней мере, вы можете надеяться на использование массивов и арифметических операций.

Все `bash`-переменные представляют собой строковые значения, поэтому оболочка `bash` не делает различия в присваиваниях между числом 1 и символьной строкой "1". Различие лежит в использовании переменных. Следующий код иллюстрирует это различие.

```
#!/bin/bash

a=1
b=$((2))

c=$a+$b
d=$(( $a+$b ))

echo "$a + $b = $c \t(знак плюс как строковый литерал) "
echo "$a + $b = $d \t(знак плюс как арифметическое сложение) "
```

При выполнении этого сценария получим такой результат.

```
1 + 2 = 1+2 (знак плюс как строковый литерал)
1 + 2 = 3 (знак плюс как арифметическое сложение)
```

Обратите внимание на то, что знак "плюс" в присваивании переменной `$c` не работает как оператор конкатенации для строк. Он остается всего лишь литеральным символом, и посему эта строка эквивалентна следующей.

```
c="$a+$b"
```

Для того чтобы добиться вычисления, необходимо заключить выражение в двойные скобки: `$ ( ( . . . ) )`, как показано выше в присваивании переменной `$d`. Но даже эта мера предосторожности не позволяет получить в переменной `$d` числового значения; это значение по-прежнему хранится в виде строки "3".

В оболочке `bash` реализован обычный ассортимент операторов: арифметических, логических и отношений (подробнее см. соответствующие man-страницы).

Массивы в командной оболочке `bash` могут показаться немного странными объектами (да они и используются не очень часто). Тем не менее при необходимости их можно применять. Литеральные массивы ограничиваются круглыми скобками, а отдельные элементы разделяются пробельными символами. Для включения литеральных пробелов в элемент можно использовать кавычки.

```
example=(aa 'bb cc' dd)
```

Для доступа к отдельным элементам массива используйте выражение `${имя_массива [индекс]}`. Индексация начинается с нуля. Такие индексы, как `"*"` и `"@"`,

относятся к массиву в целом, а специальные конструкции `${#имя_массива[*]}` и `${#имя_массива[@]}` возвращают количество элементов в массиве. Не спутайте эти выражения с конструкцией `${#имя_массива}` — и хотя эта форма кажется более логичной, но в действительности она содержит указатель на первый элемент массива (эквивалент для `${имя_массива[0]}`).

Можно подумать, что выражение `$example[1]` должно служить однозначной ссылкой на второй элемент массива, но оболочка `bash` интерпретирует эту строку так: `$example` (обозначение ссылки на `$example[0]`) плюс литеральная строка `[1]`. Отсюда вывод: ссылаясь на переменные массива, всегда используйте фигурные скобки (без каких-либо исключений).

Рассмотрим сценарий, который иллюстрирует некоторые особенности `bash`-массивов и подводные камни, на которые можно наткнуться при управлении ими.

```
#!/bin/bash

example=(aa 'bb cc' dd)
example[3]=ee

echo "example[@] = ${example[@]}"
echo "Массив example содержит ${#example[@]} элементов"

for elt in "${example[@]}"; do
    echo "Элемент = $elt"
done
```

Вот как выглядит результат выполнения этого сценария.

```
$ sh arrays
example [@] = aa bb cc dd ee
Массив example содержит 4 элемента
Элемент = aa
Элемент = bb cc
Элемент = dd
Элемент = ee
```

Этот пример кажется простым, но лишь потому, что мы организовали его “хорошее поведение”. Но если потерять бдительность, вы можете попасть в ловушку. Например, сценарий с заменой `for`-строки вариантом

```
for elt in ${example[@]}; do
```

(т.е. без кавычек, в которых заключено выражение массива) также работает, но вместо четырех элементов он выведет пять: `aa`, `bb`, `cc`, `dd` и `ee`.

Важно помнить, что все `bash`-переменные все равно остаются строками, поэтому работа с массивами — в некотором роде иллюзия. В тонкостях, связанных с тем, когда и как строки разбиваются на элементы, можно утонуть. Для того чтобы не рисковать, лучше используйте язык `Perl` или `Python`. Настойчивые читатели, желающие разобраться в нюансах, могут с помощью `Google` обратиться к руководству Менделя Купера (Mendel Cooper) *Advanced Bash-Scripting Guide*.

2.3. Регулярные выражения

Регулярные выражения поддерживаются большинством современных языков, хотя одни языки “принимают их ближе к сердцу”, чем другие. Они также используются в таких UNIX-командах, как `grep` и `vi`. Регулярные выражения настолько распространены,

что их название в оригинальной литературе часто сокращали до слова “regex” (*regular expressions*). О том, как использовать их немалые возможности, написаны уже целые книги, и неудивительно, что они стали объектом исследования многочисленных докторских диссертаций.

Сопоставление имен файлов и их раскрытие, выполненное оболочкой в процессе интерпретации таких командных строк, как `ws -l *.pl`, не является формой сопоставления с регулярным выражением. Это совсем другая система, именуемая “универсализацией файловых имен с помощью оболочки” (*shell globbing*), в которой используется другой, причем более простой, синтаксис.

Регулярные выражения — очень мощный инструмент, но они не могут распознавать все возможные грамматики. Их самый большой недостаток состоит в том, что они не в состоянии различать вложенные ограничители. Например, невозможно написать регулярное выражение, которое распознает допустимые арифметические выражения для ситуации, когда для группирования членов выражения разрешено использовать круглые скобки.

Регулярные выражения в мощности и совершенстве достигли кульминации в языке Perl. Функции сопоставления шаблонов в Perl настолько тщательно разработаны, что в действительности не совсем правильно называть их реализацией регулярных выражений. Шаблоны Perl могут включать вложенные разделители, они могут распознавать палиндромы (перевертни или перевертыши) и сопоставляться, например, с произвольной строкой букв “А”, за которыми последует такое же количество букв “Б”, — словом, пойдут в ход всевозможные вариации на тему регулярных выражений. При этом в Perl можно обрабатывать и довольно примитивные регулярные выражения.

Язык сопоставления Perl-шаблонов остается промышленным мерилем качества, и он принят на вооружение во многих других языках и инструментах. Библиотека Филиппа Гейзела (Philip Hazel) PCRE (*Perl-compatible /regular expression*) несколько упрощает для разработчиков включение этого языка в проекты.

Сами по себе регулярные выражения не являются частью языка написания сценариев, но они настолько полезны, что заслуживают подробного рассмотрения в любом обсуждении “сценарной” темы, а уж тем более в этом разделе.⁹ Здесь мы рассматриваем их в базовой форме с учетом некоторых Perl-нюансов.

Процесс сопоставления

Код, который использует регулярное выражение, пытается сопоставить одну заданную текстовую строку с одним заданным шаблоном. Сопоставляемая текстовая строка может иметь очень большой размер и содержать встроенные символы перехода на новую строку. Зачастую регулярное выражение удобно использовать для сопоставления с содержимым целого файла или HTML-документа.

Для того чтобы обнаружитель совпадений мог доложить о благоприятном исходе, шаблон поиска должен целиком совпасть с непрерывной областью исследуемого текста. При этом шаблон может совпасть в любом месте исследуемого текста. После обнаружения успешного совпадения вычислитель возвращает текст совпадения вместе со

⁹ Известный знаток Perl, Том Кристиансен (Tom Christiansen) сказал: “Я не знаю, что такое язык написания сценариев, но я согласен с тем, что регулярные выражения нельзя отнести ни к процедурным, ни к функциональным языкам. Скорее, это язык, основанный на логике, или декларативный язык, относящийся к группе языков, в которую также входят Prolog и Makefile. И формы Бэкуса-Наура (BFS). Его также можно назвать языком, основанным на системе правил. Лично я предпочитаю называть подобные языки декларативными”.

списком совпадений для любых частей (подразделов) шаблона, ограниченных специальным образом.

Литеральные символы

В общем случае символы регулярного выражения при сопоставлении должны соответствовать самим себе. Так, шаблон

```
I am the walrus
```

совпадает со строкой “I am the walrus” и только с этой строкой. Поскольку совпадение может быть обнаружено в любом месте исследуемого текста, этот шаблон может дать успешный результат совпадения со строкой “I am the egg man. I am the walrus. Koo koo ka-choo!” При этом реальное совпадение ограничено фрагментом “I am the walrus”. Процесс сопоставления чувствителен к регистру букв.

Специальные символы

В табл. 2.4 собраны значения некоторых распространенных специальных символов, которые могут использоваться в регулярных выражениях (вообще, этих специальных символов гораздо больше).

Таблица 2.4. Распространенные специальные символы регулярных выражений

Символ	С чем совпадает или что делает
•	Совпадает с любым символом
[chars]	Совпадает с любым символом из заданного набора
[^chars]	Совпадает с любым символом не из заданного набора
^	Совпадает с началом строки
\$	Совпадает с концом строки
\w	Совпадает с любым алфавитно-цифровым символом (аналогично набору [A-Za-z0-9_])
\s	Совпадает с любым пробельным символом (аналогично набору [\f\t\n\r]) ^a
\d	Совпадает с любой цифрой (аналогично набору [0-9])
	Совпадает с элементом либо слева, либо справа
(expr)	Ограничивает область действия, группирует элементы, позволяя формировать группы совпадений с “захватом” и без “захвата”
?	Позволяет одно повторение (или ни одного) предшествующего элемента
*	Позволяет множество повторений (или ни одного) предшествующего элемента
+	Позволяет одно или больше повторений предшествующего элемента
{n}	Ровно n повторений предшествующего элемента
{min, }	Не менее min повторений (обратите внимание на запятую)
{min,max}	Любое число (от min до max) повторений

^a пробел, символ прогона страницы, табуляции, новой строки или возврата каретки.

Многие такие специальные конструкции, как + и |, оказывают влияние на сопоставление подстрок слева или справа. В общем случае подстроку может составлять один символ, или “подшаблон”, заключенный в круглые скобки, или класс символов, заключенный в квадратные скобки. Однако для символа “|” подстрока распространяется нео-

граниченно как влево, так и вправо. Если вы хотите ограничить область действия вертикальной черты, заключите ее и обе подстроки в их собственный набор круглых скобок. Например, шаблон

```
I am the (walrus | egg man)\.
```

даст совпадение либо со строкой “I am the walrus.”, либо со строкой “I am the egg man.”. Этот пример демонстрирует также способ “экранирования” специальных символов (в данном случае точки) с помощью обратной косой черты (\). Шаблон

```
(I am the (walrus | egg man)\. ?) {1,2}
```

совпадет с любой из следующих строк.

- I am the walrus.
- I am the egg man.
- I am the walrus. I am the egg man.
- I am the egg man. I am the walrus.

К сожалению, он также даст совпадение со строкой “I am the egg man. I am the egg man.”. (И какой в этом смысл?) Важнее то, что приведенный выше шаблон также совпадет со строкой “I am the walrus. I am the egg man. I am the walrus.”, хотя количество повторений явно ограничено числом два. Дело в том, что шаблон не обязательно должен совпадать полностью с исследуемым текстом. В данном случае после обнаружения совпадений этого регулярного выражения с двумя предложениями дальнейшее его сопоставление прекратилось с объявлением об успешном завершении. После выполнения условия, заданного в регулярном выражении, уже не важно, что в данном тексте есть еще одно совпадение с шаблоном.

Часто метасимвол регулярного выражения (квантификатор, равный нулю или больше нуля) путают с символом используемым командной оболочкой для универсализации файловых имен. В системе регулярных выражений используйте метасимвол в случае, если для совпадения приемлема любая последовательность символов (включая их отсутствие).

Примеры использования регулярных выражений

В Соединенных Штатах почтовые индексы (zip-коды) имеют либо пять цифр, либо пять цифр с последующим тире и еще четырьмя цифрами. При создании регулярного выражения для zip-кода необходимо обеспечить сопоставление пятизначного номера. Следующее регулярное выражение отвечает всем этим требованиям.

```
^\d{5}$
```

Символы “^” и “\$” сопоставляются с началом и концом обрабатываемого текста, не соответствуя при этом реальным символам в тексте; они представляют собой “утверждения нулевой длины” (т.е. это не символы, а лишь “позиции” в тексте). Эти символы гарантируют, что только те текстовые строки, которые состоят ровно из пяти цифр, должны совпадать с регулярным выражением (строки большей длины не должны давать совпадения). Сочетание символов \d обеспечивает совпадение с цифрой, а квантификатор {5} выражает требование совпадения ровно пяти цифр.

Для того чтобы можно было выявить либо пятизначный почтовый индекс, либо расширенный zip-код (zip+4), добавим в качестве необязательной части тире и четыре дополнительных цифры.

```
^\d{5}(-\d{4})?$
```

Круглые скобки объединяют в группу тире и дополнительные цифры, чтобы они считались одной необязательной единицей. Например, это регулярное выражение не даст совпадения с пятизначным почтовым индексом, за которым следует “голое” тире (без последующих цифр). Если тире существует, также должно существовать четырехзначное расширение, в противном случае совпадение зафиксировано не будет.

Классический пример использования регулярного выражения демонстрирует следующее выражение.

```
M[ou]?'?am+[ae]r ([AEae]l[- ])?[GKQ]h?[aeu]+([dtz][dhz]?)+af[iy]
```

Оно дает совпадения со многими вариантами произношения имени ныне покойного лидера Ливии Муаммара Каддафи (Moammar Gadhafi), например, с такими, как:

- Muammar al-Kaddafi (BBC);
- Moammar Gadhafi (Associated Press);
- Muammar al-Qadhafi (Al-Jazeera);
- Mu'ammār Al-Qadhafi (U.S. Department of State).

Вы понимаете, почему каждый из приведенных вариантов успешно совпал с шаблоном?

Приведенное выше регулярное выражение также иллюстрирует, насколько быстро могут быть достигнуты пределы удобочитаемости. Многие системы регулярных выражений (включая используемую в Perl) поддерживают опцию `x`, которая игнорирует литеральные пробельные символы в шаблоне и легитимизирует комментарии, разрешая “растягивать” шаблон и располагать его на нескольких строках. Затем вы можете использовать пробельные символы для выделения логических групп и “прояснения отношений” между ними подобно тому, как это делается в процедурных языках.

```
M [ou] ' ? a m+ [ae] r      # Имя: Mu'ammar, Moamar и т.д.
\s                          # Пробельный символ; здесь нельзя
                             # использовать литеральный пробел
(
    [AEae] l                # Группа для необязательного префикса-фамилии
    [-\s ]                  # Al, El, al или el
)?
[GKQ] h? [aeu]+            # Начальная группа символов фамилии: Kha, Qua и т.д.
(
    [dtz] [dhz]?           # Группа для согласных в начале 2-го слога
    dd, dh и т.д.
) +
af [iy]
```

Такой способ описания регулярного выражения, вероятно, слегка облегчит понимание, но все же и он в состоянии замучить потенциальных читателей. Поэтому лучше использовать иерархический подход и строить множество небольших сопоставлений вместо попытки описать все возможные ситуации в одном огромном регулярном выражении.

Захваты

В случае, если совпадение происходит, каждый набор круглых скобок становится “группой захвата”, которая фиксирует реально совпавший текст. То, как именно эти элементы становятся доступными для вас, зависит от конкретной реализации и контекста. В Perl доступ к результатам можно получить в виде списка или последовательности пронумерованных переменных.

Поскольку круглые скобки могут быть вложенными, как узнать, чему соответствует каждое совпадение? Ответ простой: совпадения выстраиваются в результате в том же порядке, в котором располагаются открывающие скобки. Количество “захватов” равно количеству открывающих скобок, независимо от роли (или ее отсутствия), которую играла каждая взятая в скобки группа в реальном совпадении. Если взятая в скобки группа не используется (например, при сопоставлении регулярного выражения `Mu(')?ammar` со строкой “Muammar”), соответствующий “захват” будет пустым.

Если обнаружится несколько совпадений группы, в качестве результата возвращается содержимое только последнего совпадения. Например, при шаблоне

```
(I am the (walrus | egg man)\. ?){1,2}
```

и таком варианте обрабатываемого текста

```
I am the egg man. I am the walrus,
```

существует два результата (по одному для каждого набора круглых скобок).

```
I am the walrus.  
Walrus
```

Обратите внимание на то, что обе захваченные группы в действительности совпали дважды. Но был захвачен только последний текст, совпавший с каждым набором круглых скобок.

Жадность, лень и катастрофический поиск с возвратом

Регулярные выражения сопоставляются слева направо. Если каждый компонент шаблона перед переходом к следующему компоненту стремится “захватить” максимально возможную строку, то такая характеристика поведения называется *жадностью* (greediness).

Если обработчик регулярных выражений (анализатор) достигает состояния, из которого сопоставление выполнить уже невозможно, он немного “откатывается” назад от кандидата на совпадение и заставляет одного из “жадных” компонентов отказаться от части текста. Например, рассмотрим регулярное выражение `a*aa` применительно к входному тексту “aaaaaa”.

Вначале анализатор присваивает весь входной текст компоненту `a*`, поскольку он (т.е. `a*`) является “жадным”. Когда окажется, что больше букв “а” не осталось, анализатор перейдет к поиску совпадений со следующим компонентом регулярного выражения. Но поскольку им является `a` и во входном тексте больше нет ничего, что можно было бы сопоставить с `a`, пора делать “откат”. Компонент `a*` вынужден отказаться от одной из букв “а”, с которыми уже было зафиксировано совпадение.

Теперь анализатор может сопоставлять компонент `a*a`, но он по-прежнему не может это сделать для последней буквы “а” в шаблоне. Поэтому он снова откатывается и “отнимает” вторую букву “а” из “добычи” компонента `a*`. На этот раз вторая и третья буквы “а” в шаблоне имеют буквы “а” для фиксации совпадения, и на этом обработка текста завершена. Этот простой пример иллюстрирует некоторые важные общие моменты. Прежде всего, “жадное” сопоставление с “откатами” делает использование таких простых шаблонов, как `<img.*>`, дорогим удовольствием при обработке целых

файлов.¹⁰ Действие порции `.*` начинается с сопоставления всего содержимого от первой найденной подстроки `<img` до конца входного текста, и только благодаря повторным откатам область поиска сузится, чтобы “подобраться” к локальным тегам.

Более того, порция `></tr>`, с которой связан этот шаблон, — это *последнее возможное* допустимое вхождение искомого элемента во входном тексте, что, наверняка, совсем не отвечает вашим намерениям. Вероятно, вы хотели отыскать совпадение с подстрокой `<img>`, за которой следует тег `</tr>`. Тогда лучше переписать этот шаблон в виде `<img[^>]*></tr>`, что позволит распространить совпадение с начальными групповыми символами только до конца текущего тега благодаря явно заданной невозможности пересечь границу, обозначенную правой угловой скобкой.

Можно также использовать “ленивые” (в противоположность “жадным”) операторные выражения: `*?` вместо `*` и `+` вместо `+`. Эти варианты квантификаторов ограничивают количество вхождений искомого символа во входном тексте до минимально возможного. Во многих ситуациях эти операторы работают эффективнее и дают результаты, более близкие к желаемым, чем “жадные” варианты.

Однако обратите внимание на то, что “ленивые” квантификаторы (в отличие от “жадных”) могут давать различные совпадения; разница состоит не просто в используемой реализации. В нашем HTML-примере “ленивый” шаблон выглядел бы так: `<img.*?></tr>`. Но даже в этом случае элемент `.*` мог бы стать причиной внесения в результат ненужных нам символов `>`, поскольку следующим после тега `<img>` может быть не тег `</tr>`. А такой результат вас, скорее всего, не устроит.

Шаблоны с несколькими секциями групповых символов могут “спровоцировать” анализатор регулярных выражений на экспоненциальное поведение, особенно в случае, если порции текста могут совпадать с несколькими шаблонными выражениями, и тем более в случае, если искомым текст в действительности не соответствует шаблону. Эта ситуация не является такой уж необычной, как может показаться, главным образом тогда, когда шаблон сопоставляется с HTML-кодом. Очень часто вам придется искать конкретные теги, за которыми следуют другие теги, возможно, разделенные третьими тегами, и так далее, одним словом, вам придется создавать задание, которое потребует, чтобы анализатор проверил массу возможных комбинаций.

Большой специалист в области регулярных выражений Ян Гойвертс (Jan Goyvaerts) называет этот эффект *катастрофическим откатом* (catastrophic backtracking) и описывает его в своем блоге (за подробностями и некоторыми удачными решениями обращайтесь по адресу: regular-expressions.info/catastrophic.html).

Из всего сказанного выше можно сделать такие выводы.

- Если вы можете организовать построчное сопоставление шаблона, а не пофайловое (т.е. с просмотром сразу всего файла), значит, вы значительно уменьшаете риск получения, мягко говоря, низкой производительности.
- Несмотря на то что регулярные выражения являются “жадными” по умолчанию, вам, скорее всего, такие не нужны. Используйте “ленивые” операторы.

¹⁰ Несмотря на то что в этом разделе в качестве примеров обрабатываемого текста показаны фрагменты HTML-кода, регулярные выражения — не самый подходящий инструмент в данном случае (что не преминули отметить и наши рецензенты). Языки Perl и Python имеют прекрасные расширения, которые анализируют HTML-документы надлежащим образом. Вы можете получить доступ к интересующим вас порциям с помощью Xpath-селекторов. Подробнее о модульных репозиториях соответствующих языков можно узнать, обратившись к странице Википедии, посвященной языку запросов Xpath (XML Path Language).

- Все экземпляры специальных символов “. * ” по своему существу подозрительны и должны быть тщательно исследованы.

2.4. Программирование на языке Perl

Язык Perl, созданный Ларри Уоллом (Larry Wall), был первым из самых популярных языков написания сценариев. Он предлагает чрезвычайно больше возможностей, чем `bash`, а написанные на нем программы (если они написаны хорошо) довольно просты для понимания. Поскольку в языке Perl не предусмотрена обязательность соблюдения разработчиками стилистических правил создания кода, читабельность Perl-кода зависит от их дисциплинированности. Не без основания Perl называют языком “только для записи”.

Здесь описывается версия Perl 5, которая была стандартом в течение последнего десятилетия. Версия Perl 6 все еще находится в стадии разработки. Подробнее можно узнать на сайте perl6.org. Языки Perl и Python (см. раздел 2.5) больше подходят для работы в области системного администрирования, чем такие традиционные языки программирования, как C, C++, C# и Java. Их отличает большая эффективность: при лаконичности кода и менее мучительном процессе отладки, а также без компиляционной “волокиты” они могут продемонстрировать большие возможности.

Выбор языка обычно зависит от личных предпочтений или стандартов, используемых вашим работодателем. Как Perl, так и Python предлагают библиотеки модулей и языковые расширения. По популярности язык Perl “старше”, чем Python, поэтому его предложения “вылились” в довольно длинный список возможностей. Но при решении каждодневных задач системного администрирования библиотеки поддержки обоих языков практически эквивалентны.

Для языка Perl характерен следующий девиз: “Достичь цель можно несколькими путями”. Поэтому имейте в виду, что в большинстве примеров, описанных в этом разделе, существуют и другие (помимо приведенного здесь) способы выполнения поставленной задачи.

Операторы Perl разделяются точкой с запятой.¹¹ Комментарии начинаются знаком `#` и продолжаются до конца строки. Блоки операторов заключаются в фигурные скобки. Вот пример простой программы “Привет, мир!”.

```
#!/usr/bin/perl
print "Привет, мир!\n";
```

Как и для `bash`-программ, вы должны либо изменить права доступа (`chmod +x`) для исполняемого файла, либо непосредственно вызвать Perl-интерпретатор.

```
$ chmod +x helloworld
$ ./helloworld
Привет, мир!
```

Строки в Perl-сценарии не являются командами оболочки (они представляют собой Perl-код). В отличие от командной оболочки `bash`, которая позволяет объединить некоторую последовательность команд в сценарий, Perl не “видит дальше своего носа”, если ему не дать соответствующего указания. Тем не менее Perl обеспечивает действие многих таких же соглашений, которые действуют в оболочке `bash`, например использование обратных апострофов, заключающих в себя некоторую команду, для перехвата результата выполнения этой команды.

¹¹ Поскольку знаки “точка с запятой” являются разделителями, а не ограничителями (терминаторами), последнюю точку с запятой в блоке ставить необязательно.

Переменные и массивы

В Perl используются три базовых типа данных: скаляры (т.е. такие унитарные значения, как числа и строки), массивы и хеши (известные также как ассоциативные массивы). Тип переменной всегда очевиден, поскольку он “встроен” в имя переменной: скалярные переменные начинаются знаком “\$”, переменные массивов — символом и хеш-переменные — знаком “%”.

В Perl понятия “список” и “массив” часто взаимозаменяемы, но, пожалуй, корректнее сказать, что список — это некоторая последовательность значений, а массив — это переменная, которая может содержать такой список. Отдельные элементы массива являются скалярами, поэтому, подобно обычным скалярным переменным, их имена начинаются со знака “\$”. Индексация массивов начинается с нуля, а индекс самого последнего элемента в массиве @a определяется значением \$#a. Чтобы получить размер массива, достаточно добавить к этому значению единицу.

Массив @ARGV содержит аргументы командной строки сценария. Его можно использовать подобно любому другому массиву.

Пример использования массивов демонстрирует следующий сценарий.

```
#!/usr/bin/perl
```

```
@items = ("носки", "туфли", "шорты");  
printf "Есть %d вида одежды.\n", $#items + 1;  
print "Наденьте ${items[2]} первыми, затем ", join(" и ", @items[0,1]), ".\n";
```

Результат выполнения этого сценария таков.

```
$ perl clothes
```

```
Есть 3 вида одежды.
```

```
Наденьте шорты первыми, затем носки и туфли.
```

В этих нескольких строках сценария есть много интересного. Рискую отвлечь ваше внимание, мы все же включаем в каждый наш Perl-пример несколько распространенных идиом, разъясняя сложные моменты в тексте, следующем за примером. Если вы внимательно прочитаете все пояснения к примерам (не пугайтесь, они короткие!), то к концу этой главы приобретете практические знания в использовании большинства распространенных Perl-конструкций.

Массив и строковые литералы

В этом примере обратите внимание, прежде всего, на то, что набор круглых скобок (...) создает список литералов. Отдельными элементами этого списка являются строки (которые разделяются запятыми). Созданный список присваивается переменной @items.

В языке Perl нет жесткого требования заключать все строки в кавычки. В конкретном случае начальное присваивание значения переменной @items с тем же успехом работает и без кавычек.

```
@items = (носки, туфли, шорты);
```

Такие строки без кавычек называются в Perl “голыми словами” (barewords), и они интерпретируются таковыми в качестве “последнего средства”. Это значит, что, если для некоторого объекта невозможно найти смысловую нагрузку, Perl пытается интерпретировать его как строку. В некоторых случаях это имеет смысл, если сохранится читабельность кода. Однако наш пример не относится к таким случаям. Даже если вы сами пред-

почитаете всегда заключать строки в кавычки, будьте готовы к тому, что вам придется “разгадывать” намерения других программистов, разбираясь в их “голословном” коде.

В Perl существует еще один способ инициализировать этот массив. Он состоит в использовании оператора `qw` (*quote words* — т.е. брать слова в кавычки). Это, по сути, форма заключения строки в кавычки, которая предоставляет вам право самим выбирать разделители (как и во многих других случаях использования кавычек в Perl). Форма

```
@items = qw (носки туфли шорты);
```

является самой традиционной, но при этом может ввести в заблуждение, поскольку часть после оператора `qw` — уже не список. В действительности это строка, разделенная пробельными символами для формирования списка. Версия

```
@items = qw [носки туфли шорты];
```

тоже работает и, пожалуй, точнее передает смысл происходящего. Обратите внимание на то, что запятых здесь нет, поскольку их нагрузку “взял на себя” оператор `qw`.

Вызовы функций

Оба варианта — `print` и `printf` — принимают произвольное количество аргументов, которые разделяются запятыми. Но вот вы видите `join(...)`. Не напоминает ли вам это вызов функций? Чем же “этот `join`” отличается от `print` и `printf`?

По сути, ничем, `print`, `printf` и `join` — это обычные функции. Perl позволяет опускать круглые скобки в вызовах функций, если эта “экономия” не послужит причиной неоднозначности, поэтому обе формы приемлемы. В приведенной выше `print`-строке форма записи со скобками (`join (...)`) используется для явного отделения аргументов, передаваемых функции `join`, от аргументов, передаваемых функции `print`.

Выражение `@items[0,1]` должно дать “списочный” результат, поскольку оно начинается с символа `@`. В действительности это выражение означает “кусочек массива” (или подмассив), а список индексов `0,1` перечисляет индексы элементов исходного массива, которые должны быть включены в упомянутый “кусочек”. Здесь Perl принимает диапазон значений, как и в эквивалентном выражении `@items [0..1]`. Здесь также было бы допустимо использовать одиночный числовой индекс: например, выражение `@items[0]` означало бы список, содержащий один скаляр, т.е. строку “носки”, что в данном случае было бы эквивалентно литералу (“носки”).

Массивы в вызовах функций автоматически раскрываются, поэтому в выражении

```
join(" и ", @items[0,1])
```

функция `join` получает три строковых аргумента: “ и ”, “носки” и “туфли”. Она конкатенирует свой второй и последующие аргументы, вставляя между каждой парой копию первого аргумента. В результате получаем “носки и туфли”.

Преобразования типов в выражениях

В `printf`-строке при вычислении выражения `$#items + 1` получим число 3. По определению выражение `$#items` содержит числовое значение, но это не дает основания считать, что выражение `$#items + 1` предполагает выполнение арифметического действия; смешанный вариант “2” + 1 работает так же. “Магия” кроется в операторе “+”, который всегда подразумевает арифметику, т.е. он преобразует свои аргументы в числа и генерирует числовой результат. Аналогично ему оператор “точка” (`.`), назначение которого — конкатенировать строки, преобразует свои операнды “по контексту”, т.е. при вычислении выражения “2”. (12 ** 2) получим в результате “2144”.

Раскрытие строк и устранение неоднозначности при ссылках на переменные

Как и в командной оболочке `bash`, строки, заключенные в двойные кавычки, являются объектами для раскрытия переменных. Так же, как и в `bash`, вы можете заключить имена переменных в фигурные скобки, чтобы при необходимости устранить неоднозначность, как в выражении `${items[2]}`. (В данном случае фигурные скобки используются только для иллюстрации; на самом деле они не нужны.) Символ “\$” информирует о том, что результат вычисления выражения должен быть скалярным. Переменная `@items` определяет массив, но его любой элемент в отдельности — это скаляр, и сей факт отражен в соглашениях о присваивании имен переменным.

Хеши

Хеш (также именуемый ассоциативным массивом или хеш-массивом) представляет набор пар “ключ/значение”. Хеш можно представить себе как массив, индексы которого (ключи) являются произвольными скалярными значениями, т.е. они необязательно должны быть числами. Но на практике в качестве ключей используются именно числа и строки.

Имена хеш-переменных начинаются с символа “%” (например, `%myhash`), но их отдельными значениями являются скаляры, поэтому для работы с хеш-массивами нужно, как и в обычных массивах, использовать префикс разыменования. Индексация обозначается фигурными скобками, а не квадратными, например `$myhash{'ron'}`.

Для системных администраторов хеши — это очень важный инструмент, который вы будете использовать практически в каждом сценарии. В приведенном ниже коде мы считываем содержимое файла, анализируем его в соответствии с правилами, заданными структурой файла `/etc/passwd`, и формируем хеш с именем `%names_by_uid`. Значением каждого элемента в хеше становится имя пользователя, связанное с идентификатором UID.

```
#!/usr/bin/perl

while ($_ = <>) {
    ($name, $pw, $uid, $gid, $gecos, $path, $sh) = split /:/;
    $names_by_uid{$uid} = $name;
}

%uids_by_name = reverse %names_by_uid;

print "\$names_by_uid{0} is $names_by_uid{0}\n";
print "\$uids_by_name{'root'} is $uids_by_name{'root'}\n";
```

Как и в предыдущем примере сценария, мы “начинили” эти программные строки некоторыми новыми понятиями. Прежде чем рассматривать анонсированные нюансы, ознакомьтесь с результатом работы этого сценария.

```
$ perl hashexample /etc/passwd
$names_by_uid{0} is root
$uids_by_name{'root'} is 0
```

В каждом проходе цикла `while` (`$_ = <>`) считывается одна входная строка, которая присваивается переменной `$_` (напомним: значение всего оператора присваивания, как в языке C, определяется значением его правой части). При достижении конца вход-

ного потока, оператор `<>` возвратит значение “Ложь”, что послужит причиной завершения цикла.

Интерпретируя оператор `<>`, Perl проверяет командную строку, чтобы узнать, указаны ли там какие-нибудь имена файлов. Если вы не забыли это сделать, Perl открывает каждый файл по очереди и “прогоняет” его содержимое через цикл. Если вы не назвали ни одного файла в командной строке, Perl возьмет входные данные для циклической обработки из стандартного входного канала.

Внутри цикла указанные в скобках переменные получают значения, возвращаемые функцией `split`, которая “разрезает” входную строку, используя регулярное выражение, переданное ей в качестве разделителя полей. Здесь наш шаблон ограничивается слешами (символами “/”), действие которых подобно двойным кавычкам. Мы могли бы с таким же успехом написать `split ':'` или `split ":",`.

Строка, которую функция `split` должна разбить на подстроки в позициях сопоставления с двоеточиями, нигде явно не задана. Если второй аргумент функции `split`, т.е. подлежащая разбиению строка, опущен, Perl предполагает, что вы хотите разбить на подстроки значение `$_`. Яснее не бывает! По правде говоря, иногда можно опустить даже шаблон — в этом случае разбиение строки производится по пробельному символу, при этом начальные пробелы в каждой подстроке игнорируются.

Но и это еще не все. Даже исходное присваивание переменной `$_` в первой строке цикла необязательно. Если просто написать так:

```
while (<>) {,
```

то Perl автоматически сохранит каждую входную строку в переменной `$_`. Можно обрабатывать строки без создания явной ссылки на переменную, в которой они будут сохраняться. Использование переменной `$_` в качестве операнда, действующего по умолчанию, — распространенная практика, и Perl позволяет такие “вольности” везде, где это имеет смысл.

Во множественном присваивании, которое “захватывает” содержимое каждого поля `passwd`, наличие списка в левой части

```
($name, $pw, $uid, $gid, $gecos, $path, $sh) = split /:/;
```

создает для функции `split` “контекст списка”, что предписывает ей возвращать в результате список всех полей. Если бы в присваивании участвовала скалярная переменная, например

```
$n_fields = split
```

функция `split` “переключилась” бы на “скалярный контекст” и вернула бы только количество обнаруженных полей. Пользовательские функции также могут отличать скаляры от списочного контекста с помощью функции `wantarray`. Она возвращает значение “Истина” в контексте списка, “Ложь” — в контексте скаляра и неопределенное значение — в void-контексте (для пустых типов данных).

Строка

```
%uids_by_name = reverse %names_by_uid;
```

также заслуживает внимания. Хеш в контексте списка (здесь в качестве аргумента функции `reverse`) приводится к списку в форме `(key1, value1, key2, value2, ...)`. Функция `reverse` меняет порядок следования элементов в списке на обратный: `(valueN, keyN, ..., value1, key1)`. Наконец, присваивание хеш-переменной `%uids_by_name` преобразует этот список в вариант `(key1, value1, ...)`, создавая тем самым перестановочный индекс.

Ссылки и их самооживление

Обозначенная в заголовке тема относится к более сложным, но мы посчитали себя не вправе обойти ее молчанием. По крайней мере, мы должны изложить здесь основные положения. Массивы и хеши могут хранить скалярные значения, но не исключено, что вам понадобится сохранить в них другие массивы и хеши. Например, возвращаясь к нашему предыдущему примеру анализа файла `/etc/passwd`, вы могли бы сохранить все поля каждой строки `passwd` в хеше, индексируемом значением идентификатора `UID`.

Если нельзя хранить массивы и хеши, то можно хранить ссылки (указатели) на массивы и хеши, которые сами по себе являются скалярами. Чтобы создать ссылку на массив или хеш, необходимо предварить имя соответствующей переменной обратной косой чертой (например, `\@array`) или использовать литеральный синтаксис ссылок на массив или на хеш. Например, наш цикл `passwd`-анализа мог выглядеть так.

```
while (<>) {  
    $array_ref = [ split /:/ ];  
    $passwd_by_uid{$array_ref->[2]} = $array_ref;  
}
```

Квадратные скобки возвращают ссылку на массив, содержащий результаты работы функции `split`. Выражение `$array_ref->[2]` ссылается на поле `UID` (это третий член массива, адресуемого переменной `$array_ref`).

Выражение `$array_ref[2]` здесь не работает, поскольку мы не определили массив `@array_ref`; `$array_ref` и `@array_ref` — это разные переменные. Хуже того, вы не получите сообщение об ошибке, если ошибочно использовали здесь выражение `$array_ref [2]`, поскольку `@array_ref` — это абсолютно легитимное имя для массива; вы ведь не присваивали ему никаких значений.

Отсутствие предупреждающих сообщений может показаться проблемой, но это, возможно, одно из лучших качеств Perl, именуемое *самооживлением* (autovivification). Поскольку имена переменных и синтаксис использования ссылок всегда делают довольно ясной структуру данных, к которой вы пытаетесь получить доступ, вам не придется создавать промежуточные структуры данных вручную. Достаточно организовать присваивание на самом низком уровне, и промежуточные структуры возникают автоматически. Например, используя только один оператор присваивания, можно создать хеш ссылок на массивы, в качестве содержимого которых будут выступать ссылки на хеши.

Регулярные выражения в Perl

Регулярные выражения в Perl используются путем “привязки” строк к операциям над регулярными выражениями, а именно с помощью оператора связывания `=~`. Например, при выполнении кода

```
if ($text =~ m/ab+c/) {
```

проверяется, совпадает ли строка, хранимая в переменной `$text`, с регулярным выражением `ab+c`. Для выполнения действий по умолчанию над строкой, в которой хранится результат предыдущей операции, т.е. `$_`, можно просто опустить имя переменной и оператор связывания. На самом деле можно опустить и оператор `m`, поскольку эта операция по умолчанию приводит к совпадению.

```
if (/ab+c/) {
```

Подстановки работают аналогично.


```
$text =~ s/etc\.\/and so on/g; # Замена текста в строке $text, ИЛИ
s/etc\.\/and so on/g;         # Применительно к переменной $_
```

Мы вставили здесь модификатор `g`, чтобы заменить не просто первый экземпляр подстроки “etc.” вместе с подстрокой “and so on”, а все найденные экземпляры. Помимо модификатора `g` (который применяется для поиска множественных совпадений), часто используются и другие: модификатор `i`, чтобы игнорировать регистр букв; модификатор `s`, чтобы “нацелить” точку (.) на совпадение с символами новой строки; и модификатор `m`, чтобы с помощью маркеров “^” и “\$” искать совпадения в начале и конце отдельных строк, а не только в начале и конце обрабатываемого текста.

Для того чтобы обратить ваше внимание на другие важные нюансы, рассмотрим следующий сценарий.

```
#!/usr/bin/perl

$names = "huey dewey louie";
$regex = '(\w+) \s+(\w+)\s+(\w+)';

if ($names =~ m/$regex/) {
    print "1-ое имя $1.\n2-ое имя $2.\n3-е имя $3.\n";
    $names =~ s/$regex/\2 \1/;
    print "Новые имена \"${names}\".\n";
} else {
    print qq{ "$ names" не совпадает с "$regex". \n};
}
```

Вот как выглядит результат выполнения этого сценария.

```
$ perl testregex
1-ое имя huey.
2-ое имя dewey.
3-е имя louie.
Новые имена "dewey huey".
```

Этот пример показывает, что символы `//` позволяют раскрыть переменные, т.е. регулярное выражение обязательно должно быть фиксированной строкой. Обратите внимание на оператор `qq` — это еще одно имя для оператора заключения в двойные кавычки.

После реализации совпадения или подстановки переменные `$1`, `$2` и так далее соотносятся с содержимым круглых скобок в регулярном выражении. Содержимое этих переменных также доступно в самом процессе замены, в контексте которого они “именуются” как `\1`, `\2` и т.д.

Ввод и вывод данных

Открывая файл для чтения или записи, вы определяете так называемый *дескриптор файла*, чтобы идентифицировать соответствующий канал. В приведенном ниже примере переменная `INFILE` служит дескриптором для файла `/etc/passwd`, а переменная `OUTFILE` — дескриптором, связанным с файлом `/tmp/passwd`. Используемое в цикле `while` условие `<INFILE>` аналогично рассмотренному выше выражению `<>`, но имеет свою специфику, связанную с данным конкретным дескриптором файла. Здесь считываются строки из файла с дескриптором `INFILE` до тех пор, пока не обнаружится конец файла, что и остановит работу цикла `while`. Каждая строка размещается в переменной `$_`.

```
#!/usr/bin/perl

open(INFILE, "</etc/passwd") or die "Не удается открыть /etc/passwd";
open(OUTFILE, ">/tmp/passwd") or die "Не удается открыть /tmp/passwd";

while (<INFILE>) {
    ($name, $pw, $uid, $gid, $gecos, $path, $sh) = split /:/;
    print OUTFILE "$uid\t$name\n";
}
```

Функция `open` возвратит значение “Истина”, если файл успешно откроется, тем самым “замкнув” (т.е. сделав необязательным) вычисление частей `die`, поскольку результат вычисления всего выражения уже не изменится. Perl-оператор `or` подобен оператору `||` (который также есть в Perl), но с более низким приоритетом. Оператор `or` в общем случае предпочтительнее, если вы хотите подчеркнуть, что прежде, чем Perl переключит “свое внимание” на последствия неудачного исхода операции, должны быть выполнены все действия, указанные в левой части выражения.

Синтаксис Perl для задания способа использования каждого файла (для чтения, записи или добавления в конец) отражает возможности, заложенные в командной оболочке. Кроме того, использование таких имен файлов, как `"/bin/df|"`, позволит открывать входные или выходные каналы связи с командами оболочки.

Поток управления

В приведенном ниже примере представлена Perl-версия уже знакомого вам `bash`-сценария, в котором проверялась достоверность аргументов командной строки (для сравнения можете вернуться к разделу “Функции и аргументы командной строки” этой главы). Обратите внимание на то, что Perl-конструкция `if` не оснащена ни ключевым словом `then`, ни каким-либо признаком завершения — блок операторов здесь просто заключен в фигурные скобки.

Для того чтобы обеспечить выполнение любого отдельного оператора при соблюдении некоторого условия, вы можете добавлять постфиксный вариант оператора `if` (или его обратную версию `unless`).

```
#!/usr/bin/perl

sub show_usage {
    print shift, "\n" if scalar(@_);
    print "Применение: $0 source_dir dest_dir\n";
    exit scalar(@_) ? shift : 1;
}

if (@ARGV != 2) {
    show_usage;
} else { # Существует два аргумента
    ($source_dir, $dest_dir) = @ARGV;
    show_usage "Недопустимый каталог-источник" unless -d $source_dir;
    -d $dest_dir or show_usage "Недопустимый каталог-приемник";
}
```

Здесь две строки, которые используют унарный Perl-оператор `-d` для проверки существования каталогов `$source_dir` и `$dest_dir`, эквивалентны. Вторая форма (с оператором `-d` в начале строки) предпочтительнее, поскольку утверждение используется в начале строки, где оно заметнее всего. Однако применение оператора `or`, обозначающего “в противном случае”, несколько сомнительно; и многих, кому приходится разбираться в чужом коде, это сбивает с толку.

Переменная массива в скалярном контексте (заданном оператором `scalar` в данном примере) возвращает количество элементов в массиве. Это значение на единицу превышает значение `$#array` (как всегда, в Perl предусмотрено несколько способов получения одного и того же результата).

Функции в Perl принимают аргументы в массив с именем `@_`. Обычной практикой считается в Perl получать доступ к ним с помощью оператора `shift`, который удаляет первый элемент массива аргументов и возвращает его значение.

Эта версия функции `show_usage` принимает необязательное сообщение об ошибке для вывода на печать. Если вы предоставляете сообщение об ошибке, то можете также предоставить специфический код выхода. Тернарный оператор `?:` оценивает свой первый аргумент; если оценка даст значение “Истина”, то результатом всего выражения станет значение второго аргумента, в противном случае — третьего.

Как и в среде `bash`, Perl имеет специальное условие “else if”, но его ключевым словом является `elsif`, а не `elif`. (Для тех, кто использует оба языка, эти забавные пустяковые различия либо заострят их ум, либо — наоборот.)

Как показано в табл. 2.5, Perl-операторы сравнения не совпадают с аналогичными операторами в оболочке `bash`: строки используют текстуальные операторы, а числа — традиционную алгебраическую запись. Сравните эту таблицу с табл. 2.2.

Таблица 2.5. Элементарные Perl-операторы сравнения

Строки	Числа	Истина, если
<code>x eq y</code>	<code>x = y</code>	x равно y
<code>x ne y</code>	<code>x != y</code>	x не равно y
<code>x lt y</code>	<code>x < y</code>	x меньше y
<code>x le y</code>	<code>x <= y</code>	x меньше или равно y
<code>x gt y</code>	<code>x > y</code>	x больше y
<code>x ge y</code>	<code>x >= y</code>	x больше или равно y

В Perl у вас есть все операторы тестирования файлов, показанные в табл. 2.3, за исключением операторов `-nt` и `-ot`, которые доступны только в среде `bash`.

Подобно оболочке `bash`, язык Perl предоставляет два типа цикла `for`. При использовании более распространенной формы цикла `for` обеспечивается выполнение итераций с проходом по всем элементам явно заданного списка аргументов. Например, в приведенном ниже коде перебираются по очереди элементы списка названий животных (`animals`) и выводятся по одному на строке.

```
@animals = qw(lions tigers bears);
foreach $animal (@animals) {
    print "$animal \n" ;
}
```

Также вы вправе использовать цикл `for` в стиле языка C.

```
for ($counter=1; $counter <= 10; $counter++) {
    printf "$counter ";
```

Хотя мы продемонстрировали применение циклов с разными ключевыми словами — `for` и `foreach`, на самом деле в обоих случаях работает одно и то же ключевое слово, т.е. в Perl вы можете использовать любую форму, которая кажется вам предпочтительнее.

В версиях языка Perl до 5.10 (2007 г.) не было явно определенных операторов `case` или `switch`, но, как уже подчеркивалось не раз, один результат можно получить несколькими путями. В дополнение к очевидному, но громоздкому варианту каскадирования операторов `if`, можно прибегнуть к еще одному способу использования оператора `for`, в котором главное — установить переменную `$_`, сравнить ее с нужными значениями и после выполнения соответствующих действий обеспечить выход из контекста (с помощью оператора `last`).

```
for ($ARGV[0]) {

    m/^websphere/  && do {print "Инсталляция для websphere\n"; last; };
    m/^tomcat/     && do {print "Инсталляция для tomcat\n" ; last; };
    m/^geronimo/   && do {print "Инсталляция для geronimo\n"; last; };

    print "Предоставлен недопустимый вариант.\n"; exit 1;
}
```

Здесь регулярные выражения по очереди сравниваются с аргументом, сохраненным в переменной `$_`. При неуспешном результате сравнения в левой части операции `&&` правая часть уже не вычисляется и управление передается следующему оператору. Если же происходит совпадение с некоторым регулярным выражением, выполняется соответствующий ему блок `do`, после чего остается лишь “красиво уйти”. Немедленный выход из блока `for` обеспечивают операторы `last`.

Прием входных данных и проверка их достоверности

В приведенном ниже сценарии объединены многие уже рассмотренные нами конструкции Perl, включая использование подпрограммы, нескольких постфиксных операторов `if` и цикла `for`. Сама программа — это просто “обертка”, в которую “завернута” основная функция `get_string`, обеспечивающая проверку достоверности входных данных. Эта функция приглашает пользователя ввести строку, удаляет замыкающий символ новой строки и проверяет, не является ли введенная строка нулевой. После приема нулевой строки пользователю предлагается повторить ввод, но после трех неудачных попыток сценарий “умоет руки”.

```
#!/usr/bin/perl

$maxatt = 3; # Максимальное число попыток ввести корректные данные

sub get_string {
    my ($prompt, $response) = shift;
    # Пытаемся прочитать входную строку до $maxatt раз
    for (my $attempts = 0; $attempts < $maxatt; $attempts++) {
        print "Пожалуйста, попробуйте еще раз.\n" if $attempts;
        print "$prompt: ";
        $response = readline(*STDIN);
        chomp($response);
        return $response if $response;
    }
    die "Слишком много неудачных попыток ввести данные";
}

# Прием имен с помощью функции get_string и перевод в прописные буквы
$fname = uc get_string "Имя";
```